CM3015 Project – Handwritten Digits

This notebook compares a few ML approaches on the scikit-learn Digits dataset.

- **From scratch (NumPy):** kNN, Linear regression, Polynomial Regression
- **Using libraries:** PCA, Decision Tree

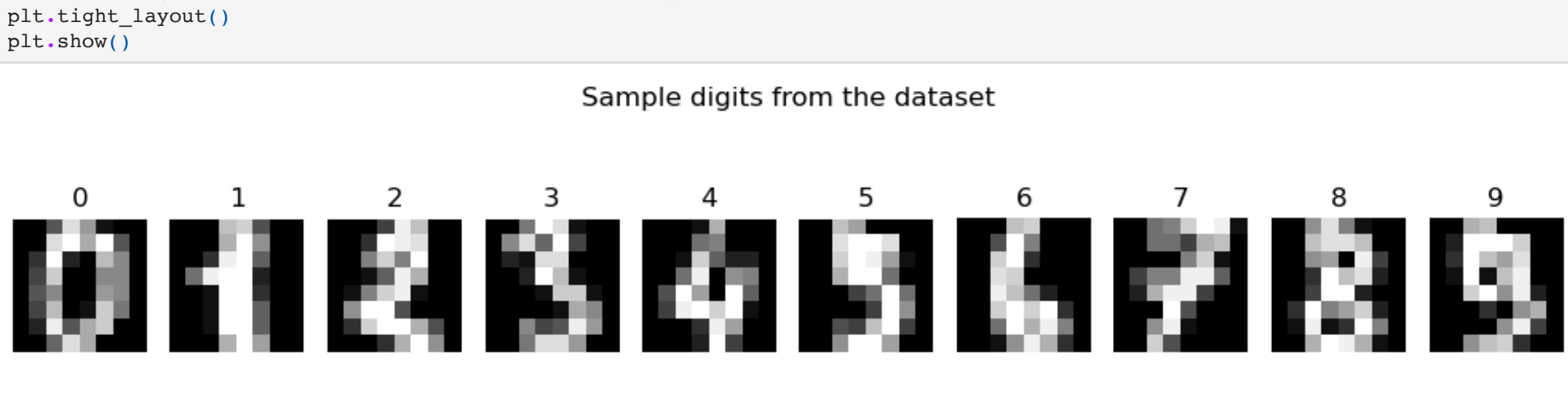
In [1]:
```python
# ----------------------------
# Cell goal:
# Set up all the libraries we'll use, and lock down randomness so results stay the same each run.
# ----------------------------

import numpy as np                      # NumPy = fast math with arrays (vectors/matrices), used for our "from scratch" parts
import matplotlib.pyplot as plt         # Matplotlib = plotting library for graphs and images
import time                             # time = used to measure how long training/prediction takes

from sklearn.datasets import load_digits            # Built-in handwritten digits dataset (8x8 images of 0–9)
from sklearn.model_selection import train_test_split # Utility to split data into training and testing sets
from sklearn.preprocessing import StandardScaler     # Standardizes features (mean=0, std=1) so models behave better
from sklearn.decomposition import PCA                # PCA = reduces feature dimensions while keeping most information
from sklearn.tree import DecisionTreeClassifier      # A decision tree classifier (library baseline model)

# One place to control randomness (repeatable runs)
RANDOM_STATE = 42                 # Any "random" operation (like splitting data) will use this number
np.random.seed(RANDOM_STATE)      # Makes NumPy's random results repeatable (important for fair comparisons)
```

Load dataset and quick look

In [2]:
```python
# ----------------------------
# Cell purpose:
# Load the handwritten digits dataset and examine its structure
# to understand the shape and contents of the data.
# ----------------------------

digits = load_digits()
# Loads the digits dataset provided by scikit-learn.
# The dataset contains pixel data, labels, and image representations.

X = digits.data
# Stores the input features.
# Each image is flattened into a 1D vector of length 64 (8 × 8 pixels).
# Shape: (1797 samples, 64 features)

y = digits.target
# Stores the target labels.
# Each value represents the digit class (0 through 9).
# Shape: (1797,)

images = digits.images
# Stores the original image format.
# Each digit is represented as an 8 × 8 grayscale image.
# Shape: (1797, 8, 8)

print("X shape:", X.shape)
# Displays the number of samples and features in the dataset

print("y shape:", y.shape)
# Displays the number of labels and confirms it matches the number of samples

print("Unique labels:", np.unique(y))
# Displays all distinct digit classes present in the dataset
# Confirms this is a 10-class classification problem
```

```
X shape: (1797, 64)
y shape: (1797,)
Unique labels: [0 1 2 3 4 5 6 7 8 9]
```

In [3]:
```python
# A quick visual check: first 10 digits in the dataset
plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(str(y[i]))
    plt.axis("off")
plt.suptitle("Sample digits from the dataset", y=1.05)
plt.tight_layout()
plt.show()
```


Sample digits from the dataset
0
1
2
3
4
5
6
7
8
9

Train/test split + scaling

In [4]:
```python
# Stratified split keeps class balance similar in train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
# Splits the dataset into training and testing subsets.
# test_size=0.2 means 20% of the data is reserved for testing.
# random_state ensures the same split is produced every run.
# stratify=y keeps the proportion of each digit class (0–9) similar in both sets,
# which prevents one set from accidentally having far fewer examples of a class.

# Scaling matters for distance-based methods (kNN / k-Means)
scaler = StandardScaler()
# Creates a scaler that standardizes each feature:
# - subtracts the training mean
# - divides by the training standard deviation
# This makes all features comparable on the same scale.

X_train_scaled = scaler.fit_transform(X_train).astype(np.float64)
# fit_transform learns scaling parameters (mean/std) from the training set,
# then applies the scaling to the training data.
# astype(np.float64) ensures consistent numeric precision for custom NumPy code.

X_test_scaled = scaler.transform(X_test).astype(np.float64)
# transform applies the SAME scaling learned from the training set to the test set.
# Using fit_transform on the test set would cause data leakage by using test statistics.

print("Train size:", X_train_scaled.shape[0])
# Displays the number of training samples after the split

print("Test size :", X_test_scaled.shape[0])
# Displays the number of testing samples after the split
```

```
Train size: 1437
Test size : 360
```

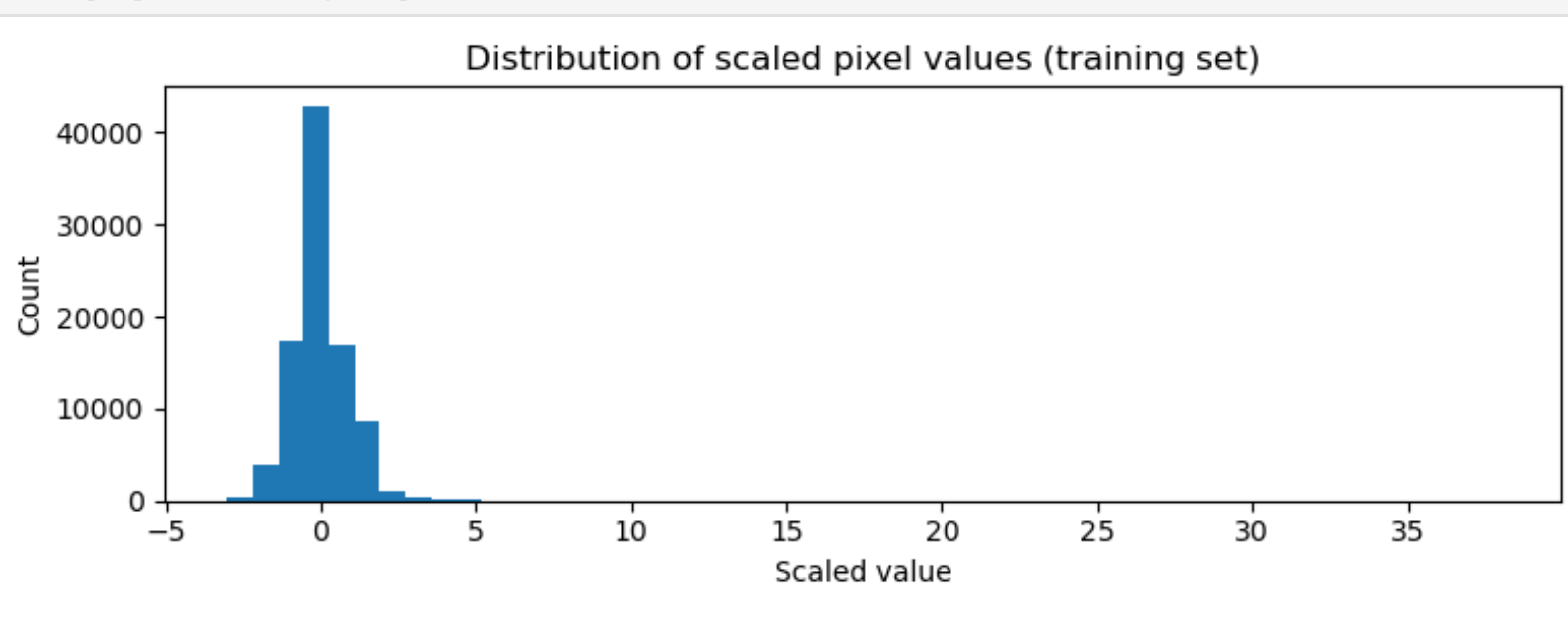
In [5]:
```python
# ----------------------------
# Cell purpose:
# Perform basic sanity checks on the dataset to ensure data quality
# before training any models.
# ----------------------------

print("Any NaNs in X?", np.isnan(X).any())
# Checks whether any missing values (NaN) exist in the feature matrix.
# A result of False confirms all pixel values are valid numbers.

print("Any NaNs in y?", np.isnan(y).any())
# Checks whether any missing values exist in the label array.
# Labels must be complete for supervised learning.

# Pixel values in this dataset are small integers (0..16 usually). Confirming the range.
print("Raw X min/max:", X.min(), X.max())
# Displays the minimum and maximum pixel values in the dataset.
# This helps verify that the data matches the expected grayscale intensity range.

# Class balance check (not strictly preprocessing, but good to know)
unique, counts = np.unique(y, return_counts=True)
# Extracts each unique digit label and counts how many times it appears.

print("Class counts:", dict(zip(unique, counts)))
# Displays the number of samples for each digit class.
# This confirms whether the dataset is reasonably balanced across classes.
```

```
Any NaNs in X? False
Any NaNs in y? False
Raw X min/max: 0.0 16.0
Class counts: {0: 178, 1: 182, 2: 177, 3: 183, 4: 181, 5: 182, 6: 181, 7: 179, 8: 174, 9: 180}
```

In [6]:
```python
# ----------------------------
# Cell purpose:
# Ensure that all scaled feature data uses a consistent numeric type
# to prevent unexpected issues during NumPy-based computations.
# ----------------------------

# NumPy sometimes keeps this as float already, but the type is explicitly enforced
# to avoid unexpected dtype-related problems in later calculations.
X_train_scaled = X_train_scaled.astype(np.float64)
# Converts the training feature matrix to 64-bit floating-point format.
# This is important for numerical stability and for custom gradient-based algorithms.

X_test_scaled = X_test_scaled.astype(np.float64)
# Converts the testing feature matrix to the same numeric type as the training data.
# Keeping train and test data in the same format avoids subtle computation errors.

print(X_train_scaled.dtype)
# Confirms the data type of the training features after conversion

print(X_test_scaled.dtype)
# Confirms the data type of the testing features after conversion
```

```
float64
float64
```

In [7]:
```python
# ----------------------------
# Cell purpose:
# Visualize the distribution of scaled feature values to verify that
# standardization worked as expected.
# ----------------------------

# Just checking the scaled features distribution.
# After StandardScaler, values are usually centered around 0.
plt.figure(figsize=(8, 3))
# Creates a figure with a wide, short layout for better readability.

plt.hist(X_train_scaled.ravel(), bins=50)
# Flattens the 2D training feature array into 1D using ravel().
# A histogram is plotted to show how scaled pixel values are distributed.
# Using many bins helps reveal the overall shape of the distribution.

plt.title("Distribution of scaled pixel values (training set)")
# Adds a descriptive title to explain what the plot represents.

plt.xlabel("Scaled value")
# Labels the x-axis to indicate standardized pixel values.

plt.ylabel("Count")
# Labels the y-axis to indicate how many values fall into each bin.

plt.tight_layout()
# Automatically adjusts spacing to prevent label and title overlap.

plt.show()
# Displays the histogram plot.
```



This histogram visualizes the distribution of all pixel values in the training set after standardization using StandardScaler. The x-axis ("Scaled value") shows pixel intensities after scaling. The y-axis ("Count") shows how frequently those scaled values occur. The data plotted comes from X_train_scaled.ravel(), meaning: all pixels from all training images are flattened into a single 1D array, the histogram therefore reflects the global pixel-value distribution of the training data. Pixel features have zero mean after scaling, ensuring no feature dominates due to offset alone. This histogram confirms that standardization was applied correctly, producing pixel values centered around zero with comparable variance, which is essential for reliable PCA projection and distance-based kNN classification.

NumPy-only evaluation metrics (no sklearn.metrics)

In [8]:
```python
# ----------------------------
# Cell purpose:
# Define evaluation functions using only NumPy.
# These functions compute accuracy, confusion matrix,
# and macro-averaged precision, recall, and F1-score.
# ----------------------------

def accuracy_numpy(y_true, y_pred):
    # Compares true labels with predicted labels element by element.
    # The result is a boolean array where True means a correct prediction.
    # Taking the mean converts True/False into 1/0 and produces accuracy.
    return float(np.mean(y_true == y_pred))


def confusion_matrix_numpy(y_true, y_pred, n_classes):
    # Creates an empty confusion matrix of shape (n_classes × n_classes).
    # Rows represent true classes, columns represent predicted classes.
    cm = np.zeros((n_classes, n_classes), dtype=int)

    # Iterates through each true label and corresponding predicted label.
    for t, p in zip(y_true, y_pred):
        # Increments the cell that matches the true class and predicted class.
        # int() ensures the indices are valid integers.
        cm[int(t), int(p)] += 1

    # Returns the filled confusion matrix.
    return cm


def precision_recall_f1_macro_numpy(y_true, y_pred, n_classes):
    # Macro averaging treats all classes equally,
    # regardless of how many samples each class has.

    # First compute the confusion matrix.
    cm = confusion_matrix_numpy(y_true, y_pred, n_classes)

    precisions, recalls, f1s = [], [], []
    # Lists are used to store per-class metric values
    # before computing the macro (mean) score.

    for c in range(n_classes):
        # True positives: correct predictions for class c.
        tp = cm[c, c]

        # False positives: predicted as class c but actually belong to another class.
        fp = cm[:, c].sum() - tp

        # False negatives: actually class c but predicted as another class.
        fn = cm[c, :].sum() - tp

        # Precision = TP / (TP + FP)
        # A safety check avoids division by zero.
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0

        # Recall = TP / (TP + FN)
        # A safety check avoids division by zero.
        rec = tp / (tp + fn) if (tp + fn) > 0 else 0.0

        # F1-score = harmonic mean of precision and recall.
        # A safety check avoids division by zero.
        f1 = (2 * prec * rec / (prec + rec)) if (prec + rec) > 0 else 0.0

        # Store per-class metrics.
        precisions.append(prec)
        recalls.append(rec)
        f1s.append(f1)

    # Compute macro averages by taking the mean across classes.
    return float(np.mean(precisions)), float(np.mean(recalls)), float(np.mean(f1s))


def print_metrics_block(title, y_true, y_pred, n_classes=10):
    # Computes accuracy using the custom NumPy-based function.
    acc = accuracy_numpy(y_true, y_pred)

    # Computes macro-averaged precision, recall, and F1-score.
    p, r, f1 = precision_recall_f1_macro_numpy(y_true, y_pred, n_classes)

    # Prints a formatted block of evaluation results.
    print("=" * 58)
    print(title)
    print("=" * 58)
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {p:.4f}  (macro)")
    print(f"Recall   : {r:.4f}  (macro)")
    print(f"F1-score : {f1:.4f}  (macro)")
    print("=" * 58)

    # Returns the metrics for optional later use or comparison.
    return acc, p, r, f1
```

In [9]:
```python
# ----------------------------
# Cell purpose:
# Provide advanced confusion-matrix tools:
# - build a confusion matrix with NumPy only
# - plot it as counts or as row-normalized values (recall view)
# - compute TP/FP/FN/TN per class using a one-vs-rest interpretation
# ----------------------------

def confusion_matrix_numpy(y_true, y_pred, n_classes=10):
    # Converts inputs into NumPy arrays, forces integer labels, and flattens to 1D.
    # ravel() removes any extra dimensions (e.g., shape (N,1) becomes (N,)).
    y_true = np.asarray(y_true).astype(int).ravel()
    y_pred = np.asarray(y_pred).astype(int).ravel()

    # Initializes an empty confusion matrix.
    # Rows = true class, columns = predicted class.
    cm = np.zeros((n_classes, n_classes), dtype=int)

    # Steps through each true/predicted pair and increments the corresponding cell.
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1

    # Returns the filled confusion matrix.
    return cm

def plot_confusion_matrix_individual(cm, title, normalize=None, annotate=True):
    """
    normalize:
      None      -> raw counts (how many samples fall in each cell)
      'true'    -> row-normalized so each row sums to 1 (recall-style view)

    annotate:
      True      -> writes values in the cells for readability
    """
    # Converts the matrix to float so division works cleanly during normalization.
    cm = np.asarray(cm, dtype=np.float64)

    if normalize == "true":
        # Computes the sum of each row (total samples per true class).
        # keepdims=True keeps the shape as (C,1) so broadcasting works correctly.
        row_sums = cm.sum(axis=1, keepdims=True)

        # Prevents division by zero for any class that has zero samples.
        row_sums[row_sums == 0] = 1.0

        # Divides each row by its sum → each row becomes proportions adding up to 1.
        # This makes it easier to see recall-related behavior per class.
        cm_show = cm / row_sums
        subtitle = " (Normalized by True / Recall view)"
    else:
        # Uses raw counts when normalization is not requested.
        cm_show = cm
        subtitle = " (Counts)"

    plt.figure(figsize=(7, 6))
    # Creates a new figure so the plot does not overwrite previous plots.

    plt.imshow(cm_show, cmap="Blues")
    # Displays the matrix as an image, where darker color usually means larger values.

    plt.title(title + subtitle)
    # Adds a title that also indicates whether normalization is active.

    plt.xlabel("Predicted label")
    # X-axis represents predicted classes (columns).

    plt.ylabel("True label")
    # Y-axis represents true classes (rows).

    plt.colorbar()
    # Adds a color scale legend to interpret the heatmap intensities.

    plt.xticks(range(cm.shape[0]))
    plt.yticks(range(cm.shape[0]))
    # Places tick marks for each class index along both axes.

    if annotate:
        # Writes the number (or proportion) inside each cell.
        for i in range(cm.shape[0]):
            for j in range(cm.shape[1]):
                if normalize == "true":
                    # For normalized view, show decimals with two places.
                    text = f"{cm_show[i, j]:.2f}"
                else:
                    # For counts view, show integer counts from the original matrix.
                    text = f"{int(cm[i, j])}"
                plt.text(j, i, text, ha="center", va="center", fontsize=8)
                # Places text at column j, row i; centered inside the cell.

    plt.tight_layout()
    # Adjusts spacing so labels and title are not cut off.

    plt.show()
    # Renders the final plot.

def tp_fp_fn_tn_from_cm(cm):
    """
    Multiclass TP/FP/FN/TN per class using one-vs-rest interpretation.

    For each class c:
      TP = cm[c, c]
      FP = sum of column c minus TP  (predicted c but actually not c)
      FN = sum of row c minus TP     (true c but predicted not c)
      TN = total - (TP + FP + FN)    (all remaining cases)
    """
    # Ensures the confusion matrix is an integer NumPy array.
    cm = np.asarray(cm, dtype=np.int64)

    # Total number of samples represented by the matrix.
    total = cm.sum()

    # True positives for each class are on the diagonal.
    TP = np.diag(cm)

    # False positives: everything predicted as class c (column sum) minus true positives.
    FP = cm.sum(axis=0) - TP

    # False negatives: everything actually class c (row sum) minus true positives.
    FN = cm.sum(axis=1) - TP

    # True negatives: everything not counted in TP/FP/FN for that class.
    TN = total - (TP + FP + FN)

    return TP, FP, FN, TN

def precision_recall_f1_from_tp_fp_fn(TP, FP, FN, eps=1e-12):
    # eps is added to denominators to prevent division by zero.
    # The value is very small so it does not meaningfully change real results.

    precision = TP / (TP + FP + eps)
    # Precision answers: "When the model predicts class c, how often is it correct?"

    recall    = TP / (TP + FN + eps)
    # Recall answers: "Out of all true class c samples, how many were found?"

    f1        = 2 * precision * recall / (precision + recall + eps)
    # F1-score is the harmonic mean of precision and recall,
    # balancing both into a single value.

    return precision, recall, f1

def deep_eval_from_confusion_matrix(cm, model_name, top_k_confusions=8):
    """
    Prints a detailed evaluation report for one model based on its confusion matrix.

    Includes:
    - accuracy calculation from the matrix
    - macro averages (each class weighted equally)
    - weighted F1 (weighted by class support)
    - TP/FP/FN/TN per class (one-vs-rest explanation)
    - worst classes by recall and precision
    - most common misclassification pairs (True → Predicted)
    - ready-to-use phrasing ideas for a written report
    """
    # Converts input confusion matrix to integer type to match count interpretation.
    cm = np.asarray(cm, dtype=np.int64)

    # Number of classes equals the matrix size (C x C).
    C = cm.shape[0]

    # Total number of samples accounted for by the confusion matrix.
    total = int(cm.sum())

    # Correct predictions are the diagonal sum.
    correct = int(np.trace(cm))

    # Accuracy computed from confusion matrix counts.
    accuracy = correct / total if total > 0 else 0.0

    TP, FP, FN, TN = tp_fp_fn_tn_from_cm(cm)
    # Converts the multiclass confusion matrix into per-class TP/FP/FN/TN counts.

    precision, recall, f1 = precision_recall_f1_from_tp_fp_fn(TP, FP, FN)
    # Produces per-class precision/recall/F1 arrays.

    support = cm.sum(axis=1)
    # Support for class c = number of true samples of class c (row sum).

    macro_p = float(np.mean(precision))
    macro_r = float(np.mean(recall))
    macro_f1 = float(np.mean(f1))
    # Macro averages treat each class equally by taking the mean across classes.

    weighted_f1 = float(np.sum(f1 * support) / (np.sum(support) + 1e-12))
    # Weighted F1 gives more influence to classes that appear more often.

    print("\n" + "="*95)
    print(f"DETAILED CONFUSION MATRIX EVALUATION — {model_name}")
    print("="*95)

    print("Interpretation (one-vs-rest for each class c):")
    print("  TP(c): predicted c AND true c (diagonal cell for c)")
    print("  FP(c): predicted c BUT true was not c (other rows in column c)")
    print("  FN(c): true c BUT predicted not c (other columns in row c)")
    print("  TN(c): not c AND predicted not c (everything not in row/col c)")
    # This section explains how TP/FP/FN/TN are defined in multiclass settings.
    # Each class is treated as "positive" while all other classes become "negative."

    print("-"*95)

    print(f"Overall Accuracy = (sum diagonal) / total = {correct}/{total} = {accuracy:.4f}")
    # Accuracy from confusion matrix: correct predictions divided by total samples.

    print(f"Macro avg (P/R/F1): {macro_p:.4f} / {macro_r:.4f} / {macro_f1:.4f}")
    # Macro averages highlight performance consistency across classes.

    print(f"Weighted F1       : {weighted_f1:.4f}")
    # Weighted F1 reflects performance while accounting for class frequency.

    print("-"*95)

    print("Class |   TP   FP   FN   TN | Precision  Recall    F1   | Support")
    print("-"*95)
    # Prints a table of per-class diagnostic counts and metrics.

    for c in range(C):
        print(f"{c:>5} | {TP[c]:>4} {FP[c]:>4} {FN[c]:>4} {TN[c]:>4} |"
              f" {precision[c]:>9.3f} {recall[c]:>7.3f} {f1[c]:>7.3f} | {support[c]:>7}")
        # TP/FP/FN/TN explain why precision/recall/F1 values happen for each class.

    # Worst classes by recall: smallest recall means many false negatives.
    worst_recall = np.argsort(recall)[:3]

    # Worst classes by precision: smallest precision means many false positives.
    worst_prec = np.argsort(precision)[:3]

    print("-"*95)
    print("Worst classes by Recall (high FN → true class often missed):")
    for c in worst_recall:
        print(f"  Class {c}: Recall={recall[c]:.3f} | FN={FN[c]} | Support={support[c]}")
        # Low recall often happens when the model confuses that class with others.

    print("Worst classes by Precision (high FP → many false alarms for that predicted class):")
    for c in worst_prec:
        predicted_as_c = cm[:, c].sum()
        # Total predictions made as class c equals the column sum for c.
        print(f"  Class {c}: Precision={precision[c]:.3f} | FP={FP[c]} | Predicted={predicted_as_c}")
        # Low precision often means the model assigns this label too easily.

    # Extract most common off-diagonal confusions (true i predicted as j).
    cm_off = cm.copy()
    # Creates a copy so the diagonal can be changed without affecting the original.

    np.fill_diagonal(cm_off, 0)
    # Sets diagonal to zero so only mistakes remain.

    pairs = []
    # Collects misclassification pairs as (count, true_class, predicted_class).

    for i in range(C):
        for j in range(C):
            if i != j and cm_off[i, j] > 0:
                pairs.append((int(cm_off[i, j]), i, j))
                # Adds only non-zero confusions to keep the list smaller.

    pairs.sort(reverse=True, key=lambda x: x[0])
    # Sorts confusions by count, highest first.

    print("-"*95)
    print(f"Top {min(top_k_confusions, len(pairs))} confusions (True → Predicted):")
    for cnt, i, j in pairs[:top_k_confusions]:
        frac = cnt / (support[i] + 1e-12)
        # Fraction shows how much of true class i is being misclassified as j.
        print(f"  {i} → {j}: {cnt} cases ({frac:.2%} of true class {i})")
```

In [10]:
```python
# ==========================================================
# METRICS UTILITIES (NumPy-only)
# ==========================================================
# Purpose:
# Define reusable evaluation functions without using sklearn.metrics.
# These utilities calculate:
# - confusion matrix (counts of true vs predicted classes)
# - accuracy (overall correctness)
# - macro precision/recall/F1 (each class weighted equally)
# - a simple per-class "classification report" style printout
# ==========================================================

def confusion_matrix_numpy(y_true, y_pred, n_classes=10):
    # Creates an empty confusion matrix with integer counts.
    # Shape: (n_classes, n_classes)
    # Rows represent true labels, columns represent predicted labels.
    cm = np.zeros((n_classes, n_classes), dtype=int)

    # Loops through each true/predicted label pair.
    for t, p in zip(y_true, y_pred):
        # int(...) ensures valid integer indexing even if inputs are float-like.
        # Increments the cell corresponding to (true label, predicted label).
        cm[int(t), int(p)] += 1

    # Returns the completed confusion matrix.
    return cm

def accuracy_numpy(y_true, y_pred):
    # Converts inputs to NumPy arrays for consistent element-wise comparison.
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    # Compares arrays element by element and returns the fraction that match.
    # True/False values are treated as 1/0 when averaged.
    return float(np.mean(y_true == y_pred))

def precision_recall_f1_macro_numpy(y_true, y_pred, n_classes=10):
    # Builds the confusion matrix first, since TP/FP/FN are derived from it.
    cm = confusion_matrix_numpy(y_true, y_pred, n_classes)

    # These lists store per-class values before macro averaging.
    precisions, recalls, f1s = [], [], []

    # Iterates over each class index.
    for c in range(n_classes):
        # True positives for class c are located on the diagonal.
        tp = cm[c, c]

        # False positives for class c:
        # everything predicted as c (column sum) minus correct predictions (tp).
        fp = cm[:, c].sum() - tp

        # False negatives for class c:
        # everything truly c (row sum) minus correct predictions (tp).
        fn = cm[c, :].sum() - tp

        # Precision = TP / (TP + FP)
        # If (TP + FP) is zero, precision is defined as 0.0 to avoid division by zero.
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0

        # Recall = TP / (TP + FN)
        # If (TP + FN) is zero, recall is defined as 0.0 to avoid division by zero.
        rec  = tp / (tp + fn) if (tp + fn) > 0 else 0.0

        # F1-score = harmonic mean of precision and recall.
        # If precision + recall is zero, F1 is defined as 0.0 to avoid division by zero.
        f1   = (2 * prec * rec / (prec + rec)) if (prec + rec) > 0 else 0.0

        # Stores per-class metrics for later averaging.
        precisions.append(prec)
        recalls.append(rec)
        f1s.append(f1)

    # Macro average treats every class equally by averaging across all classes.
    return float(np.mean(precisions)), float(np.mean(recalls)), float(np.mean(f1s))

def class_report_numpy(y_true, y_pred, n_classes=10):
    # Builds the confusion matrix once and reuses it for all class calculations.
    cm = confusion_matrix_numpy(y_true, y_pred, n_classes)

    # Prints a header similar to a simplified classification report.
    print("Class | Precision | Recall | F1-score | Support")
    print("-" * 55)

    # Computes and prints metrics for each class separately.
    for c in range(n_classes):
        # Diagonal element: correctly predicted samples for class c.
        tp = cm[c, c]

        # False positives: predicted as c but actually not c.
        fp = cm[:, c].sum() - tp

        # False negatives: actually c but predicted as something else.
        fn = cm[c, :].sum() - tp

        # Support: total true samples belonging to class c (row sum).
        support = cm[c, :].sum()

        # Precision for class c.
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0

        # Recall for class c.
        rec  = tp / (tp + fn) if (tp + fn) > 0 else 0.0

        # F1-score for class c.
        f1   = (2 * prec * rec / (prec + rec)) if (prec + rec) > 0 else 0.0

        # Prints formatted per-class values with fixed decimal places.
        print(f"{c:>5} | {prec:>9.4f} | {rec:>6.4f} | {f1:>7.4f} | {support:>7}")

    # Overall accuracy across all classes.
    acc = accuracy_numpy(y_true, y_pred)

    # Macro averages across classes (all classes weighted equally).
    p_macro, r_macro, f1_macro = precision_recall_f1_macro_numpy(y_true, y_pred, n_classes)

    print("-" * 55)
    # Prints overall summary metrics for quick comparison across models.
    print(f"Accuracy  : {acc:.4f}")
    print(f"Macro avg : Precision={p_macro:.4f}, Recall={r_macro:.4f}, F1={f1_macro:.4f}")
```

Algorithm 1 — kNN (from scratch, NumPy)

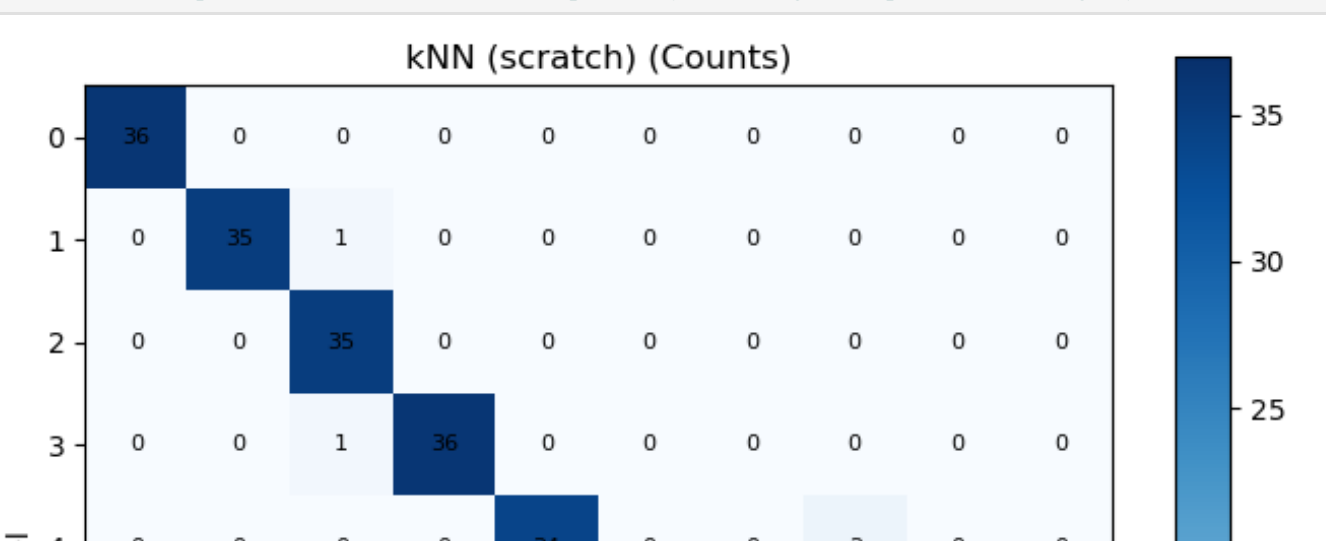
Implementation: Euclidean distance + majority vote.

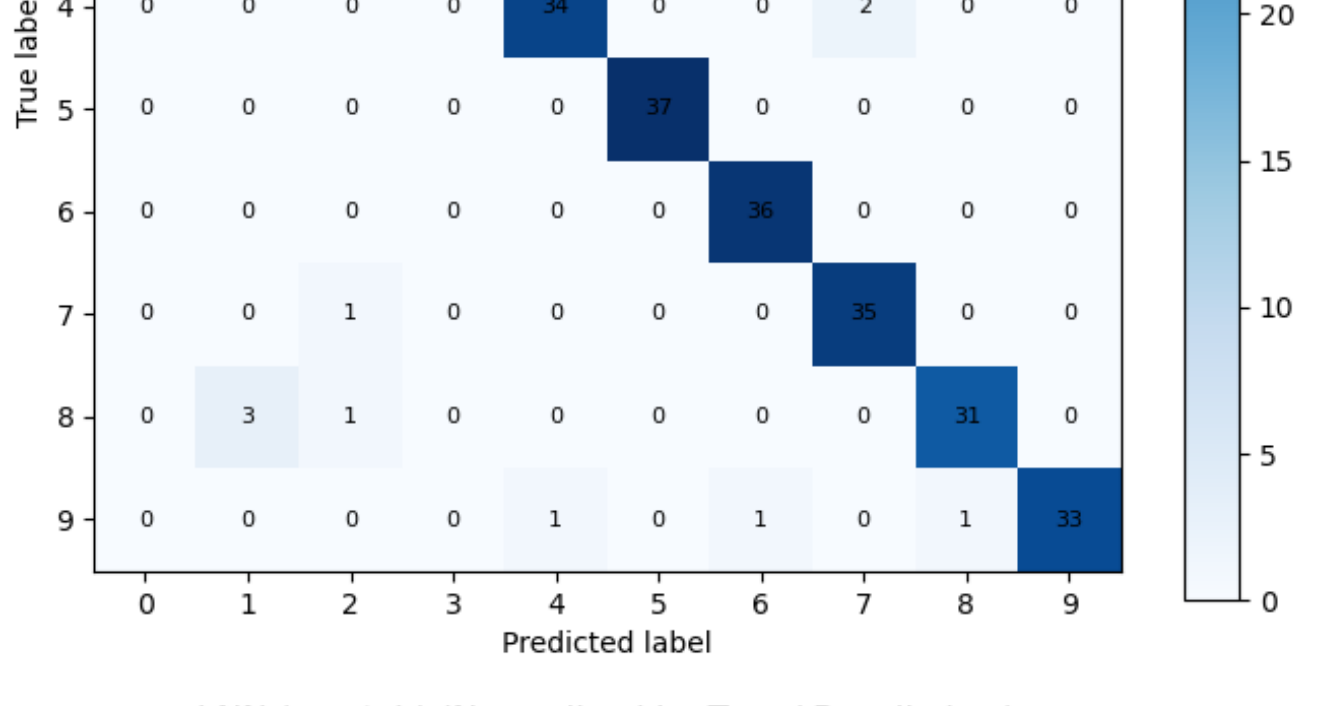
In [11]:
```python
# ----------------------------
# Cell purpose:
# Implement k-Nearest Neighbours (kNN) from scratch.
# kNN is a "lazy" classifier: there is no training phase.
# Prediction is done by comparing a test point to all training points,
# then using the majority label among the k closest neighbours.
# ----------------------------

def knn_predict_one(x, X_train, y_train, k):
    # x is a single test sample (one row of features).
    # X_train is the full training feature matrix (many rows).
    # y_train holds the class label for each row in X_train.
    # k controls how many neighbours participate in voting.

    # Distances from this test point to every training point
    dists = np.sqrt(((X_train - x) ** 2).sum(axis=1))
    # Step-by-step meaning:
    # 1) (X_train - x) subtracts x from every training row using broadcasting.
    #    Result: one difference vector per training sample.
    # 2) ** 2 squares each difference so negatives do not cancel positives.
    # 3) .sum(axis=1) adds squared differences across all features (per training sample),
    #    giving the squared Euclidean distance for each training row.
    # 4) np.sqrt(...) converts squared distance into Euclidean distance.
    #
    # Final result: dists is a 1D array where dists[i] is the distance from x
    # to the i-th training sample.

    # Indices of k closest points
    nn_idx = np.argsort(dists)[:k]
    # np.argsort(dists) returns indices that would sort the distance array from smallest to largest.
    # The first k indices correspond to the k nearest training samples.

    nn_labels = y_train[nn_idx]
    # Uses the neighbour indices to collect the labels of the nearest training samples.

    # Majority vote (ties go to the smallest label; consistent and deterministic)
    counts = np.bincount(nn_labels, minlength=10)
    # np.bincount counts how many times each class label appears among nn_labels.
    # minlength=10 forces the count array to always have length 10 (digits 0–9),
    # even if some classes do not appear in the neighbours.

    return int(np.argmax(counts))
    # np.argmax returns the index of the highest count, which corresponds to the predicted class.
    # If two classes tie with the same count, argmax returns the smaller index,
    # making tie-breaking consistent and repeatable.


def knn_predict(X_test, X_train, y_train, k=3):
    # Predicts labels for an entire test set by calling knn_predict_one on each sample.

    preds = np.empty(X_test.shape[0], dtype=int)
    # Pre-allocates an array to store predictions for every test sample.
    # X_test.shape[0] is the number of test samples.

    for i, x in enumerate(X_test):
        # Loops through the test set one sample at a time.
        # enumerate provides both the index i and the sample x.

        preds[i] = knn_predict_one(x, X_train, y_train, k)
        # Runs the "find nearest neighbours + vote" process for this single test sample,
        # then stores the predicted label into the correct position in preds.

    return preds
    # Returns the full prediction vector for the test set.
```

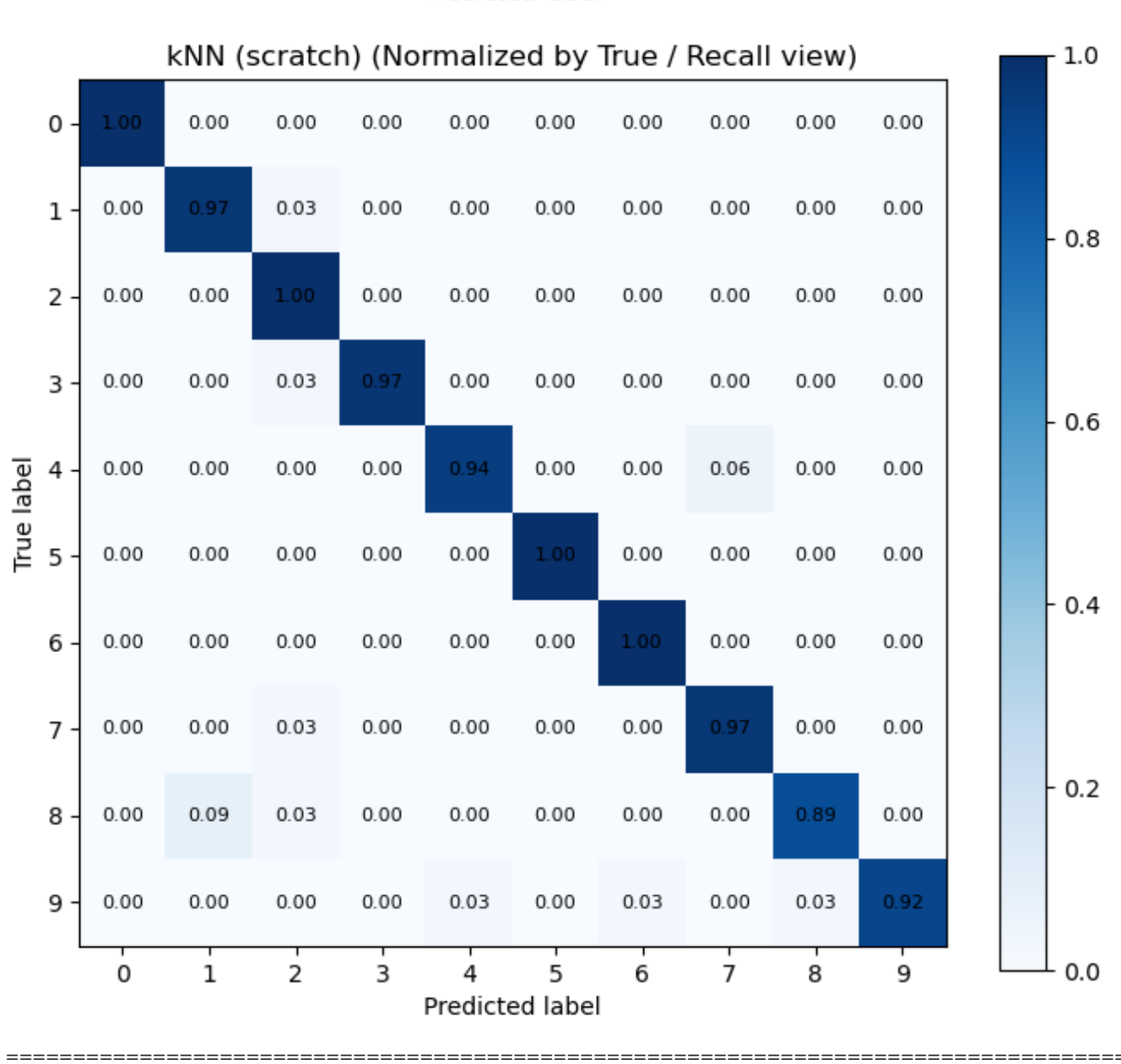
In [12]:
```python
# ----------------------------
# Cell purpose:
# Run the from-scratch kNN model, measure prediction runtime,
# and evaluate performance using NumPy-based metrics and a confusion matrix.
# ----------------------------

k = 3
# Sets the number of neighbours used in kNN voting.
# Smaller k can be sensitive to noise; larger k smooths predictions but may blur boundaries.

t0 = time.time()
# Records the start time so runtime can be measured.

y_pred_knn = knn_predict(X_test_scaled, X_train_scaled, y_train, k=k)
# Generates predictions for every test sample using the scaled features.
# X_test_scaled: test samples to classify
# X_train_scaled: training samples used as the reference set
# y_train: labels for the training samples
# k=k: number of nearest neighbours used for the majority vote

time_knn = time.time() - t0
# Computes elapsed time for prediction only (no training step exists for kNN).

acc_knn, p_knn, r_knn, f1_knn = print_metrics_block(
    f"kNN (from scratch) — k={k}",
    y_test, y_pred_knn, n_classes=10
)
# Evaluates predictions using NumPy-only metric utilities.
# print_metrics_block prints accuracy plus macro precision/recall/F1 in a formatted way.
# The returned values are stored so they can be compared later with other models.

cm_knn = confusion_matrix_numpy(y_test, y_pred_knn, n_classes=10)
# Builds the confusion matrix for kNN predictions.
# Each cell (i, j) shows how many samples of true class i were predicted as class j.

print(f"Runtime (predict only): {time_knn:.4f} seconds")
# Prints prediction runtime to compare computational cost across different models.
```

```
==========================================================
kNN (from scratch) — k=3
==========================================================
Accuracy : 0.9667
Precision: 0.9678  (macro)
Recall   : 0.9664  (macro)
F1-score : 0.9663  (macro)
==========================================================
Runtime (predict only): 0.0618 seconds
```

In [13]:
```python
# ----------------------------
# Cell purpose:
# Visualize and deeply analyze the confusion matrix produced by the
# from-scratch kNN classifier.
# This cell focuses on understanding *where* and *why* the model makes mistakes.
# ----------------------------

cm_knn = confusion_matrix_numpy(y_test, y_pred_knn, n_classes=10)
# Recomputes (or confirms) the confusion matrix for kNN predictions.
# Rows correspond to the true digit labels.
# Columns correspond to the predicted digit labels.

plot_confusion_matrix_individual(cm_knn, "kNN (scratch)", normalize=None)
# Plots the confusion matrix using raw counts.
# Each cell shows the exact number of samples classified into that (true, predicted) pair.
# Darker cells on the diagonal indicate strong correct classification.

plot_confusion_matrix_individual(cm_knn, "kNN (scratch)", normalize="true")
# Plots a row-normalized version of the confusion matrix.
# Each row sums to 1, making it easier to interpret recall per class.
# This view highlights which digits are most often confused with others.

deep_eval_from_confusion_matrix(cm_knn, "kNN (scratch)")
# Prints a detailed, text-based evaluation derived from the confusion matrix.
# This includes:
# - overall accuracy
# - macro and weighted F1 scores
# - TP, FP, FN, TN counts per class
# - weakest classes by recall and precision
# - most frequent misclassification pairs (true digit → predicted digit)
```





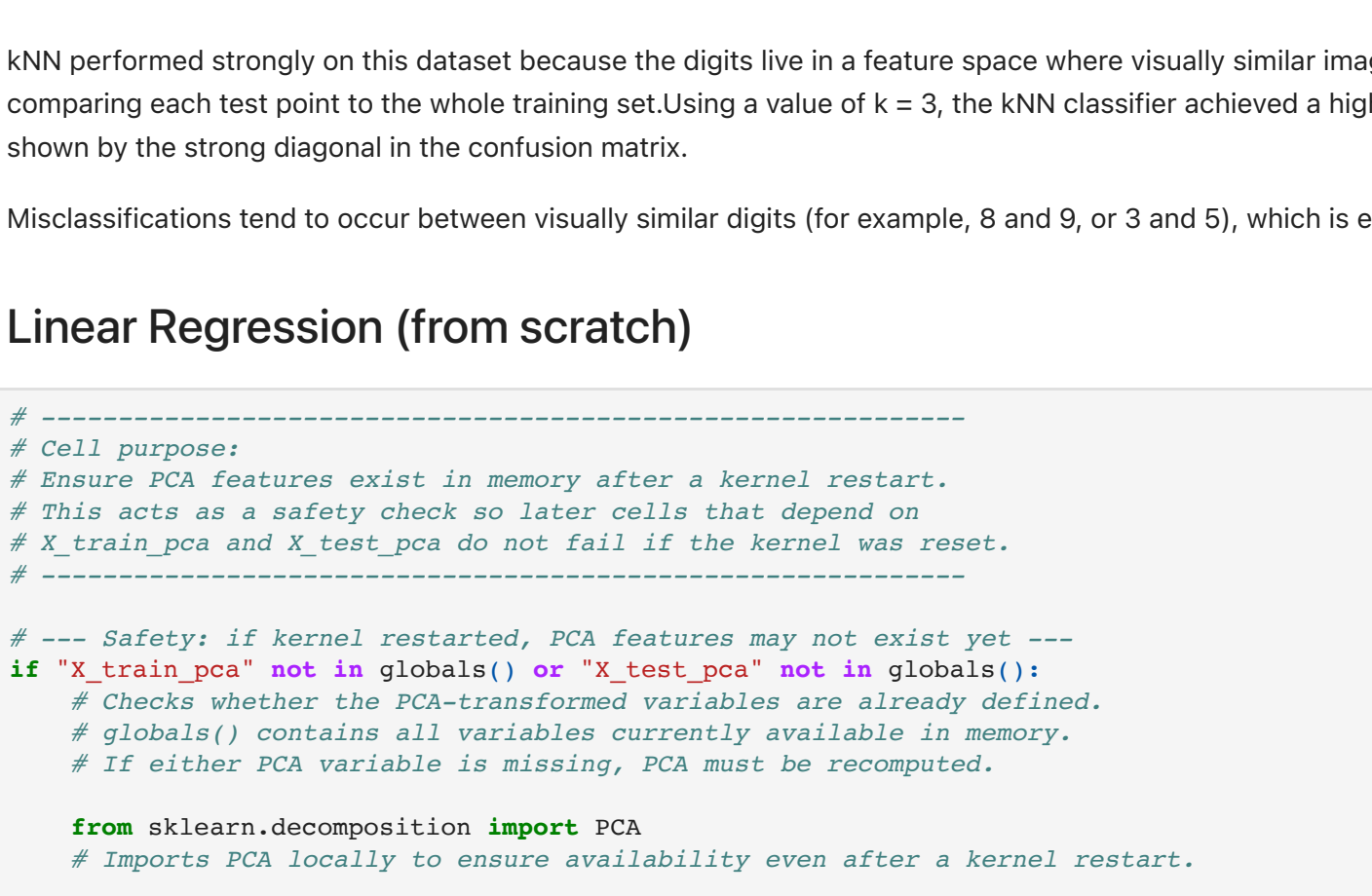

kNN (scratch) (Normalized by True / Recall view)

```
==============================================================================================
DETAILED CONFUSION MATRIX EVALUATION — kNN (scratch)
==============================================================================================
Interpretation (one-vs-rest for each class c):
  TP(c): predicted c AND true c (diagonal cell for c)
  FP(c): predicted c BUT true was not c (other rows in column c)
  FN(c): true c BUT predicted not c (other columns in row c)
  TN(c): not c AND predicted not c (everything not in row/col c)
----------------------------------------------------------------------------------------------
Overall Accuracy = (sum diagonal) / total = 348/360 = 0.9667
Macro avg (P/R/F1): 0.9678 / 0.9664 / 0.9663
Weighted F1       : 0.9666
----------------------------------------------------------------------------------------------
Class |    TP   FP   FN   TN | Precision  Recall    F1    | Support
----------------------------------------------------------------------------------------------
    0 |    36    0    0  324 |     1.000   1.000   1.000 |      36
    1 |    35    3    1  321 |     0.921   0.972   0.946 |      36
    2 |    35    4    0  321 |     0.897   1.000   0.946 |      35
    3 |    36    0    1  323 |     1.000   0.973   0.986 |      37
    4 |    34    1    2  323 |     0.971   0.944   0.958 |      36
    5 |    37    0    0  323 |     1.000   1.000   1.000 |      37
    6 |    36    1    0  323 |     0.973   1.000   0.986 |      36
    7 |    35    2    1  322 |     0.946   0.972   0.959 |      36
    8 |    31    1    4  324 |     0.969   0.886   0.925 |      35
    9 |    33    0    3  324 |     1.000   0.917   0.957 |      36
----------------------------------------------------------------------------------------------
Worst classes by Recall (high FN → true class often missed):
  Class 8: Recall=0.886 | FN=4 | Support=35
  Class 9: Recall=0.917 | FN=3 | Support=36
  Class 4: Recall=0.944 | FN=2 | Support=36
Worst classes by Precision (high FP → many false alarms for that predicted class):
  Class 2: Precision=0.897 | FP=4 | Predicted=39
  Class 1: Precision=0.921 | FP=3 | Predicted=38
  Class 7: Precision=0.946 | FP=2 | Predicted=37
----------------------------------------------------------------------------------------------
Top 8 confusions (True → Predicted):
  8 → 1: 3 cases (8.57% of true class 8)
  4 → 7: 2 cases (5.56% of true class 4)
  1 → 2: 1 cases (2.78% of true class 1)
  3 → 2: 1 cases (2.70% of true class 3)
  7 → 2: 1 cases (2.78% of true class 7)
  8 → 2: 1 cases (2.86% of true class 8)
  9 → 4: 1 cases (2.78% of true class 9)
  9 → 6: 1 cases (2.78% of true class 9)
```

In [14]:

```python
# ============================================================
# Effect of k on kNN performance (from scratch)
# ============================================================
# Cell purpose:
# Study how changing the number of neighbors (k) affects
# the accuracy of the kNN classifier.
# This helps justify the final choice of k in a principled way.
# ============================================================

k_values = [1, 3, 5, 7, 9, 11]
# Defines a range of k values to test.
# Small k values focus on very local patterns.
# Larger k values include more neighbors and produce smoother decisions.

acc_k = []
# Stores the test accuracy corresponding to each k value.

for k_val in k_values:
    # Iterates over each selected k value.

    y_pred_k = knn_predict(
        X_test_scaled,
        X_train_scaled,
        y_train,
        k=k_val
    )
    # Runs the kNN classifier using the current k value.
    # Predictions are generated for the test set using scaled features.

    acc = accuracy_numpy(y_test, y_pred_k)
    # Computes test accuracy for the current k.

    acc_k.append(acc)
    # Saves the accuracy so it can later be plotted against k.

    print(f"k = {k_val:<2} | Test Accuracy = {acc:.4f}")
    # Prints accuracy results immediately for transparency and quick comparison.

# Plot accuracy vs k
plt.figure(figsize=(7, 4))
# Creates a new figure to visualize the relationship between k and accuracy.

plt.plot(k_values, acc_k, marker="o")
# Plots accuracy as a function of k.
# Markers highlight individual experiment points.

plt.xlabel("Number of Neighbors (k)")
# Labels the x-axis to indicate the kNN hyperparameter being varied.

plt.ylabel("Test Accuracy")
# Labels the y-axis to indicate classification performance.

plt.title("Effect of k on kNN Accuracy (Scratch Implementation)")
# Adds a descriptive title explaining the purpose of the plot.

plt.grid(True)
# Adds grid lines to improve readability and comparison across values.

plt.tight_layout()
# Adjusts spacing so labels and title are clearly visible.

plt.show()
# Displays the plot.

print("""
INTERPRETATION (Effect of k in kNN):

This experiment demonstrates how the choice of k influences classification accuracy.

Very small values of k (such as k = 1) rely heavily on the closest single neighbor.
This makes predictions sensitive to noise and individual outliers, which can reduce stability.

As k increases, predictions become smoother because more neighbors participate in voting.
This generally improves robustness and reduces overfitting.

However, excessively large k values include many distant neighbors.
This introduces bias and causes local class structure to be ignored, leading to lower accuracy.

The results show that k = 3 provides a strong balance between bias and variance.
It achieves high accuracy while maintaining stable and consistent predictions.
For this reason, k = 3 is selected as the final value for the kNN model.
""")
```

```
k = 1  | Test Accuracy = 0.9667
k = 3  | Test Accuracy = 0.9667
k = 5  | Test Accuracy = 0.9639
k = 7  | Test Accuracy = 0.9667
k = 9  | Test Accuracy = 0.9639
k = 11 | Test Accuracy = 0.9639
```


Effect of k on kNN Accuracy (Scratch Implementation)

```
INTERPRETATION (Effect of k in kNN):

This experiment demonstrates how the choice of k influences classification accuracy.

Very small values of k (such as k = 1) rely heavily on the closest single neighbor.
This makes predictions sensitive to noise and individual outliers, which can reduce stability.

As k increases, predictions become smoother because more neighbors participate in voting.
This generally improves robustness and reduces overfitting.

However, excessively large k values include many distant neighbors.
This introduces bias and causes local class structure to be ignored, leading to lower accuracy.

The results show that k = 3 provides a strong balance between bias and variance.
It achieves high accuracy while maintaining stable and consistent predictions.
For this reason, k = 3 is selected as the final value for the kNN model.
```

kNN evaluation:

kNN performed strongly on this dataset because the digits live in a feature space where visually similar images tend to be close together. The trade-off is speed: prediction requires comparing each test point to the whole training set.Using a value of k = 3, the kNN classifier achieved a high classification accuracy on the test set. Most digits are classified correctly, as shown by the strong diagonal in the confusion matrix.

Misclassifications tend to occur between visually similar digits (for example, 8 and 9, or 3 and 5), which is expected given the simple pixel-based representation of the data.

Linear Regression (from scratch)

In [15]:

```python
# ------------------------------------------------------------
# Cell purpose:
# Ensure PCA features exist in memory after a kernel restart.
# This acts as a safety check so later cells that depend on
# X_train_pca and X_test_pca do not fail if the kernel was reset.
# ------------------------------------------------------------

# --- Safety: if kernel restarted, PCA features may not exist yet ---
if "X_train_pca" not in globals() or "X_test_pca" not in globals():
    # Checks whether the PCA-transformed variables are already defined.
    # globals() contains all variables currently available in memory.
    # If either PCA variable is missing, PCA must be recomputed.

    from sklearn.decomposition import PCA
    # Imports PCA locally to ensure availability even after a kernel restart.

    n_components = 20
    # Sets the number of principal components to keep.
    # This reduces the original 64-dimensional feature space down to 20 dimensions,
    # while retaining most of the important variance in the data.

    pca = PCA(n_components=n_components, random_state=RANDOM_STATE)
    # Creates a PCA object.
    # random_state ensures reproducibility of results.

    X_train_pca = pca.fit_transform(X_train_scaled)
    # Fits PCA using only the training data and transforms it.
    # Fitting on training data avoids information leakage from the test set.

    X_test_pca  = pca.transform(X_test_scaled)
    # Applies the same PCA transformation to the test data.
    # No fitting occurs here; the transformation learned from training data is reused.

    print("Recomputed PCA features after kernel restart.")
    # Confirms that PCA was recomputed due to missing variables.

    print("X_train_pca:", X_train_pca.shape, "| X_test_pca:", X_test_pca.shape)
    # Displays the shapes of the PCA-transformed training and test sets
    # to verify that dimensionality reduction was applied correctly.
```

```
Recomputed PCA features after kernel restart.
X_train_pca: (1437, 20) | X_test_pca: (360, 20)
```

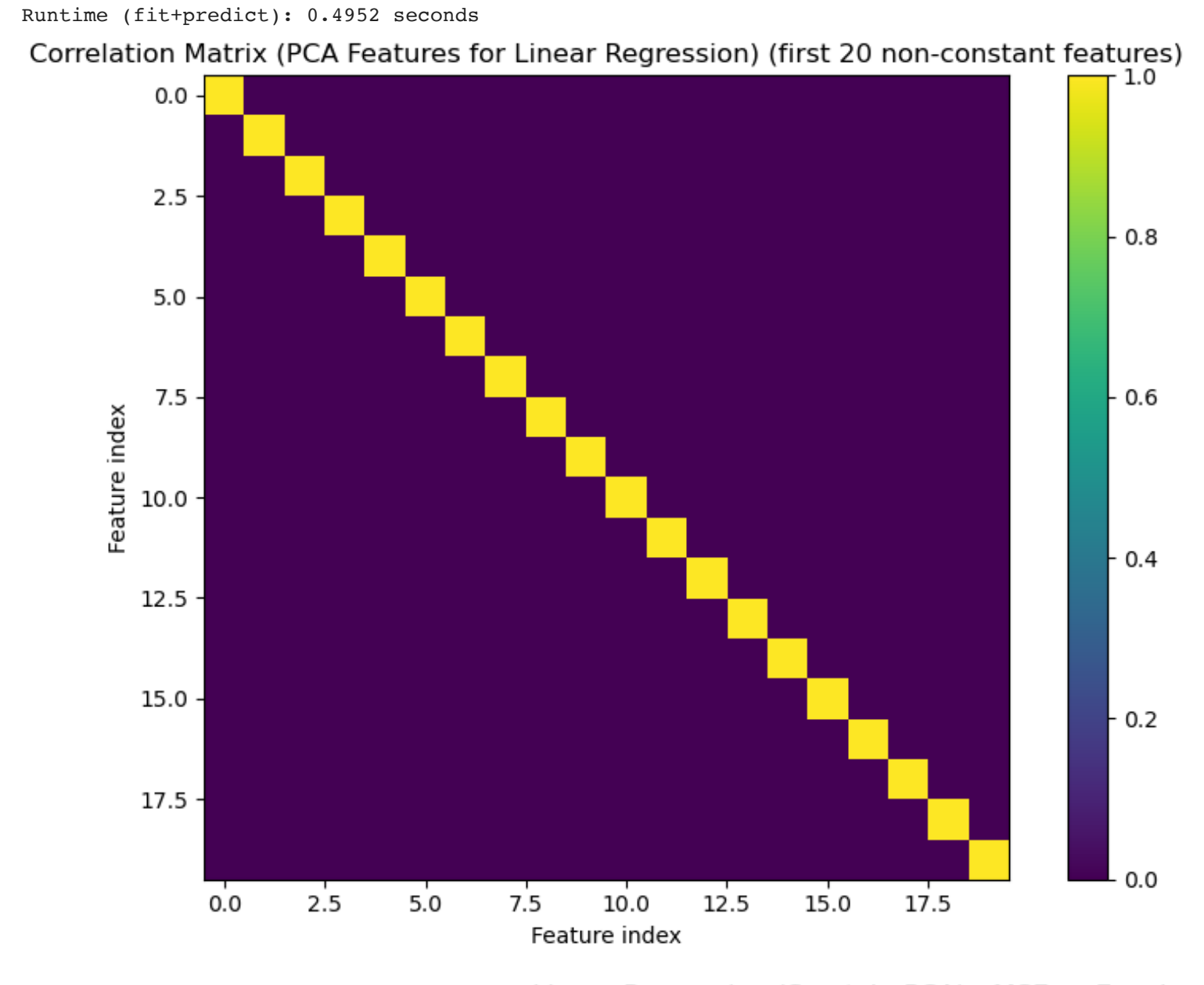
In [16]:

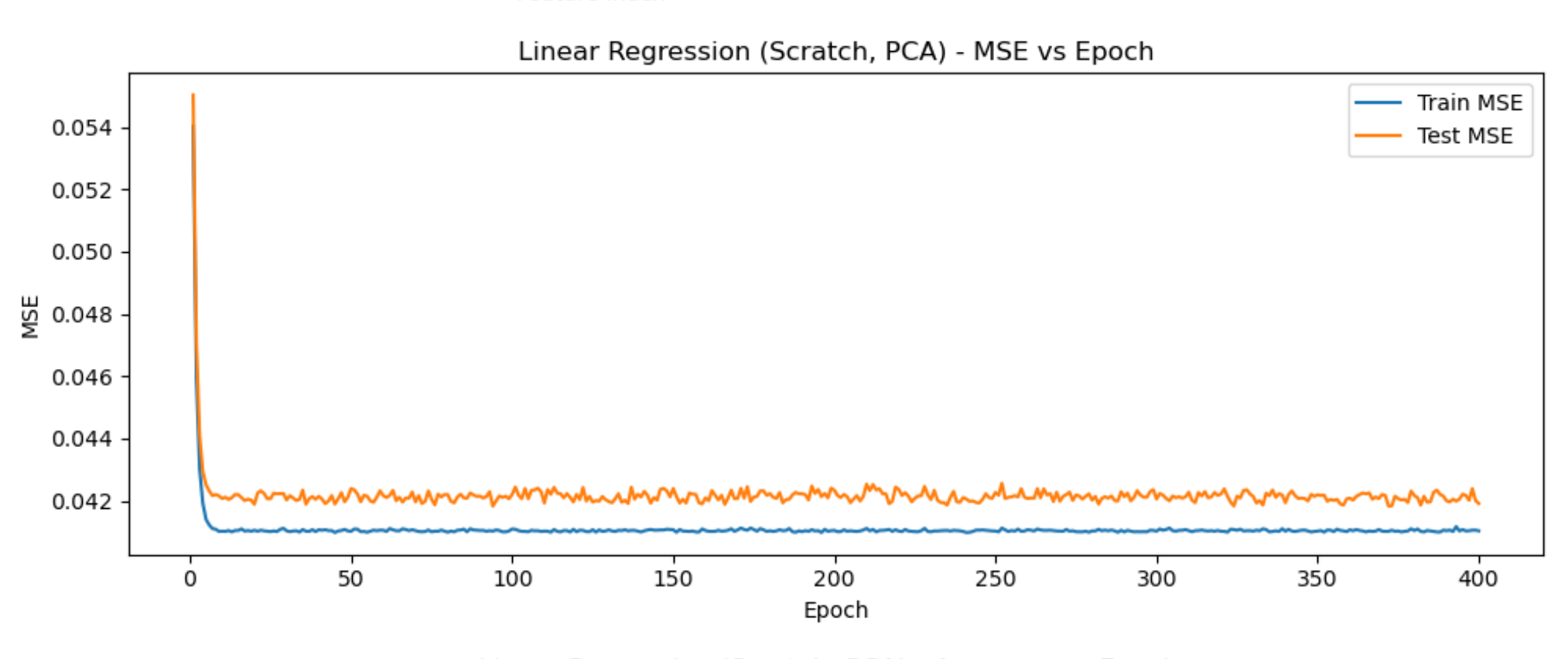
```python
# ============================================================
# Linear / Polynomial Regression (from scratch) utilities
# (NumPy + Matplotlib only; integrates with existing notebook flow)
# ============================================================
# Cell purpose:
# Provide helper functions needed for:
# - multiclass linear regression trained with gradient descent
# - multiclass evaluation (classification report, confusion matrix)
# - learning-curve and diagnostic plots
# - polynomial feature expansion (degree-2)
#
# Important idea:
# Linear regression is used here as a multiclass classifier by learning a weight
# matrix W that maps features to 10 class scores. Prediction selects the class
# with the highest score (argmax).
# ============================================================

def one_hot_numpy(y, n_classes=10):
    # Converts labels into one-hot encoded targets.
    # Example for digit classes: label 3 becomes [0,0,0,1,0,0,0,0,0,0]
    y = np.asarray(y).astype(int).ravel()
    # Forces labels into a clean 1D integer array.

    Y = np.zeros((y.shape[0], n_classes), dtype=np.float64)
    # Creates a matrix of zeros with shape (number of samples, number of classes).

    Y[np.arange(y.shape[0]), y] = 1.0
    # For each row i, sets the column matching label y[i] to 1.

    return Y
    # Returns the one-hot encoded target matrix.

def add_bias_numpy(X):
    # Adds a bias (intercept) column of ones to the feature matrix.
    # This allows the model to learn an offset term instead of forcing the decision
    # boundary to pass through the origin.
    X = np.asarray(X, dtype=np.float64)
    # Ensures features are in float64 format for stable numeric operations.

    return np.hstack([np.ones((X.shape[0], 1), dtype=np.float64), X])
    # Concatenates a (N,1) column of ones with the original feature matrix (N,D),
    # producing (N, D+1).

def mse_numpy(Y_true, Y_pred):
    # Mean Squared Error (MSE) for matrix targets.
    # Measures average squared difference between predictions and targets.
    return float(np.mean((Y_true - Y_pred) ** 2))

def predict_multiclass_from_scores(scores):
    # Converts a score matrix into predicted labels.
    # scores shape: (N, C), where each row is the class scores for one sample.
    # argmax selects the class index with the largest score.
    return np.argmax(scores, axis=1)

def accuracy_numpy(y_true, y_pred):
    # Computes accuracy as the fraction of correct predictions.
    y_true = np.asarray(y_true).astype(int).ravel()
    y_pred = np.asarray(y_pred).astype(int).ravel()
    return float(np.mean(y_true == y_pred))

def train_linear_regression_multiclass_gd(
    X_train, y_train,
    X_eval,  y_eval,
    n_classes=10,
    lr=0.01,
    epochs=600,
    batch_size=256,
    l2=0.0,
    grad_clip=1.0,        # gradient clipping prevents extremely large updates
    lr_decay=0.0,         # optional learning-rate decay per epoch
    seed=42
):
    # Trains a multiclass linear regression model using mini-batch gradient descent.
    #
    # Output model:
    #   scores = Xb @ W
    #   Xb includes a bias column, so W includes bias weights as its first row.
    #
    # Training objective:
    #   Minimize MSE between predicted scores and one-hot labels.
    #
    # Notes about stability:
    # - grad_clip limits update sizes to reduce exploding gradients
    # - lr_decay reduces learning rate over epochs to help convergence

    rng = np.random.default_rng(seed)
    # Creates a random number generator for reproducible shuffling and initialization.

    Xb_train = add_bias_numpy(X_train)
    # Adds the bias column to training features.

    Xb_eval  = add_bias_numpy(X_eval)
    # Adds the bias column to evaluation/test features.

    Y_train = one_hot_numpy(y_train, n_classes)
    # Converts training labels into one-hot targets.

    Y_eval  = one_hot_numpy(y_eval,  n_classes)
    # Converts evaluation labels into one-hot targets.

    n, d1 = Xb_train.shape
    # n = number of training samples
    # d1 = number of features + 1 bias feature

    W = rng.normal(0.0, 0.01, size=(d1, n_classes)).astype(np.float64)
    # Initializes the weight matrix with small random values.
    # Shape: (d1, n_classes)
    # Each column corresponds to weights for one class.

    hist = {"train_mse": [], "eval_mse": [], "train_acc": [], "eval_acc": []}
    # Stores metrics over epochs to support learning-curve plotting.

    for ep in range(epochs):
        # Loops over training epochs. Each epoch processes the whole training set once.

        # optional lr decay
        lr_ep = lr / (1.0 + lr_decay * ep) if lr_decay > 0 else lr
        # If lr_decay is enabled, the learning rate decreases gradually as ep increases.

        idx = rng.permutation(n)
        # Creates a shuffled index order for the training samples.

        Xs = Xb_train[idx]
        Ys = Y_train[idx]
        ys = np.asarray(y_train)[idx]
        # Applies the shuffled order so mini-batches see varied data each epoch.
        # ys is kept as label form for accuracy calculation later.

        for start in range(0, n, batch_size):
            # Iterates through shuffled data in blocks of batch_size.

            xb = Xs[start:start+batch_size]
            yb = Ys[start:start+batch_size]
            # Extracts one mini-batch of features and one-hot labels.

            scores = xb @ W
            # Computes predicted class scores for this batch.
            # scores shape: (batch_size, n_classes)

            diff = (scores - yb)
            # Computes prediction error relative to one-hot targets.

            grad = (2.0 / xb.shape[0]) * (xb.T @ diff)
            # Gradient of MSE loss with respect to W for this mini-batch.
            # xb.T @ diff accumulates how each feature contributes to the errors.
            # (2 / batch_size) is the derivative factor from squared error.

            if l2 > 0.0:
                # Applies L2 regularization when enabled.
                # This discourages very large weights and can improve generalization.
                reg = 2.0 * l2 * W
                # Gradient contribution for L2 regularization is 2*l2*W.

                reg[0, :] = 0.0
                # Excludes the bias row from regularization.
                # Regularizing bias often does not help and can reduce flexibility.

                grad += reg
                # Adds regularization gradient to the main gradient.

            # gradient clipping
            grad = np.clip(grad, -grad_clip, grad_clip)
            # Limits each gradient element into the range [-grad_clip, +grad_clip].
            # This reduces the risk of unstable updates when gradients become too large.

            W -= lr_ep * grad
            # Updates weights using gradient descent:
            # new_W = old_W - learning_rate * gradient

            # detect divergence early
            if not np.isfinite(W).all():
                # Checks whether any NaN or Inf values appeared in weights.
                # NaN/Inf often indicates divergence due to too-large updates.
                print("Training diverged (NaN/Inf in weights). Stopping early.")
                return W, hist
                # Stops immediately and returns current state to avoid wasting runtime.

        # epoch metrics
        train_scores = Xb_train @ W
        eval_scores  = Xb_eval  @ W
        # Computes full-set scores for training and evaluation sets.

        # if scores go inf/nan, stop
        if (not np.isfinite(train_scores).all()) or (not np.isfinite(eval_scores).all()):
            # Stops if the model output becomes numerically invalid.
            print("Training diverged (NaN/Inf in scores). Stopping early.")
            return W, hist

        train_mse = mse_numpy(Y_train, train_scores)
        eval_mse  = mse_numpy(Y_eval,  eval_scores)
        # MSE compares predicted score vectors to one-hot targets.
        # Lower MSE generally indicates better alignment between scores and correct classes.

        y_train_pred = np.argmax(train_scores, axis=1)
        y_eval_pred  = np.argmax(eval_scores, axis=1)
        # Converts score matrices to predicted class labels using argmax.

        train_acc = accuracy_numpy(y_train, y_train_pred)
        eval_acc  = accuracy_numpy(y_eval,  y_eval_pred)
        # Computes accuracy on both training and evaluation sets.

        hist["train_mse"].append(train_mse)
        hist["eval_mse"].append(eval_mse)
        hist["train_acc"].append(train_acc)
        hist["eval_acc"].append(eval_acc)
        # Stores metrics for learning curve plots.

        if (ep + 1) % 25 == 0:
            # Prints progress every 25 epochs to monitor training without excessive output.
            print(f"Epoch {ep+1:3d}/{epochs} | Train MSE {train_mse:.4f} Eval MSE {eval_mse:.4f} "
                  f"| Train Acc {train_acc:.4f} Eval Acc {eval_acc:.4f}")

    return W, hist
    # Returns the learned weight matrix and training history for plotting.

def classification_report_numpy(y_true, y_pred, n_classes=10):
    """
    Prints per-class precision, recall, f1, support + macro/weighted averages.
    No sklearn used.
    """
    # Converts labels to clean 1D integer arrays.
    y_true = np.asarray(y_true).astype(int).ravel()
    y_pred = np.asarray(y_pred).astype(int).ravel()

    eps = 1e-12
    # Small value added to denominators to prevent division by zero.

    precisions, recalls, f1s, supports = [], [], [], []
    # Lists to accumulate per-class metrics and counts.

    print("Class | Precision | Recall | F1-score | Support")
    print("-----------------------------------------------")

    for c in range(n_classes):
        # Computes TP/FP/FN/support using boolean masks.

        tp = np.sum((y_true == c) & (y_pred == c))
        # True positives: correctly predicted as class c.

        fp = np.sum((y_true != c) & (y_pred == c))
        # False positives: predicted as c but actually not c.

        fn = np.sum((y_true == c) & (y_pred != c))
        # False negatives: actually c but predicted as something else.

        support = np.sum(y_true == c)
        # Support: total true occurrences of class c.

        precision = tp / (tp + fp + eps)
        # Precision answers: of all predictions labeled c, what fraction are correct?

        recall    = tp / (tp + fn + eps)
        # Recall answers: of all true class c samples, what fraction were recovered?

        f1        = 2 * precision * recall / (precision + recall + eps)
        # F1 balances precision and recall.

        precisions.append(precision)
        recalls.append(recall)
        f1s.append(f1)
        supports.append(support)
        # Stores values for macro and weighted averages.

        print(f"{c:5d} | {precision:9.4f} | {recall:6.4f} | {f1:8.4f} | {support:7d}")
        # Prints one line per class, formatted for readability.

    precisions = np.array(precisions)
    recalls    = np.array(recalls)
    f1s        = np.array(f1s)
    supports   = np.array(supports)
    # Converts lists to NumPy arrays for vectorized averaging.

    macro_p = precisions.mean()
    macro_r = recalls.mean()
    macro_f = f1s.mean()
    # Macro averages treat each class equally (simple mean).

    weighted_p = np.sum(precisions * supports) / (np.sum(supports) + eps)
    weighted_r = np.sum(recalls    * supports) / (np.sum(supports) + eps)
    weighted_f = np.sum(f1s        * supports) / (np.sum(supports) + eps)
    # Weighted averages give more importance to classes with larger support.

    acc = np.mean(y_true == y_pred)
    # Accuracy is computed as the fraction of correct predictions overall.

    print("-----------------------------------------------")
    print(f"Accuracy: {acc:.4f}")
    print(f"Macro avg     P/R/F1: {macro_p:.4f} / {macro_r:.4f} / {macro_f:.4f}")
    print(f"Weighted avg  P/R/F1: {weighted_p:.4f} / {weighted_r:.4f} / {weighted_f:.4f}")

    return {
        "accuracy": acc,
        "macro_p": macro_p, "macro_r": macro_r, "macro_f1": macro_f,
        "weighted_p": weighted_p, "weighted_r": weighted_r, "weighted_f1": weighted_f
    }
    # Returns metrics as a dictionary for later comparison or reporting.

def confusion_matrix_numpy(y_true, y_pred, n_classes=10):
    # Builds a confusion matrix using integer counting.
    # This is a second definition of the same function name.
    # If both exist in the same notebook, this later version overwrites the earlier one.
    cm = np.zeros((n_classes, n_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[int(t), int(p)] += 1
    return cm

# -------------------------
# Detailed plotting helpers
# -------------------------

def plot_learning_curves(hist, title_prefix):
    # Plots training vs evaluation MSE and accuracy over epochs.

    e = np.arange(1, len(hist["train_mse"]) + 1)
    # Creates an array [1, 2, 3, ..., number_of_epochs_logged] for x-axis.

    plt.figure(figsize=(10, 4))
    plt.plot(e, hist["train_mse"], label="Train MSE")
    plt.plot(e, hist["eval_mse"], label="Test MSE")
    # Draws MSE curves for training and evaluation sets.

    plt.title(f"{title_prefix} - MSE vs Epoch")
    plt.xlabel("Epoch")
    plt.ylabel("MSE")
    plt.legend()
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(10, 4))
    plt.plot(e, hist["train_acc"], label="Train Accuracy")
    plt.plot(e, hist["eval_acc"], label="Test Accuracy")
    # Draws accuracy curves for training and evaluation sets.

    plt.title(f"{title_prefix} - Accuracy vs Epoch")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_correlation_heatmap(X, max_features=40, title="Correlation Matrix"):
    # Displays a correlation matrix between features.
    # This can reveal redundancy or strong relationships between features.

    X = np.asarray(X, dtype=np.float64)
    # Forces float format so correlation is computed reliably.

    # Remove constant (zero-variance) features to avoid NaNs in correlation
    std = X.std(axis=0)
    # Computes the standard deviation for each feature column.

    keep = std > 1e-12
    # Keeps only columns with meaningful variance.
    # Tiny standard deviation values are treated as constant features.

    Xn = X[:, keep]
    # Filters out constant features.

    if Xn.shape[1] == 0:
        # If all features are constant, correlation has no meaning.
        print("All features are constant; correlation matrix cannot be computed.")
        return

    d = min(Xn.shape[1], max_features)
    # Limits the number of displayed features to avoid overly large plots.

    Xs = Xn[:, :d]
    # Takes the first d features after filtering.

    C = np.corrcoef(Xs, rowvar=False)
    # Computes correlation matrix (feature-to-feature correlation).
    # rowvar=False means columns represent variables (features).

    # Replace NaNs if any remain (extra safety)
    C = np.nan_to_num(C, nan=0.0, posinf=0.0, neginf=0.0)
    # Ensures plot will not break due to invalid values.

    plt.figure(figsize=(8, 6))
    plt.imshow(C, interpolation="nearest")
    plt.title(f"{title} (first {d} non-constant features)")
    plt.xlabel("Feature index")
    plt.ylabel("Feature index")
    plt.colorbar()
    plt.tight_layout()
    plt.show()

def plot_residual_histogram(scores, y_true, n_classes=10, title="Residual Norm Histogram"):
    # Visualizes how far predicted score vectors are from the one-hot targets.
    # Larger residual norms often indicate uncertain or incorrect predictions.

    scores = np.asarray(scores, dtype=np.float64)
    # Ensures scores are a numeric float array.

    if not np.isfinite(scores).all():
        # Prevents crashes if training diverged and scores contain NaN/Inf.
        print(f"{title}: skipped (scores contain NaN/Inf).")
        return

    Y = one_hot_numpy(y_true, n_classes)
    # Builds one-hot targets for computing residuals.

    R = Y - scores
    # Residuals: difference between target vectors and predicted score vectors.

    rnorm = np.linalg.norm(R, axis=1)
    # Computes the Euclidean norm of each residual vector.

    rnorm = rnorm[np.isfinite(rnorm)]
    # Removes any invalid residual norms as a safety step.

    if rnorm.size == 0:
        # If no valid values remain, plotting would be meaningless.
        print(f"{title}: skipped (no finite residuals).")
        return

    plt.figure(figsize=(10, 4))
    plt.hist(rnorm, bins=35)
    plt.title(title)
    plt.xlabel("||one_hot(y) - scores||")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()

def plot_true_class_score_histogram(scores, y_true, title="Score Assigned to True Class"):
    # Visualizes how strongly the model scores the correct class.
    # Higher values suggest the model is confident about the true class.

    y_true = np.asarray(y_true).astype(int).ravel()
    # Ensures y_true is a clean 1D integer label array.

    s = scores[np.arange(len(y_true)), y_true]
    # Extracts, for each sample i, the score corresponding to the true class y_true[i].

    plt.figure(figsize=(10, 4))
    plt.hist(s, bins=35)
    plt.title(title)
    plt.xlabel("score(x) for the true class")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()

def plot_weight_heatmap(W, title="Weights Heatmap (excluding bias)"):
    # Plots the learned weight matrix as a heatmap for inspection.
    # Columns correspond to classes, rows correspond to features.

    Wn = W[1:, :]
    # Removes the bias row so only feature weights are displayed.

    plt.figure(figsize=(8, 6))
    plt.imshow(Wn, aspect="auto", interpolation="nearest")
    plt.title(title)
    plt.xlabel("Class")
    plt.ylabel("Feature index")
    plt.colorbar()
    plt.tight_layout()
    plt.show()

# -------------------------
# Polynomial features (deg-2)
# -------------------------

def poly_features_degree2_numpy(X):
    """
    Degree-2 polynomial features:
    [x, x^2, x_i*x_j for i<j]
    Works best on PCA features (e.g., 20 dims) to avoid explosion.
    """
    X = np.asarray(X, dtype=np.float64)
    # Ensures a float feature matrix.

    n, d = X.shape
    # n = number of samples
    # d = number of input features

    parts = [X, X**2]
    # Starts with linear features and squared features.

    inter = []
    # Will store interaction terms x_i * x_j where i < j.

    for i in range(d):
        for j in range(i+1, d):
            inter.append((X[:, i] * X[:, j]).reshape(n, 1))
            # Computes one interaction feature for every pair (i, j).
            # reshape(n,1) keeps the term as a 2D column so concatenation is consistent.

    if inter:
        parts.append(np.hstack(inter))
        # Stacks all interaction columns into one matrix and adds it to the parts list.

    return np.hstack(parts)
    # Concatenates [X | X^2 | interactions] into one expanded feature matrix.
```

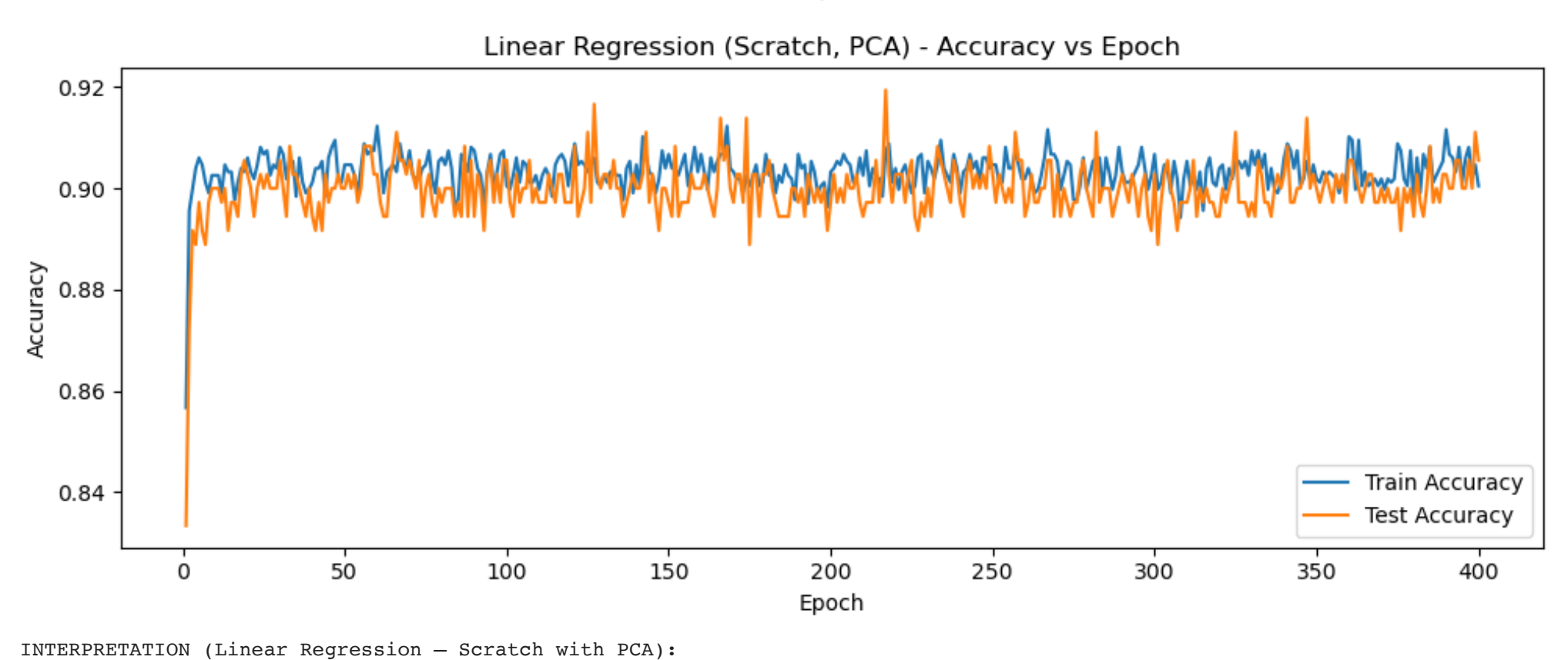
In [17]:

```python
# ============================================================
# Linear Regression (from scratch) - Multiclass via one-hot MSE
# CORRECTED: uses PCA features + tuned LR + stronger L2
# ============================================================
# Cell purpose:
# Train a multiclass linear regression model from scratch using gradient descent.
# The model learns a weight matrix W that maps features to 10 class scores.
# Targets are represented in one-hot form, and training minimizes mean squared error (MSE).
#
# Why PCA features are used:
# Digits data has 64 raw pixel features that are strongly correlated.
# PCA reduces dimensionality and removes redundancy, which makes gradient descent training
# more stable and often improves generalization.
# ============================================================

# 1) Use PCA features (important!)
X_train_lin = X_train_pca
# Selects PCA-transformed training features as input for linear regression.
# This assumes X_train_pca already exists from a previous PCA step.

X_test_lin  = X_test_pca
# Selects PCA-transformed test features so evaluation uses the same feature space.

# 2) Train + time (tuned hyperparameters)
t0 = time.time()
# Records the start time to measure total training + prediction cost.

W_lin, hist_lin = train_linear_regression_multiclass_gd(
    X_train_lin, y_train,
    X_test_lin,  y_test,
    n_classes=10,
    lr=0.03,           # Smaller learning rate reduces the risk of unstable updates or divergence.
    epochs=400,        # More epochs allow gradual improvement and convergence with a smaller lr.
    batch_size=256,    # Mini-batches reduce computation per update and add some stochasticity.
    l2=1e-2,           # Stronger L2 regularization discourages very large weights (improves stability).
    seed=RANDOM_STATE  # Ensures repeatable shuffling and repeatable weight initialization.
)
# Runs mini-batch gradient descent on one-hot targets.
# Output:
# - W_lin: learned weight matrix including bias weights
# - hist_lin: training history (MSE/accuracy per epoch for train and evaluation)

time_lin = time.time() - t0
# Computes total elapsed time for the training procedure.

# 3) Predict
scores_lin = add_bias_numpy(X_test_lin) @ W_lin
# Adds a bias column to test features, then multiplies by W_lin to produce scores.
# scores_lin shape: (number_of_test_samples, 10)
# Each row contains 10 scores, one per digit class.

y_pred_lin = np.argmax(scores_lin, axis=1)
# Converts the score matrix into class predictions.
# For each test sample, the class with the highest score is selected.

# 4) Metrics + confusion matrix (same flow as other models)
print("\nLinear Regression (scratch, PCA) - Classification Report")
# Prints a label header so output is clearly separated from other models.

lin_metrics = classification_report_numpy(y_test, y_pred_lin, n_classes=10)
# Prints per-class precision/recall/F1/support plus macro and weighted averages.
# Returns a dictionary of summary values, stored for later comparison.

cm_lin = confusion_matrix_numpy(y_test, y_pred_lin, n_classes=10)
# Builds the confusion matrix for error analysis.
# Each row represents the true class, each column represents the predicted class.

print("\nLinear Regression (scratch, PCA) - Confusion Matrix:\n", cm_lin)
# Prints the raw confusion matrix counts for direct inspection.

print(f"Runtime (fit+predict): {time_lin:.4f} seconds")
# Prints total runtime to compare computational cost with other methods.

# 5) Correlation heatmap (safe version of function recommended)
plot_correlation_heatmap(
    X_train_lin,
    max_features=min(40, X_train_lin.shape[1]),
    title="Correlation Matrix (PCA Features for Linear Regression)"
)
# Plots a correlation matrix for PCA features.
# PCA features should be less correlated than raw pixel features, so this plot often shows weaker correlations.
# max_features limits the number of displayed features for readability.

# 6) Detailed graphs (report-quality)
plot_learning_curves(hist_lin, "Linear Regression (Scratch, PCA)")
# Plots training vs evaluation MSE and accuracy over epochs.
# These curves show whether optimization is stable and whether overfitting appears.

print("""
INTERPRETATION (Linear Regression — Scratch with PCA):

These two plots show how the linear regression model learns over time when trained
on PCA-transformed features.

The first plot (MSE vs Epoch) shows a rapid decrease in both training and test MSE
during the initial epochs, confirming that gradient descent is working correctly.
After this, both curves stabilize, indicating that the model has converged. The
test MSE remains slightly higher than the training MSE, which is expected and
suggests good generalization without overfitting.

The second plot (Accuracy vs Epoch) shows that both training and test accuracy
increase quickly and then fluctuate around a stable value close to 90%. The small,
consistent gap between training and test accuracy indicates that the model performs
similarly on seen and unseen data.

Overall, these plots confirm stable optimization, proper convergence, and good
generalization of the from-scratch linear regression model in PCA feature space.
""")
# The interpretation text is printed exactly as written.
# This text can be directly used in the report to explain the learning curves.

plot_true_class_score_histogram(
    scores_lin, y_test,
    "Linear Regression (Scratch, PCA) - Test True-Class Score Histogram"
)
# Plots the distribution of scores assigned to the correct class for each test sample.
# Higher scores generally indicate stronger confidence in the correct class.

print("""
INTERPRETATION (True-Class Score Histogram — Linear Regression with PCA):

This histogram shows the distribution of the model's predicted scores for the
correct (true) class on the test set.

Most scores are concentrated in the mid-to-high range, indicating that the model
generally assigns higher confidence to the correct class for many test samples.
Scores closer to zero correspond to uncertain or misclassified samples, while
higher scores indicate more confident correct predictions.

The spread of values reflects varying confidence across different samples and
classes. The absence of a strong peak near zero suggests that the model rarely
assigns very low scores to the true class, which is consistent with the observed
high overall accuracy.

Overall, this plot shows that the linear regression model produces reasonably
confident predictions for the true class when operating in PCA feature space.
""")
# This interpretation explains how to read the true-class score histogram in report writing terms.

plot_residual_histogram(
    scores_lin, y_test, n_classes=10,
    title="Linear Regression (Scratch, PCA) - Test Residual Norms"
)
# Computes residual vectors: one_hot(y) - scores, then plots their norms.
# Residual norm acts like a "distance from the ideal one-hot target" for each sample.

print("""
INTERPRETATION (Residual Norms — Linear Regression with PCA):

This histogram shows the distribution of residual norms on the test set, where the
residual is the difference between the true one-hot label vector and the model's
predicted score vector.

Smaller residual norms indicate predictions that are closer to the true class
encoding, meaning more accurate and confident predictions. Most residuals are
concentrated in the lower-to-middle range, showing that the model generally produces
reasonable predictions for most test samples.

Larger residual values correspond to misclassified or low-confidence samples, which
appear less frequently. The absence of extreme residuals for most samples suggests
stable model behavior without severe prediction errors.

Overall, this plot confirms that the linear regression model makes mostly consistent
and well-aligned predictions in PCA feature space.
""")
# This interpretation describes what residual norms mean and how they relate to model quality.

plot_weight_heatmap(W_lin, "Linear Regression (Scratch, PCA) - Weight Heatmap (no bias)")
# Visualizes learned weights for each class across PCA features (bias excluded).
# This is useful for showing that the model learned different patterns for different digits.

print("""
INTERPRETATION (Weight Heatmap — Linear Regression with PCA):

This heatmap visualizes the learned weight matrix of the linear regression model
trained on PCA features, excluding the bias term. Each row corresponds to a PCA
feature, and each column corresponds to a class.

Color intensity indicates the magnitude and sign of the weights. Positive values
show features that increase the score for a class, while negative values reduce it.
Brighter colors represent stronger influence.

The variation across columns shows that different classes rely on different PCA
features. The absence of extremely large weights indicates that the model is well
regularized and not dominated by any single feature.

Overall, this plot illustrates how the linear model combines PCA features to form
class-specific predictions in a stable and interpretable manner.
""")
# This interpretation describes what the weight heatmap communicates for analysis/report writing.
```

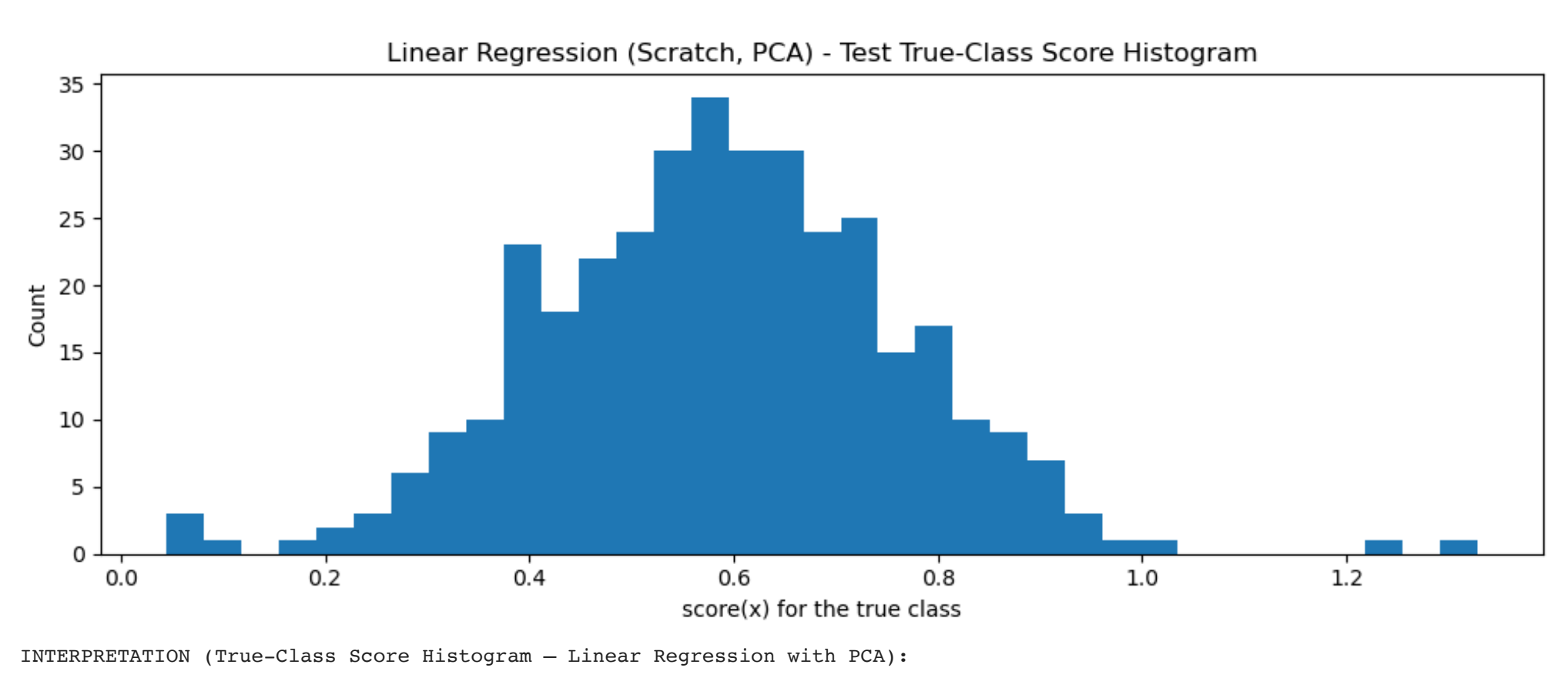
```
Epoch  25/400 | Train MSE 0.0410 Eval MSE 0.0421 | Train Acc 0.9068 Eval Acc 0.9000
Epoch  50/400 | Train MSE 0.0410 Eval MSE 0.0424 | Train Acc 0.9047 Eval Acc 0.9000
Epoch  75/400 | Train MSE 0.0410 Eval MSE 0.0421 | Train Acc 0.9047 Eval Acc 0.9000
Epoch 100/400 | Train MSE 0.0411 Eval MSE 0.0422 | Train Acc 0.9005 Eval Acc 0.9056
Epoch 125/400 | Train MSE 0.0411 Eval MSE 0.0420 | Train Acc 0.9033 Eval Acc 0.9111
Epoch 150/400 | Train MSE 0.0411 Eval MSE 0.0424 | Train Acc 0.9068 Eval Acc 0.8972
Epoch 175/400 | Train MSE 0.0411 Eval MSE 0.0421 | Train Acc 0.9005 Eval Acc 0.8889
Epoch 200/400 | Train MSE 0.0411 Eval MSE 0.0423 | Train Acc 0.9033 Eval Acc 0.8972
Epoch 225/400 | Train MSE 0.0410 Eval MSE 0.0421 | Train Acc 0.8991 Eval Acc 0.9056
Epoch 250/400 | Train MSE 0.0410 Eval MSE 0.0423 | Train Acc 0.9047 Eval Acc 0.9028
Epoch 275/400 | Train MSE 0.0410 Eval MSE 0.0422 | Train Acc 0.8977 Eval Acc 0.8972
Epoch 300/400 | Train MSE 0.0410 Eval MSE 0.0421 | Train Acc 0.9068 Eval Acc 0.9028
Epoch 325/400 | Train MSE 0.0411 Eval MSE 0.0422 | Train Acc 0.9054 Eval Acc 0.9111
Epoch 350/400 | Train MSE 0.0410 Eval MSE 0.0422 | Train Acc 0.9019 Eval Acc 0.9000
Epoch 375/400 | Train MSE 0.0410 Eval MSE 0.0421 | Train Acc 0.9088 Eval Acc 0.9000
Epoch 400/400 | Train MSE 0.0410 Eval MSE 0.0419 | Train Acc 0.9005 Eval Acc 0.9056

Linear Regression (scratch, PCA) - Classification Report
Class | Precision | Recall | F1-score | Support
----------------------------------------------
    0 |    0.9231 | 1.0000 |   0.9600 |      36
    1 |    0.7647 | 0.7222 |   0.7429 |      36
    2 |    0.8250 | 0.9429 |   0.8800 |      35
    3 |    0.9722 | 0.9459 |   0.9589 |      37
    4 |    0.9459 | 0.9722 |   0.9589 |      36
    5 |    0.9231 | 0.9730 |   0.9474 |      37
    6 |    0.9714 | 0.9444 |   0.9577 |      36
    7 |    0.8947 | 0.9444 |   0.9189 |      36
    8 |    0.9615 | 0.7143 |   0.8197 |      35
    9 |    0.8889 | 0.8889 |   0.8889 |      36
----------------------------------------------
Accuracy: 0.9056
Macro avg     P/R/F1: 0.9071 / 0.9048 / 0.9033
Weighted avg  P/R/F1: 0.9074 / 0.9056 / 0.9039

Linear Regression (scratch, PCA) - Confusion Matrix:
 [[36  0  0  0  0  0  0  0  0  0]
 [ 0 26  5  0  1  0  1  0  0  3]
 [ 1  0 33  1  0  0  0  0  0  0]
 [ 0  0  1 35  0  1  0  0  0  0]
 [ 1  0  0  0 35  0  0  0  0  0]
 [ 0  0  0  0  0 36  0  0  0  1]
 [ 1  1  0  0  0  0 34  0  0  0]
 [ 0  0  1  0  1  0  0 34  0  0]
 [ 0  7  0  0  0  2  0  1 25  0]
 [ 0  0  0  0  0  0  0  3  1 32]]
Runtime (fit+predict): 0.4952 seconds
```



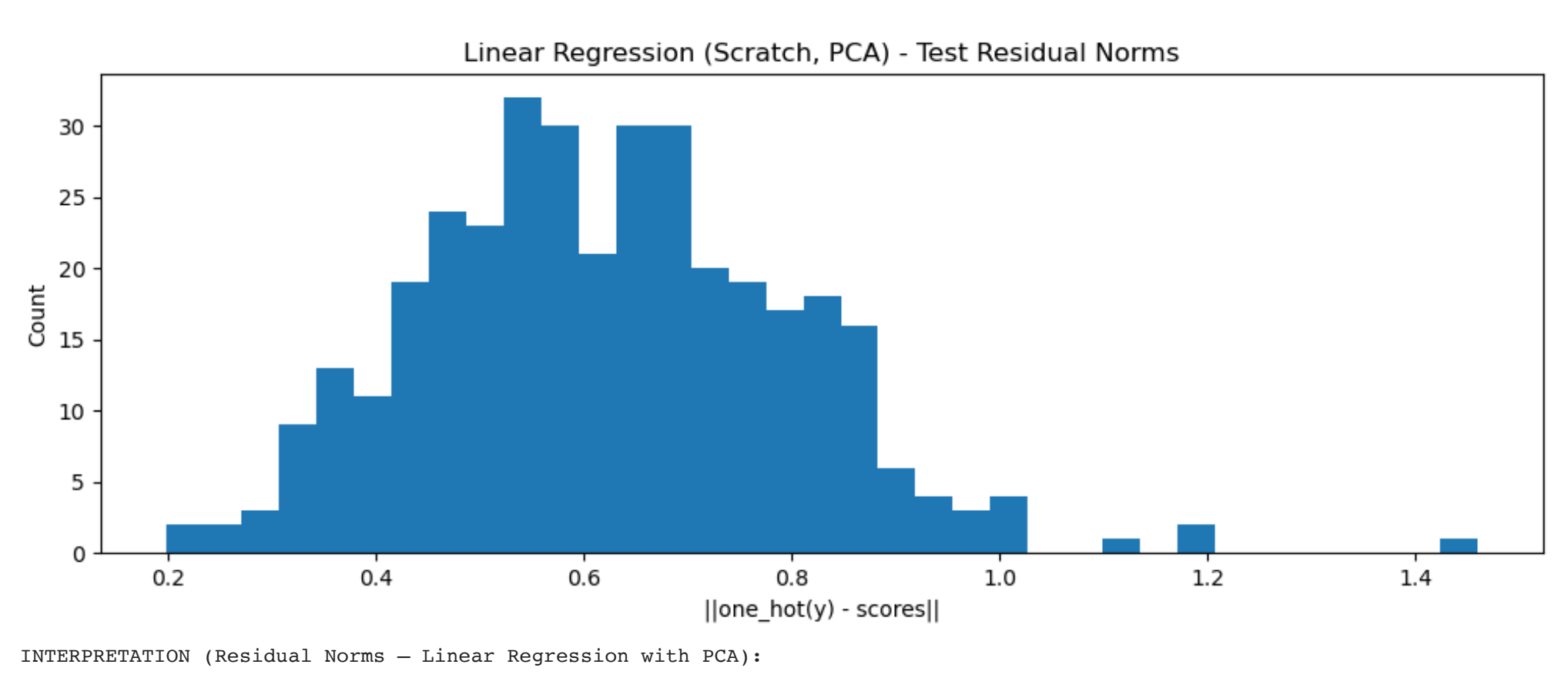




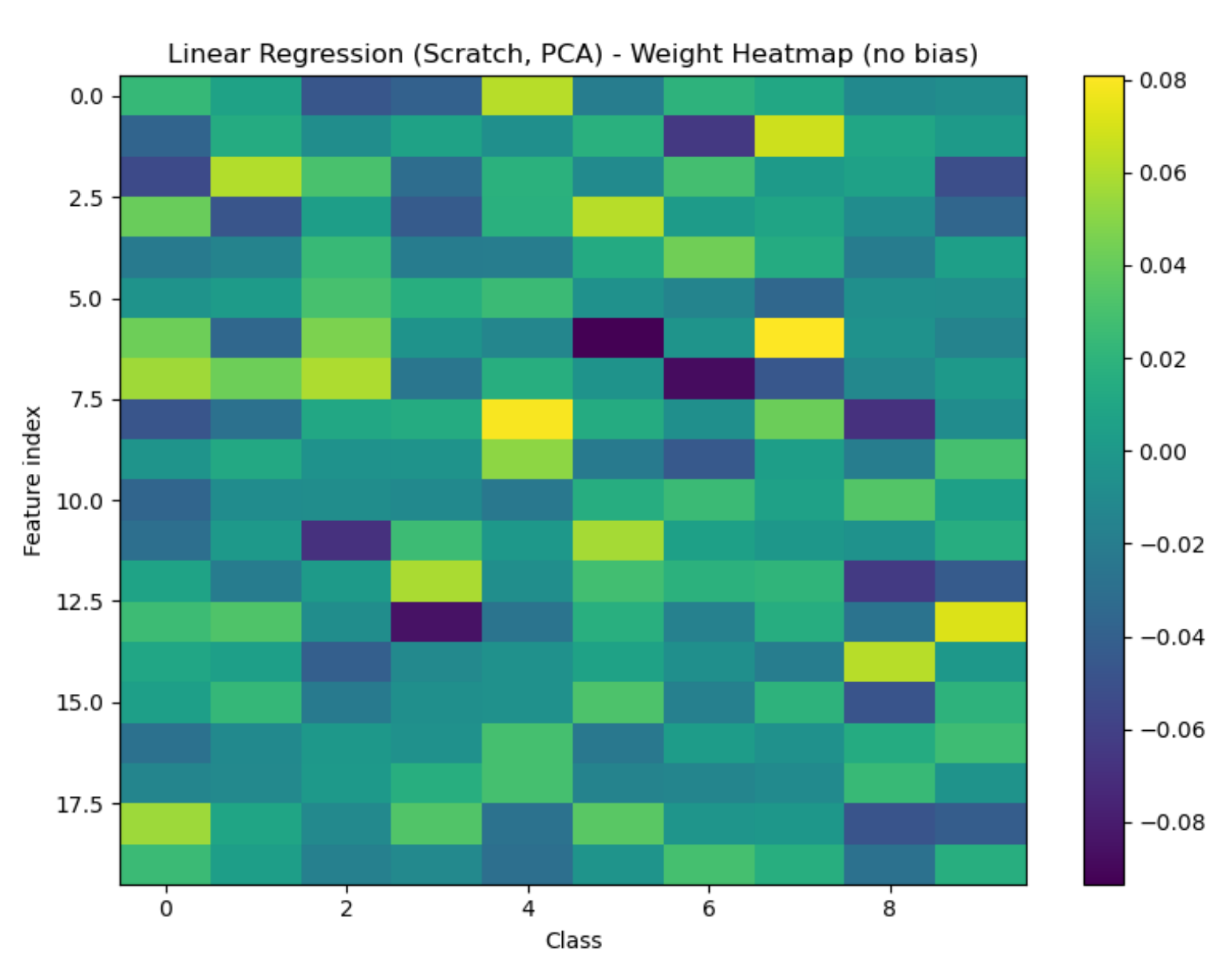
```
INTERPRETATION (Linear Regression — Scratch with PCA):

These two plots show how the linear regression model learns over time when trained
on PCA-transformed features.

The first plot (MSE vs Epoch) shows a rapid decrease in both training and test MSE
during the initial epochs, confirming that gradient descent is working correctly.
After this, both curves stabilize, indicating that the model has converged. The
test MSE remains slightly higher than the training MSE, which is expected and
suggests good generalization without overfitting.

The second plot (Accuracy vs Epoch) shows that both training and test accuracy
increase quickly and then fluctuate around a stable value close to 90%. The small,
consistent gap between training and test accuracy indicates that the model performs
similarly on seen and unseen data.

Overall, these plots confirm stable optimization, proper convergence, and good
generalization of the from-scratch linear regression model in PCA feature space.
```



```
INTERPRETATION (True-Class Score Histogram — Linear Regression with PCA):

This histogram shows the distribution of the model's predicted scores for the
correct (true) class on the test set.

Most scores are concentrated in the mid-to-high range, indicating that the model
generally assigns higher confidence to the correct class for many test samples.
Scores closer to zero correspond to uncertain or misclassified samples, while
higher scores indicate more confident correct predictions.

The spread of values reflects varying confidence across different samples and
classes. The absence of a strong peak near zero suggests that the model rarely
assigns very low scores to the true class, which is consistent with the observed
high overall accuracy.

Overall, this plot shows that the linear regression model produces reasonably
confident predictions for the true class when operating in PCA feature space.
```



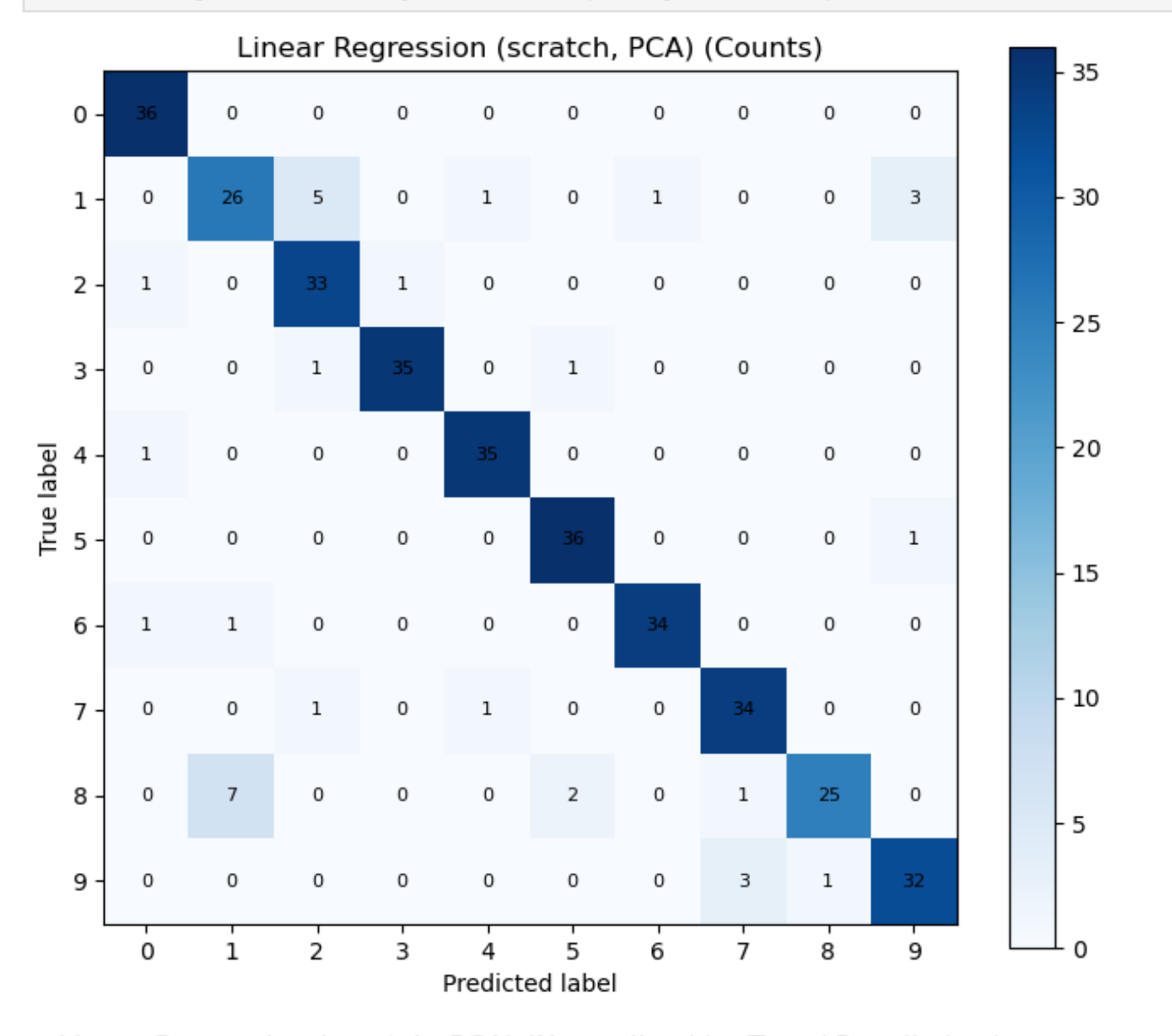
```
INTERPRETATION (Residual Norms — Linear Regression with PCA):

This histogram shows the distribution of residual norms on the test set, where the
residual is the difference between the true one-hot label vector and the model's
predicted score vector.

Smaller residual norms indicate predictions that are closer to the true class
encoding, meaning more accurate and confident predictions. Most residuals are
concentrated in the lower-to-middle range, showing that the model generally produces
reasonable predictions for most test samples.

Larger residual values correspond to misclassified or low-confidence samples, which
appear less frequently. The absence of extreme residuals for most samples suggests
stable model behavior without severe prediction errors.

Overall, this plot confirms that the linear regression model makes mostly consistent
and well-aligned predictions in PCA feature space.
```



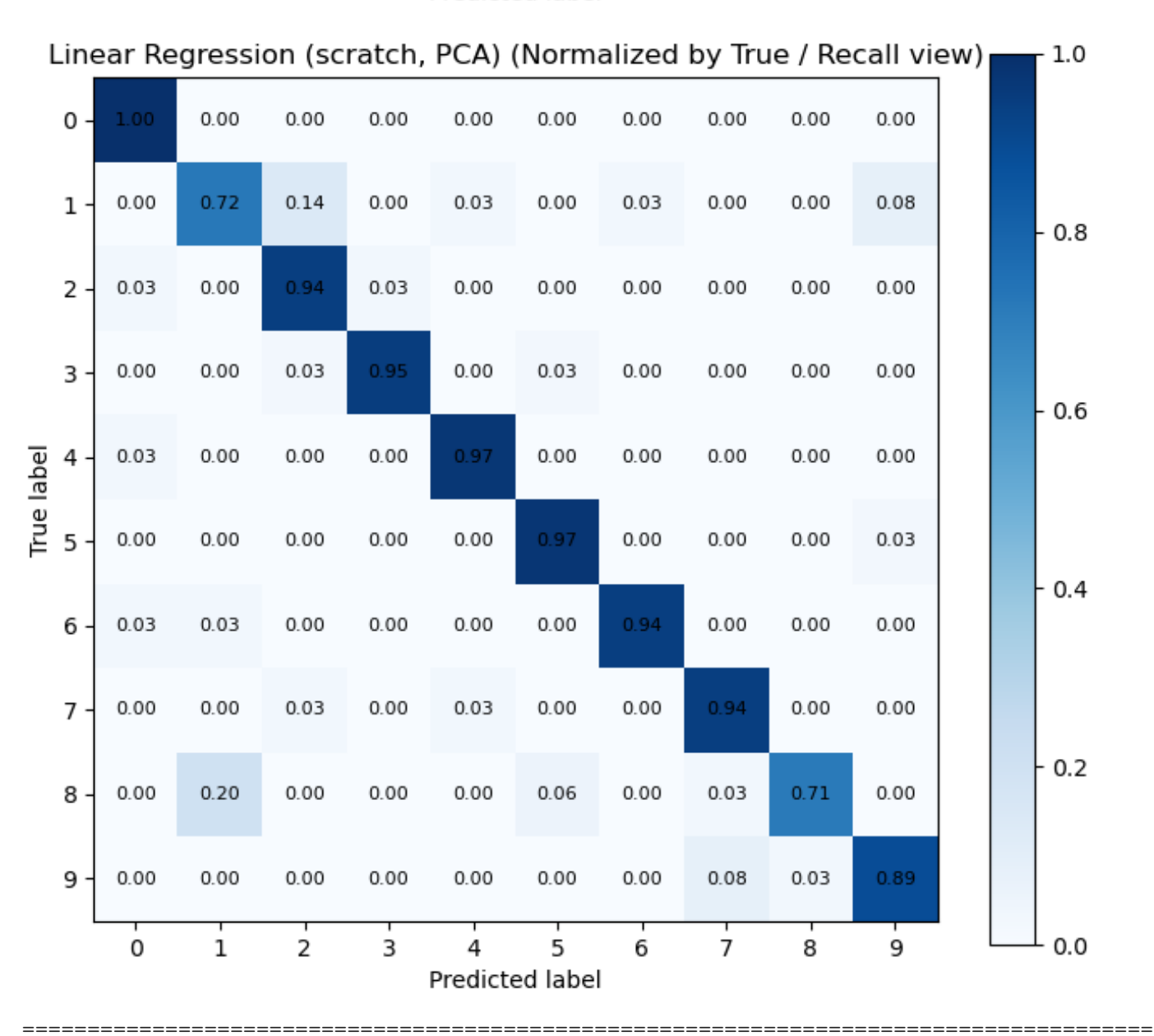
```
INTERPRETATION (Weight Heatmap — Linear Regression with PCA):

This heatmap visualizes the learned weight matrix of the linear regression model
trained on PCA features, excluding the bias term. Each row corresponds to a PCA
feature, and each column corresponds to a class.

Color intensity indicates the magnitude and sign of the weights. Positive values
show features that increase the score for a class, while negative values reduce it.
Brighter colors represent stronger influence.

The variation across columns shows that different classes rely on different PCA
features. The absence of extremely large weights indicates that the model is well
regularized and not dominated by any single feature.

Overall, this plot illustrates how the linear model combines PCA features to form
class-specific predictions in a stable and interpretable manner.
```

In [18]:

```
# ---------------------------
# Cell purpose:
# Visualize and deeply analyze the confusion matrix for the
# from-scratch Linear Regression model trained on PCA features.
# This section highlights class-by-class strengths and weaknesses,
# and identifies the most frequent misclassification patterns.
# ---------------------------

cm_lin = confusion_matrix_numpy(y_test, y_pred_lin, n_classes=10)
# Builds the confusion matrix for the linear regression predictions.
# Rows represent the true digit labels.
# Columns represent the predicted digit labels.
# Each cell (i, j) contains the number of samples with true label i predicted as j.

plot_confusion_matrix_individual(cm_lin, "Linear Regression (scratch, PCA)", normalize=None)
# Plots the confusion matrix using raw counts.
# This shows the exact number of correct predictions (diagonal) and mistakes (off-diagonal).

plot_confusion_matrix_individual(cm_lin, "Linear Regression (scratch, PCA)", normalize="true")
# Plots the confusion matrix normalized by true class (row-normalized).
# Each row sums to 1, so each cell becomes a proportion.
# This view makes recall per class easier to compare across digits.

deep_eval_from_confusion_matrix(cm_lin, "Linear Regression (scratch, PCA)")
# Prints a detailed confusion-matrix evaluation, including:
# - overall accuracy from diagonal/total
# - macro precision/recall/F1 and weighted F1
# - TP/FP/FN/TN per class using a one-vs-rest interpretation
# - weakest classes by recall (many FN) and by precision (many FP)
# - most frequent confusion pairs (true digit → predicted digit)
```



	0	1	2	3	4	5	6	7	8	9
0	36	0	0	0	0	0	0	0	0	0
1	0	26	5	0	1	0	1	0	0	3
2	1	0	33	1	0	0	0	0	0	0
3	0	0	1	35	0	1	0	0	0	0
4	1	0	0	0	35	0	0	0	0	0
5	0	0	0	0	0	36	0	0	0	1
6	1	1	0	0	0	0	34	0	0	0
7	0	0	1	0	1	0	0	34	0	0
8	0	7	0	0	0	2	0	1	25	0
9	0	0	0	0	0	0	0	3	1	32



	0	1	2	3	4	5	6	7	8	9
0	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1	0.00	0.72	0.14	0.00	0.03	0.00	0.03	0.00	0.00	0.08
2	0.03	0.00	0.94	0.03	0.00	0.00	0.00	0.00	0.00	0.00
3	0.00	0.00	0.03	0.95	0.00	0.03	0.00	0.00	0.00	0.00
4	0.03	0.00	0.00	0.00	0.97	0.00	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.00	0.00	0.97	0.00	0.00	0.00	0.03
6	0.03	0.03	0.00	0.00	0.00	0.00	0.94	0.00	0.00	0.00
7	0.00	0.00	0.03	0.00	0.03	0.00	0.00	0.94	0.00	0.00
8	0.00	0.20	0.00	0.00	0.00	0.06	0.00	0.03	0.71	0.00
9	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.08	0.03	0.89

```
==========================================================================================
DETAILED CONFUSION MATRIX EVALUATION — Linear Regression (scratch, PCA)
==========================================================================================
Interpretation (one-vs-rest for each class c):
  TP(c): predicted c AND true c (diagonal cell for c)
  FP(c): predicted c BUT true was not c (other rows in column c)
  FN(c): true c BUT predicted not c (other columns in row c)
  TN(c): not c AND predicted not c (everything not in row/col c)
------------------------------------------------------------------------------------------
Overall Accuracy = (sum diagonal) / total = 326/360 = 0.9056
Macro avg (P/R/F1): 0.9071 / 0.9048 / 0.9033
Weighted F1       : 0.9039
------------------------------------------------------------------------------------------
Class |   TP   FP   FN   TN | Precision  Recall    F1   | Support
------------------------------------------------------------------------------------------
    0 |   36    3    0  321 |    0.923   1.000   0.960 |      36
    1 |   26    8   10  316 |    0.765   0.722   0.743 |      36
    2 |   33    7    2  318 |    0.825   0.943   0.880 |      35
    3 |   35    1    2  322 |    0.972   0.946   0.959 |      37
    4 |   35    2    1  322 |    0.946   0.972   0.959 |      36
    5 |   36    3    1  320 |    0.923   0.973   0.947 |      37
    6 |   34    1    2  323 |    0.971   0.944   0.958 |      36
    7 |   34    4    2  320 |    0.895   0.944   0.919 |      36
    8 |   25    1   10  324 |    0.962   0.714   0.820 |      35
    9 |   32    4    4  320 |    0.889   0.889   0.889 |      36
------------------------------------------------------------------------------------------
Worst classes by Recall (high FN → true class often missed):
  Class 8: Recall=0.714 | FN=10 | Support=35
  Class 1: Recall=0.722 | FN=10 | Support=36
  Class 9: Recall=0.889 | FN=4 | Support=36
Worst classes by Precision (high FP → many false alarms for that predicted class):
  Class 1: Precision=0.765 | FP=8 | Predicted=34
  Class 2: Precision=0.825 | FP=7 | Predicted=40
  Class 9: Precision=0.889 | FP=4 | Predicted=36
------------------------------------------------------------------------------------------
Top 8 confusions (True → Predicted):
  8 → 1: 7 cases (20.00% of true class 8)
  1 → 2: 5 cases (13.89% of true class 1)
  1 → 9: 3 cases (8.33% of true class 1)
  9 → 7: 3 cases (8.33% of true class 9)
  8 → 5: 2 cases (5.71% of true class 8)
  1 → 4: 1 cases (2.78% of true class 1)
  1 → 6: 1 cases (2.78% of true class 1)
  2 → 0: 1 cases (2.86% of true class 2)
```

Plynomial regression from scratch

In [19]:

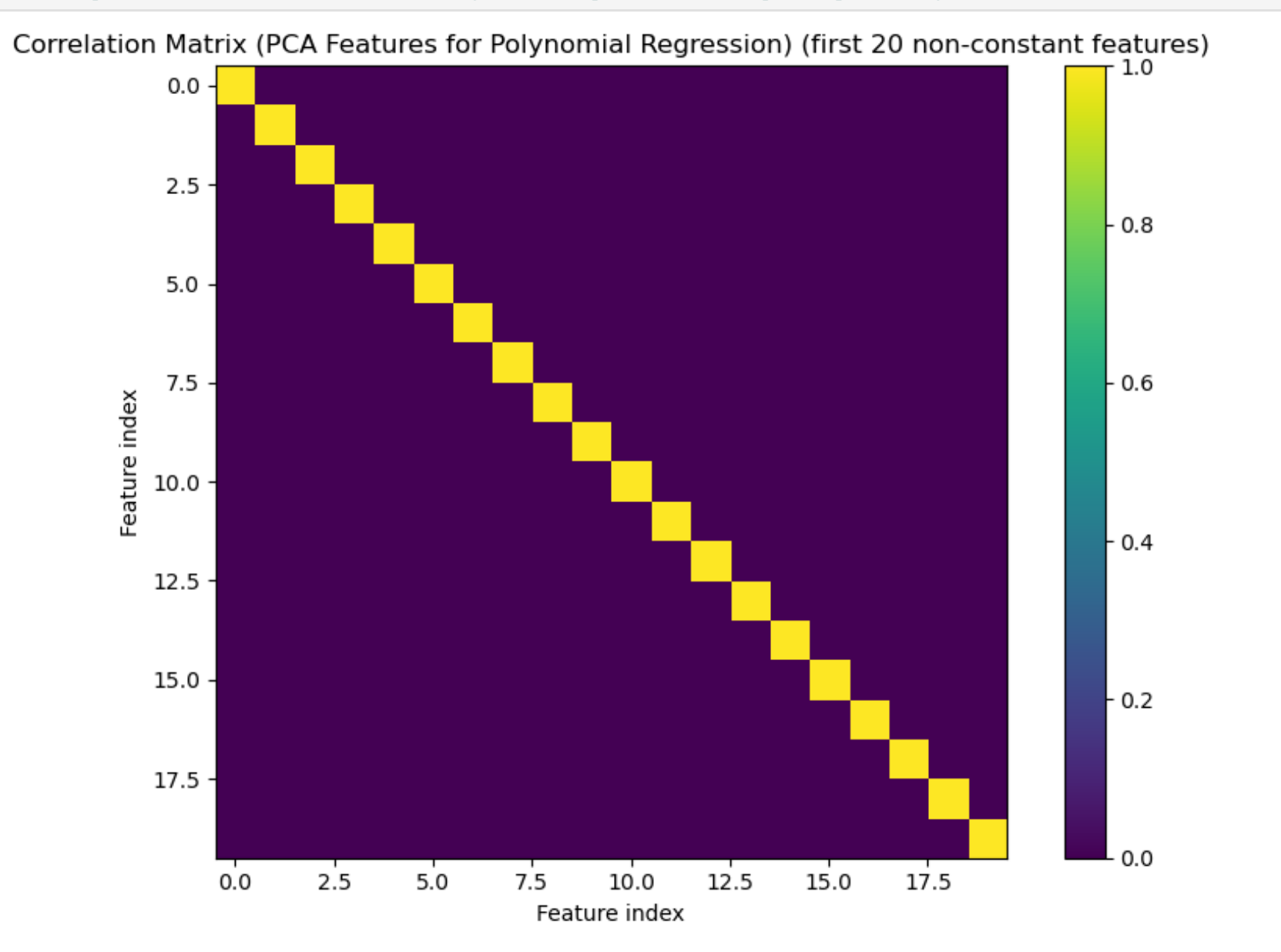
```
# ---------------------------
# Cell purpose:
# Implement feature standardization using NumPy only.
# Standardization transforms each feature column to have:
# - mean close to 0
# - standard deviation close to 1
#
# This is useful for gradient-based training (stability) and for distance-based methods.
# The standardization parameters must be learned from training data only.
# ---------------------------

def fit_standardizer_numpy(X, eps=1e-12):
    # Converts input to a float64 NumPy array for stable numeric operations.
    X = np.asarray(X, dtype=np.float64)

    mu = X.mean(axis=0)
    # Computes the mean for each feature column.
    # mu shape: (num_features,)

    sigma = X.std(axis=0)
    # Computes the standard deviation for each feature column.
    # sigma shape: (num_features,)

    sigma = np.where(sigma < eps, 1.0, sigma)  # protect zero-variance
    # Replaces very small standard deviations with 1.0.
    # This prevents division by zero (or near-zero) during standardization.
    # Zero-variance features carry no information, so scaling them by 1.0 is safe.

    return mu, sigma
    # Returns the parameters needed to standardize new data consistently.


def transform_standardizer_numpy(X, mu, sigma):
    # Converts input to float64 to match the types used for mu and sigma.
    X = np.asarray(X, dtype=np.float64)

    return (X - mu) / sigma
    # Standardizes features using the provided mean and standard deviation.
    # Subtracting mu centers each feature around 0.
    # Dividing by sigma scales each feature to roughly unit variance.
    # Using the same mu/sigma for train and test ensures consistent scaling.
```

In [20]:

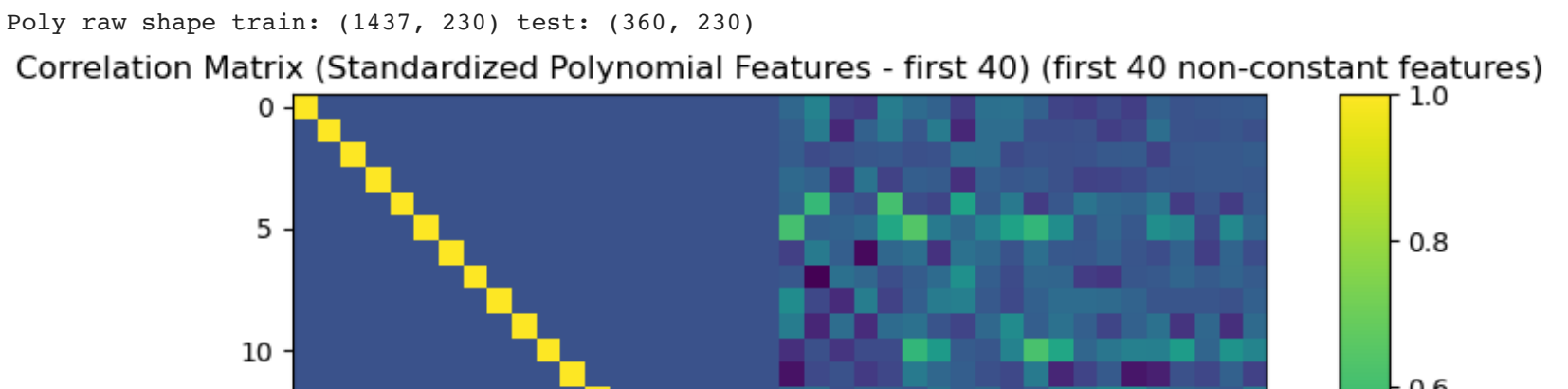
```
# ---------------------------
# Cell purpose:
# Create degree-2 polynomial features using NumPy only.
# This expands the feature space to allow the linear model
# to capture simple non-linear relationships.
#
# Feature expansion structure:
#   - original features:        x
#   - squared features:         x²
#   - pairwise interactions:    x_i * x_j  for i < j
#
# Important note:
# Polynomial expansion grows very fast with dimensionality.
# This function is intended to be used on PCA-reduced features
# (for example 10–30 dimensions) rather than raw pixel features.
# ---------------------------

def poly_features_degree2_numpy(X):
    """
    Degree-2 polynomial expansion:
      [x, x^2, x_i*x_j for i<j]
    Best used on PCA features (e.g., 10–30 dims) to avoid huge feature counts.
    """
    X = np.asarray(X, dtype=np.float64)
    # Converts input to a float64 NumPy array to ensure stable arithmetic.

    n, d = X.shape
    # n = number of samples
    # d = number of input features

    parts = [X, X**2]
    # Starts the feature list with:
    # 1) original features (linear terms)
    # 2) squared features (quadratic terms for each feature)

    inter = []
    # Will store pairwise interaction terms between different features.

    for i in range(d):
        # Loops over feature indices for the first feature in each pair.
        for j in range(i + 1, d):
            # Loops over feature indices greater than i to avoid duplicates.
            # This ensures x_i * x_j is included once, not twice.

            inter.append((X[:, i] * X[:, j]).reshape(n, 1))
            # Computes the interaction term for features i and j.
            # reshape(n, 1) keeps the result as a column vector
            # so all features can be concatenated cleanly.

    if inter:
        # Checks whether any interaction terms were created.
        parts.append(np.hstack(inter))
        # Stacks all interaction columns side-by-side
        # and adds them to the list of feature blocks.

    return np.hstack(parts)
    # Concatenates all feature blocks into a single expanded feature matrix.
    # Final output shape:
    #   [n samples, d + d + (d*(d-1)/2) features]
```

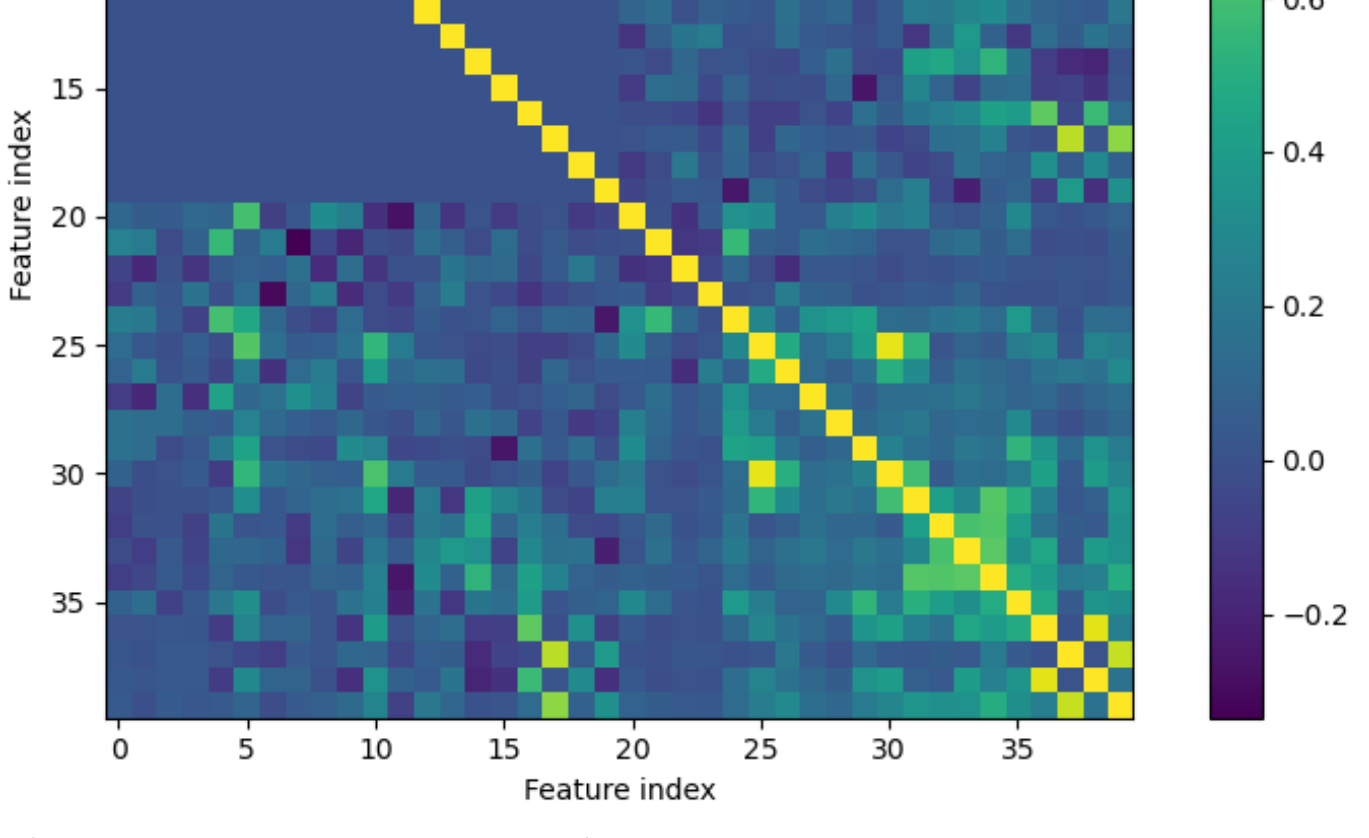
In [21]:

```
# ===========================================================
# Polynomial Regression (from scratch) - Degree 2 on PCA features
# CORRECTED: standardize polynomial features + safer hyperparameters
# ===========================================================
# Cell purpose:
# Train a degree-2 polynomial regression classifier from scratch.
# Polynomial regression here means:
# - features are expanded to include squares and pairwise interactions
# - a linear model is trained on this expanded feature set
#
# Why PCA first:
# Polynomial expansion grows quickly with the number of input features.
# Using raw pixels (64 dims) would create an extremely large feature space.
# PCA reduces dimensionality first, making degree-2 expansion manageable.
#
# Why standardization after expansion:
# Polynomial features can have very different scales (x, x^2, x_i*x_j).
# Standardizing them improves gradient descent stability and convergence.
# ===========================================================

# 1) Base features: PCA
X_train_base = X_train_pca
# Uses PCA training features as the base input to the polynomial feature generator.

X_test_base  = X_test_pca
# Uses PCA test features so evaluation is performed in the same transformed space.

plot_correlation_heatmap(
    X_train_base,
    max_features=min(40, X_train_base.shape[1]),
    title="Correlation Matrix (PCA Features for Polynomial Regression)"
)
# Shows correlation structure of the PCA base features.
# PCA features are typically less correlated than raw pixel features.
# The heatmap is limited to the first 40 features for readability.

# 2) Build polynomial features (scratch)
X_train_poly_raw = poly_features_degree2_numpy(X_train_base)
# Expands PCA features into degree-2 polynomial features for training data.
# Output contains: [x, x^2, x_i*x_j for i<j]

X_test_poly_raw  = poly_features_degree2_numpy(X_test_base)
# Expands PCA features into degree-2 polynomial features for test data.
# Must use the same expansion logic as training data.

print("Poly raw shape train:", X_train_poly_raw.shape, "test:", X_test_poly_raw.shape)
# Prints the dimensionality after feature expansion.
# Feature count increases significantly due to interaction terms.

# 3) Standardize polynomial features (scratch, train-only fit)
mu_poly, sigma_poly = fit_standardizer_numpy(X_train_poly_raw)
# Learns per-feature mean and standard deviation from training polynomial features only.
# This avoids test leakage and ensures scaling parameters come only from training data.

X_train_poly = transform_standardizer_numpy(X_train_poly_raw, mu_poly, sigma_poly)
# Applies standardization to training polynomial features:
# (X - mu) / sigma

X_test_poly  = transform_standardizer_numpy(X_test_poly_raw,  mu_poly, sigma_poly)
# Applies the same standardization parameters learned from training data to test data.
# This keeps scaling consistent and avoids using test statistics.

# Optional: correlation heatmap after standardization (first 40)
plot_correlation_heatmap(
    X_train_poly,
    max_features=min(40, X_train_poly.shape[1]),
    title="Correlation Matrix (Standardized Polynomial Features - first 40)"
)
# Visualizes correlation of the standardized polynomial features.
# Standardization does not remove correlation, but it removes scale differences.
# Limiting to the first 40 features prevents the plot from becoming unreadable.

# 4) Train with safer settings
t0 = time.time()
# Records the start time to measure training + prediction runtime.

W_poly, hist_poly = train_linear_regression_multiclass_gd(
    X_train_poly, y_train,
    X_test_poly,  y_test,
    n_classes=10,
    lr=0.01,        # Smaller learning rate helps prevent divergence on large feature spaces.
    epochs=600,     # More epochs allow gradual convergence with a smaller learning rate.
    batch_size=256, # Mini-batch gradient descent for efficiency and stability.
    l2=5e-2,        # Stronger L2 regularization controls complexity in expanded feature space.
    seed=RANDOM_STATE
)
# Trains a multiclass linear model on polynomial features using one-hot MSE.
# Returns:
# - W_poly: learned weights including bias
# - hist_poly: training history used for learning-curve plots

time_poly = time.time() - t0
# Stores total runtime of the training procedure.

# 5) Predict
scores_poly = add_bias_numpy(X_test_poly) @ W_poly
# Computes class scores for test data.
# Bias column is added, then matrix multiplication produces 10 scores per sample.

y_pred_poly = np.argmax(scores_poly, axis=1)
# Converts class scores into predicted labels by taking the index of the maximum score.

# 6) Metrics + CM
print("\nPolynomial Regression deg-2 (scratch, PCA) - Classification Report")
# Prints a header to clearly label the evaluation output.

poly_metrics = classification_report_numpy(y_test, y_pred_poly, n_classes=10)
# Prints per-class precision/recall/F1 and macro/weighted averages.
# The returned dictionary is stored for later comparison.

cm_poly = confusion_matrix_numpy(y_test, y_pred_poly, n_classes=10)
# Builds a confusion matrix for polynomial regression predictions.

print("\nPolynomial Regression deg-2 (scratch, PCA) - Confusion Matrix:\n", cm_poly)
# Prints the raw confusion matrix counts.

print(f"Runtime (fit+predict): {time_poly:.4f} seconds")
# Prints runtime to compare model cost against linear regression and kNN.

# 7) Graphs
plot_learning_curves(hist_poly, "Polynomial Regression Degree-2 (Scratch, PCA)")
# Plots training vs test MSE and training vs test accuracy over epochs.
# Useful for spotting convergence behavior and overfitting patterns.

print("""
INTERPRETATION (Polynomial Regression Degree-2 — Scratch with PCA):

The first plot (MSE vs Epoch) shows the training and test Mean Squared Error during
optimization. While the training MSE remains relatively low and stable, the test
MSE exhibits larger fluctuations. This behavior reflects the increased complexity
of the degree-2 polynomial model, which is more sensitive to noise and data
variations, even when trained on PCA features.

The second plot (Accuracy vs Epoch) shows that both training and test accuracy
increase rapidly and reach high values above 95%. The small gap between training
and test accuracy indicates strong predictive performance, while minor fluctuations
in test accuracy suggest mild overfitting controlled by regularization.

Overall, these plots show that polynomial regression captures more complex
relationships than linear regression, achieving higher accuracy, but at the cost
of less stable test error during training.
""")
# Interpretation text is printed as written and can be used directly in the report.

plot_true_class_score_histogram(
    scores_poly, y_test,
    "Polynomial Regression (Scratch, PCA) - Test True-Class Score Histogram"
)
# Shows the score given to the correct class for each test example.
# Higher values indicate the model places more confidence on the true class.

print("""
INTERPRETATION (True-Class Score Histogram — Polynomial Regression Degree-2 with PCA):

This histogram shows the distribution of predicted scores assigned to the correct
(true) class by the polynomial regression model on the test set.

Most scores are concentrated in the higher range, indicating that the model assigns
strong confidence to the correct class for the majority of test samples. Compared
to linear regression, the scores are more tightly clustered toward higher values,
reflecting the increased expressive power of the polynomial model.

A small number of low or negative scores correspond to misclassified or uncertain
samples. These occur infrequently, which is consistent with the model's high test
accuracy.

Overall, this plot demonstrates that polynomial regression produces more confident
correct-class predictions than the linear model when operating in PCA feature space.
""")
# Interpretation text explains how to read the histogram in report language.

plot_residual_histogram(
    scores_poly, y_test, n_classes=10,
    title="Polynomial Regression (Scratch, PCA) - Test Residual Norms"
)
# Plots norm of residual vectors: one_hot(y) - scores.
# Residual norms close to zero indicate predicted score vectors close to the ideal target.

print("""
INTERPRETATION (Residual Norms — Polynomial Regression Degree-2 with PCA):

This histogram shows the distribution of residual norms on the test set, where the
residual measures the difference between the true one-hot label vector and the
predicted score vector.

Most residuals are concentrated very close to zero, indicating that for the majority
of test samples, the model's predictions closely match the true class encoding.
This reflects highly confident and accurate predictions.

A small number of very large residuals appear as outliers. These correspond to
misclassified or highly uncertain samples, where the polynomial model produces
incorrect or unstable predictions. Such outliers are expected due to the increased
model complexity of polynomial regression.

Overall, this plot shows that while the polynomial model achieves strong performance
for most samples, it can produce occasional large errors, highlighting a trade-off
between higher accuracy and prediction stability.
""")
# Interpretation text describes what residual norm distribution suggests about model behavior.

plot_weight_heatmap(W_poly, "Polynomial Regression (Scratch, PCA) - Weight Heatmap (no bias)")
# Displays the learned weight matrix (excluding bias) as a heatmap.
# For polynomial regression, rows correspond to polynomial feature indices rather than original PCA indices.

print("""
INTERPRETATION (Weight Heatmap — Polynomial Regression Degree-2 with PCA):

This heatmap shows the learned weight matrix of the degree-2 polynomial regression
model trained on PCA features, excluding the bias term. Each row corresponds to a
polynomial feature derived from PCA components, and each column corresponds to a
class.

The dense and varied color patterns indicate that the model uses a large number of
features, including interaction terms, to form class-specific predictions. Compared
to the linear model, the weight structure is more complex, reflecting the increased
expressive power of polynomial regression.

Most weights remain within a moderate range, suggesting that regularization helps
control model complexity. However, the presence of stronger localized weights
highlights how the model captures non-linear relationships in the PCA feature space.

Overall, this plot illustrates how polynomial regression leverages higher-order
feature interactions to achieve higher accuracy, at the cost of reduced
interpretability compared to the linear model.
""")
# Interpretation text connects the weight heatmap to model complexity and regularization.
```

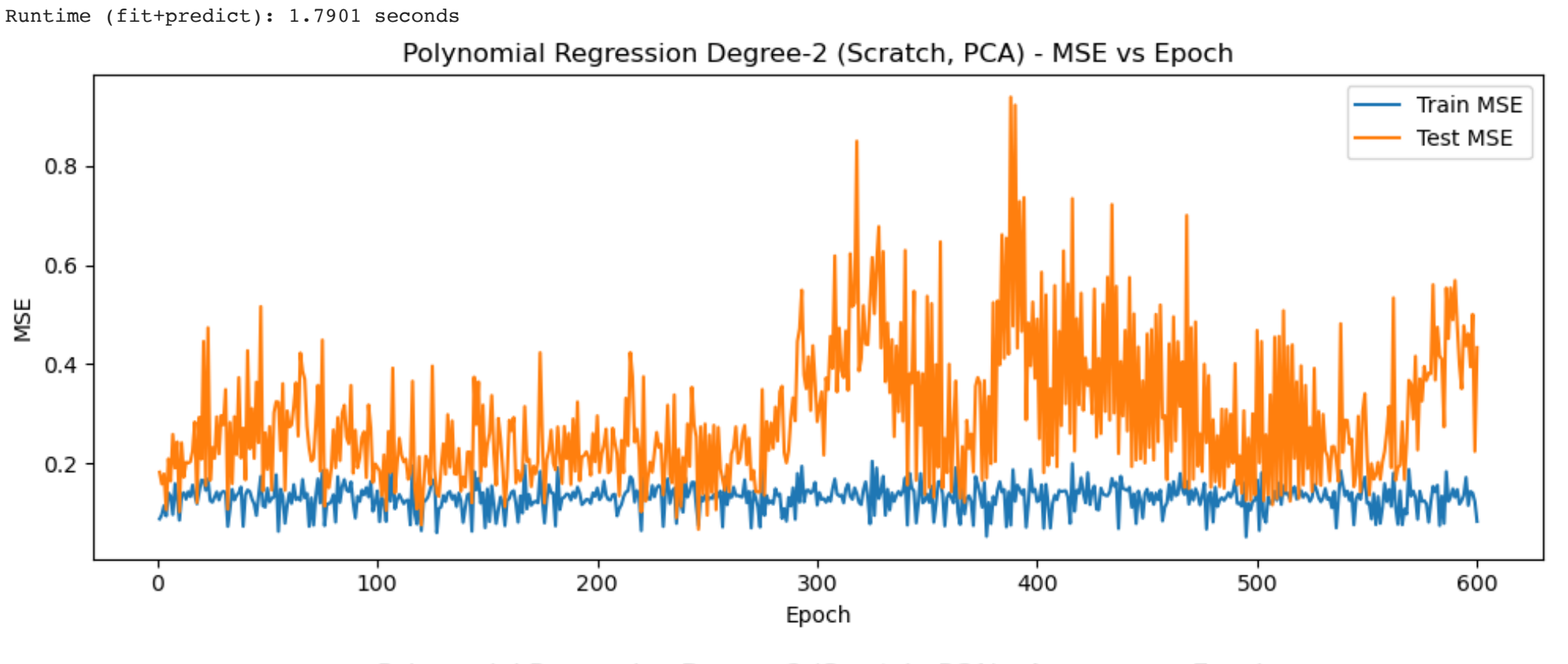


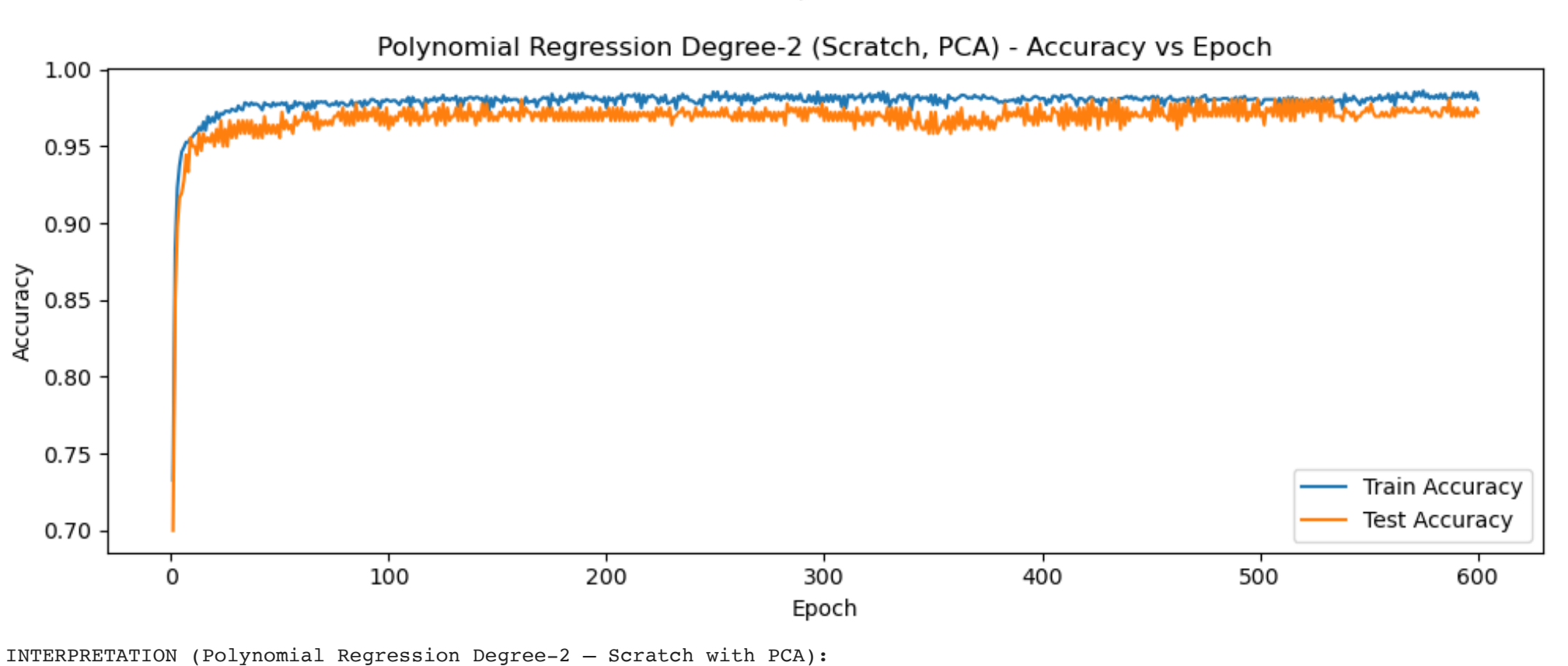
```
Poly raw shape train: (1437, 230) test: (360, 230)
```



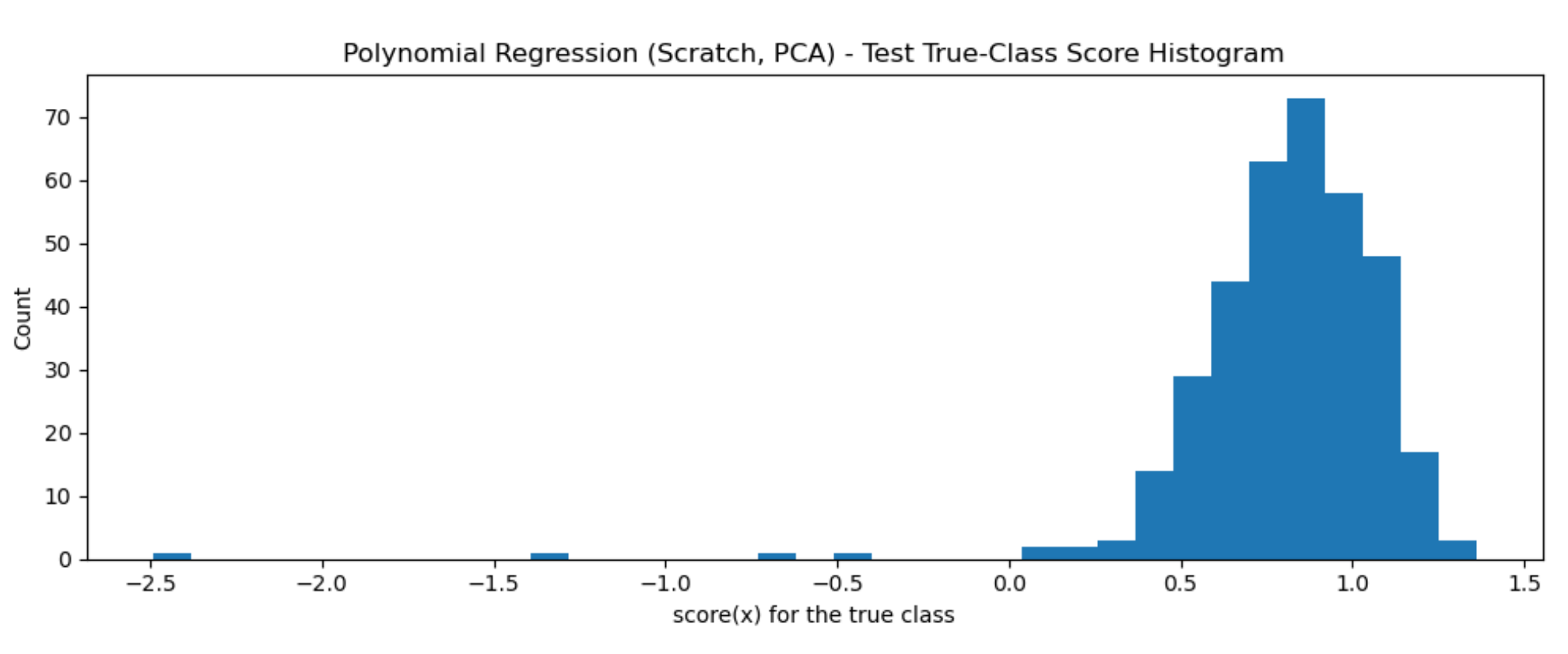


```
Epoch  25/600 | Train MSE 0.1212 Eval MSE 0.2315 | Train Acc 0.9729 Eval Acc 0.9611
Epoch  50/600 | Train MSE 0.1492 Eval MSE 0.1876 | Train Acc 0.9770 Eval Acc 0.9556
Epoch  75/600 | Train MSE 0.1851 Eval MSE 0.4485 | Train Acc 0.9770 Eval Acc 0.9694
Epoch 100/600 | Train MSE 0.1439 Eval MSE 0.1920 | Train Acc 0.9798 Eval Acc 0.9611
Epoch 125/600 | Train MSE 0.1657 Eval MSE 0.3960 | Train Acc 0.9833 Eval Acc 0.9778
Epoch 150/600 | Train MSE 0.1479 Eval MSE 0.2376 | Train Acc 0.9805 Eval Acc 0.9667
Epoch 175/600 | Train MSE 0.1277 Eval MSE 0.2527 | Train Acc 0.9840 Eval Acc 0.9750
Epoch 200/600 | Train MSE 0.1501 Eval MSE 0.2956 | Train Acc 0.9749 Eval Acc 0.9694
Epoch 225/600 | Train MSE 0.1549 Eval MSE 0.1819 | Train Acc 0.9819 Eval Acc 0.9750
Epoch 250/600 | Train MSE 0.1248 Eval MSE 0.0950 | Train Acc 0.9819 Eval Acc 0.9694
Epoch 275/600 | Train MSE 0.1867 Eval MSE 0.3487 | Train Acc 0.9840 Eval Acc 0.9722
Epoch 300/600 | Train MSE 0.1155 Eval MSE 0.2838 | Train Acc 0.9791 Eval Acc 0.9667
Epoch 325/600 | Train MSE 0.2039 Eval MSE 0.6151 | Train Acc 0.9847 Eval Acc 0.9722
Epoch 350/600 | Train MSE 0.1374 Eval MSE 0.5369 | Train Acc 0.9770 Eval Acc 0.9583
Epoch 375/600 | Train MSE 0.1112 Eval MSE 0.1657 | Train Acc 0.9812 Eval Acc 0.9694
Epoch 400/600 | Train MSE 0.1222 Eval MSE 0.4909 | Train Acc 0.9812 Eval Acc 0.9667
Epoch 425/600 | Train MSE 0.1430 Eval MSE 0.3000 | Train Acc 0.9791 Eval Acc 0.9750
Epoch 450/600 | Train MSE 0.1475 Eval MSE 0.4607 | Train Acc 0.9819 Eval Acc 0.9667
Epoch 475/600 | Train MSE 0.1328 Eval MSE 0.1830 | Train Acc 0.9833 Eval Acc 0.9778
Epoch 500/600 | Train MSE 0.1660 Eval MSE 0.4679 | Train Acc 0.9770 Eval Acc 0.9694
Epoch 525/600 | Train MSE 0.1312 Eval MSE 0.1429 | Train Acc 0.9798 Eval Acc 0.9806
Epoch 550/600 | Train MSE 0.1189 Eval MSE 0.2152 | Train Acc 0.9805 Eval Acc 0.9722
Epoch 575/600 | Train MSE 0.1230 Eval MSE 0.3254 | Train Acc 0.9861 Eval Acc 0.9750
Epoch 600/600 | Train MSE 0.0825 Eval MSE 0.4333 | Train Acc 0.9805 Eval Acc 0.9722

Polynomial Regression deg-2 (scratch, PCA) - Classification Report
Class | Precision | Recall | F1-score | Support
----------------------------------------------
    0 |    0.9231 | 1.0000 |   0.9600 |      36
    1 |    0.8974 | 0.9722 |   0.9333 |      36
    2 |    0.9722 | 1.0000 |   0.9859 |      35
    3 |    1.0000 | 0.9730 |   0.9863 |      37
    4 |    0.9714 | 0.9444 |   0.9577 |      36
    5 |    1.0000 | 1.0000 |   1.0000 |      37
    6 |    1.0000 | 1.0000 |   1.0000 |      36
    7 |    0.9730 | 1.0000 |   0.9863 |      36
    8 |    1.0000 | 0.8571 |   0.9231 |      35
    9 |    1.0000 | 0.9722 |   0.9859 |      36
----------------------------------------------
Accuracy: 0.9722
Macro avg     P/R/F1: 0.9737 / 0.9719 / 0.9719
Weighted avg  P/R/F1: 0.9738 / 0.9722 / 0.9721

Polynomial Regression deg-2 (scratch, PCA) - Confusion Matrix:
 [[36  0  0  0  0  0  0  0  0  0]
 [ 1 35  0  0  0  0  0  0  0  0]
 [ 0  0 35  0  0  0  0  0  0  0]
 [ 0  0  1 36  0  0  0  0  0  0]
 [ 2  0  0  0 34  0  0  0  0  0]
 [ 0  0  0  0  0 37  0  0  0  0]
 [ 0  0  0  0  0  0 36  0  0  0]
 [ 0  0  0  0  0  0  0 36  0  0]
 [ 0  4  0  0  1  0  0  0 30  0]
 [ 0  0  0  0  0  0  0  1  0 35]]
Runtime (fit+predict): 1.7901 seconds
```

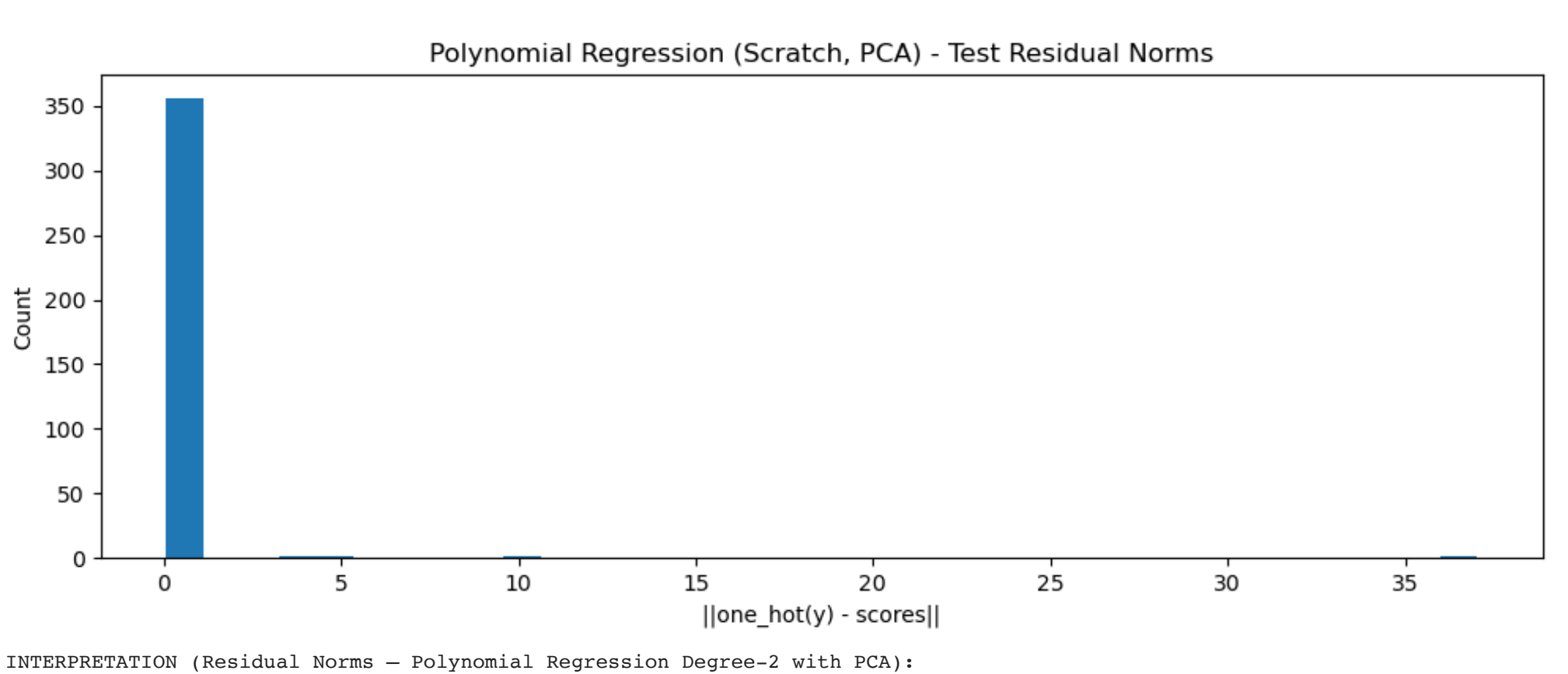




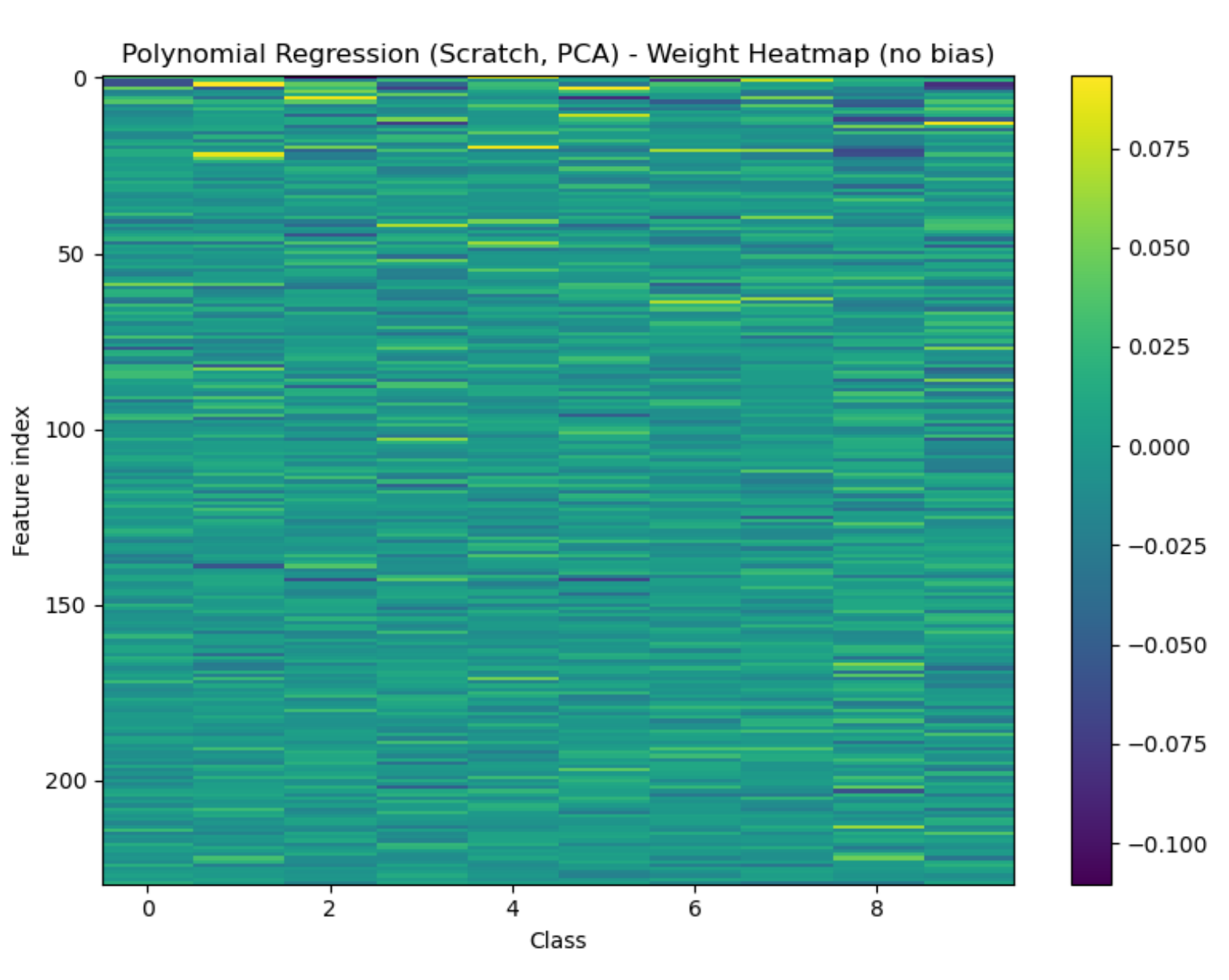
```
INTERPRETATION (Polynomial Regression Degree-2 — Scratch with PCA):

The first plot (MSE vs Epoch) shows the training and test Mean Squared Error during
optimization. While the training MSE remains relatively low and stable, the test
MSE exhibits larger fluctuations. This behavior reflects the increased complexity
of the degree-2 polynomial model, which is more sensitive to noise and data
variations, even when trained on PCA features.

The second plot (Accuracy vs Epoch) shows that both training and test accuracy
increase rapidly and reach high values above 95%. The small gap between training
and test accuracy indicates strong predictive performance, while minor fluctuations
in test accuracy suggest mild overfitting controlled by regularization.

Overall, these plots show that polynomial regression captures more complex
relationships than linear regression, achieving higher accuracy, but at the cost
of less stable test error during training.
```



```
INTERPRETATION (True-Class Score Histogram — Polynomial Regression Degree-2 with PCA):

This histogram shows the distribution of predicted scores assigned to the correct
(true) class by the polynomial regression model on the test set.

Most scores are concentrated in the higher range, indicating that the model assigns
strong confidence to the correct class for the majority of test samples. Compared
to linear regression, the scores are more tightly clustered toward higher values,
reflecting the increased expressive power of the polynomial model.

A small number of low or negative scores correspond to misclassified or uncertain
samples. These occur infrequently, which is consistent with the model's high test
accuracy.

Overall, this plot demonstrates that polynomial regression produces more confident
correct-class predictions than the linear model when operating in PCA feature space.
```



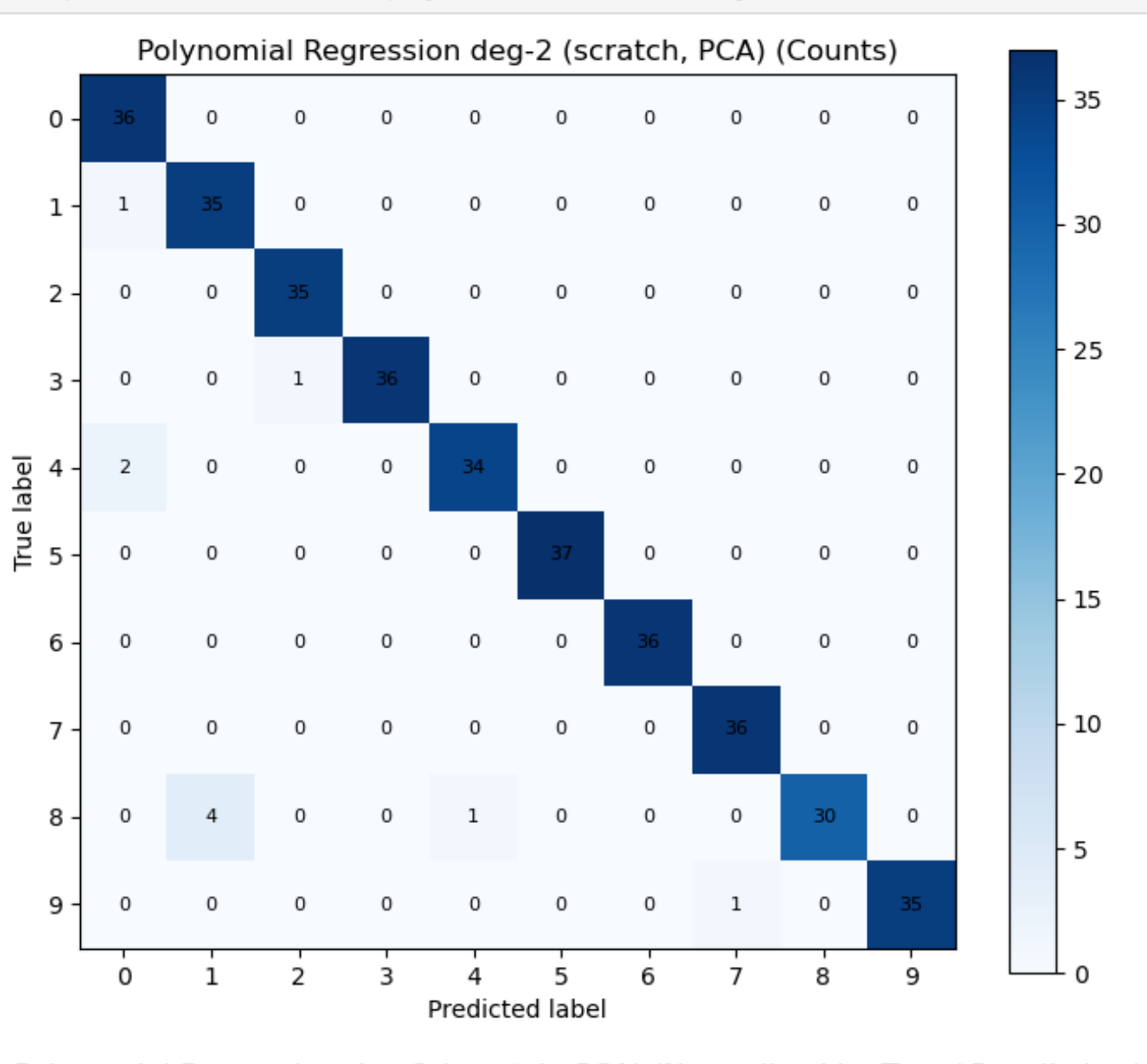
```
INTERPRETATION (Residual Norms — Polynomial Regression Degree-2 with PCA):

This histogram shows the distribution of residual norms on the test set, where the
residual measures the difference between the true one-hot label vector and the
predicted score vector.

Most residuals are concentrated very close to zero, indicating that for the majority
of test samples, the model's predictions closely match the true class encoding.
This reflects highly confident and accurate predictions.

A small number of very large residuals appear as outliers. These correspond to
misclassified or highly uncertain samples, where the polynomial model produces
incorrect or unstable predictions. Such outliers are expected due to the increased
model complexity of polynomial regression.

Overall, this plot shows that while the polynomial model achieves strong performance
for most samples, it can produce occasional large errors, highlighting a trade-off
between higher accuracy and prediction stability.
```



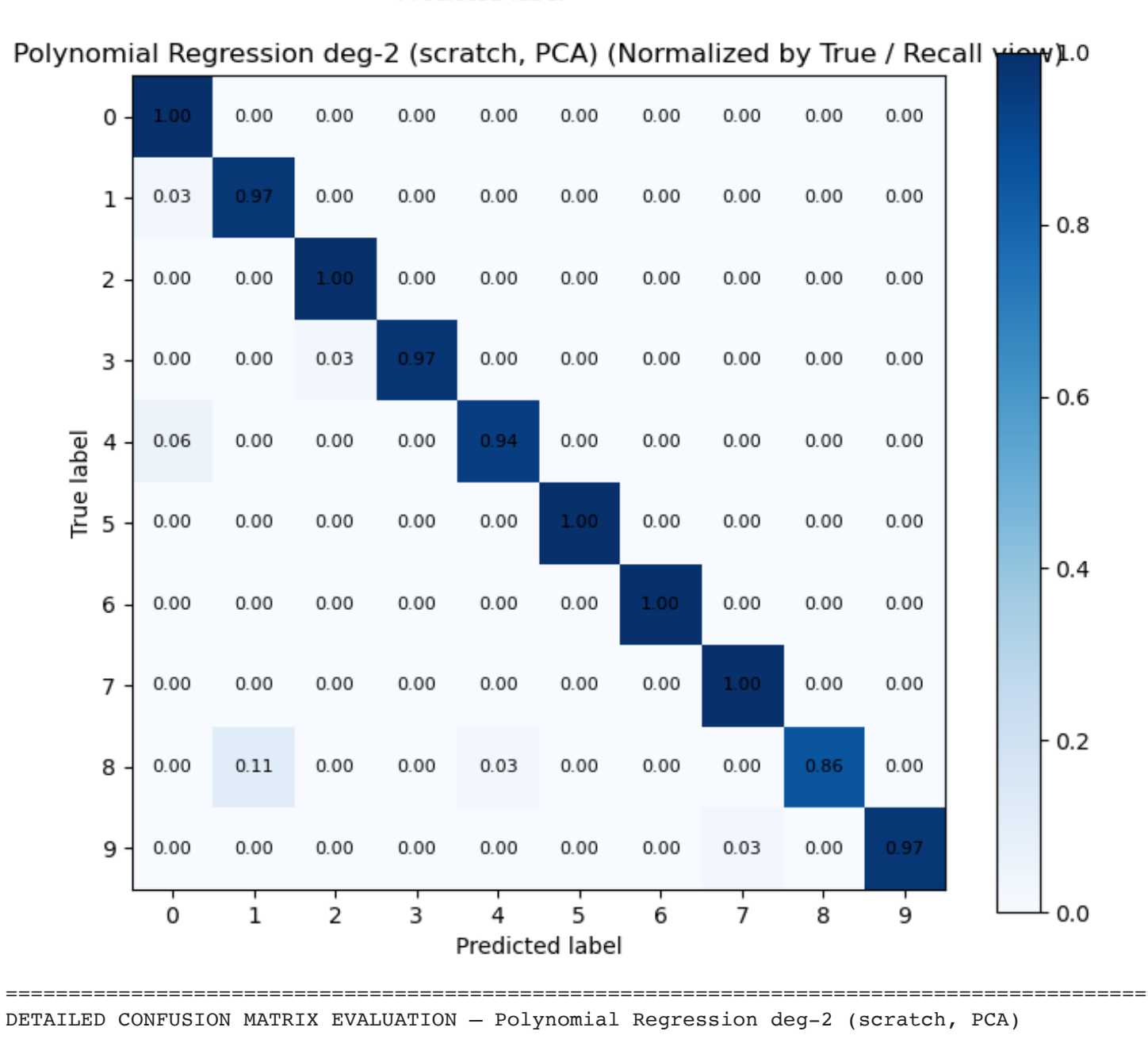
```
INTERPRETATION (Weight Heatmap — Polynomial Regression Degree-2 with PCA):

This heatmap shows the learned weight matrix of the degree-2 polynomial regression
model trained on PCA features, excluding the bias term. Each row corresponds to a
polynomial feature derived from PCA components, and each column corresponds to a
class.

The dense and varied color patterns indicate that the model uses a large number of
features, including interaction terms, to form class-specific predictions. Compared
to the linear model, the weight structure is more complex, reflecting the increased
expressive power of polynomial regression.

Most weights remain within a moderate range, suggesting that regularization helps
control model complexity. However, the presence of stronger localized weights
highlights how the model captures non-linear relationships in the PCA feature space.

Overall, this plot illustrates how polynomial regression leverages higher-order
feature interactions to achieve higher accuracy, at the cost of reduced
interpretability compared to the linear model.
```

In [22]:

```python
# ---------------------------
# Cell purpose:
# Visualize and deeply analyze the confusion matrix for the
# degree-2 polynomial regression model trained on PCA features.
# This makes it easier to describe class-level performance and common errors.
# ---------------------------

cm_poly = confusion_matrix_numpy(y_test, y_pred_poly, n_classes=10)
# Builds the confusion matrix for polynomial regression predictions.
# Rows represent true digit labels.
# Columns represent predicted digit labels.
# Each entry (i, j) counts how many true-i samples were predicted as j.

plot_confusion_matrix_individual(cm_poly, "Polynomial Regression deg-2 (scratch, PCA)", normalize=None)
# Plots the confusion matrix as raw counts.
# Diagonal cells show correct predictions.
# Off-diagonal cells show specific misclassifications.

plot_confusion_matrix_individual(cm_poly, "Polynomial Regression deg-2 (scratch, PCA)", normalize="true")
# Plots the confusion matrix normalized by true label (row-normalized).
# Each row sums to 1, so values represent proportions.
# This view highlights recall behavior: how often each true class is recovered.

deep_eval_from_confusion_matrix(cm_poly, "Polynomial Regression deg-2 (scratch, PCA)")
# Prints a detailed, report-style evaluation based on the confusion matrix.
# Includes:
# - accuracy from diagonal/total
# - macro precision/recall/F1 and weighted F1
# - TP/FP/FN/TN per class using one-vs-rest interpretation
# - weakest classes by recall and precision
# - most frequent confusion pairs (true digit → predicted digit)
# - guidance text for writing up confusion matrix analysis
```


Polynomial Regression deg-2 (scratch, PCA) (Counts)

True label	0	1	2	3	4	5	6	7	8	9
0	36	0	0	0	0	0	0	0	0	0
1	1	35	0	0	0	0	0	0	0	0
2	0	0	35	0	0	0	0	0	0	0
3	0	0	1	36	0	0	0	0	0	0
4	2	0	0	0	34	0	0	0	0	0
5	0	0	0	0	0	37	0	0	0	0
6	0	0	0	0	0	0	36	0	0	0
7	0	0	0	0	0	0	0	36	0	0
8	0	4	0	0	1	0	0	0	30	0
9	0	0	0	0	0	0	0	1	0	35


Polynomial Regression deg-2 (scratch, PCA) (Normalized by True / Recall view)

True label	0	1	2	3	4	5	6	7	8	9
0	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1	0.03	0.97	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
2	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
3	0.00	0.00	0.03	0.97	0.00	0.00	0.00	0.00	0.00	0.00
4	0.06	0.00	0.00	0.00	0.94	0.00	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00
6	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00
7	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
8	0.00	0.11	0.00	0.00	0.03	0.00	0.00	0.00	0.86	0.00
9	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03	0.00	0.97

```
==========================================================================================
DETAILED CONFUSION MATRIX EVALUATION — Polynomial Regression deg-2 (scratch, PCA)
==========================================================================================
Interpretation (one-vs-rest for each class c):
  TP(c): predicted c AND true c (diagonal cell for c)
  FP(c): predicted c BUT true was not c (other rows in column c)
  FN(c): true c BUT predicted not c (other columns in row c)
  TN(c): not c AND predicted not c (everything not in row/col c)
------------------------------------------------------------------------------------------
Overall Accuracy = (sum diagonal) / total = 350/360 = 0.9722
Macro avg (P/R/F1): 0.9737 / 0.9719 / 0.9719
Weighted F1       : 0.9721
------------------------------------------------------------------------------------------
Class |   TP   FP   FN   TN | Precision  Recall    F1    | Support
------------------------------------------------------------------------------------------
    0 |   36    3    0  321 |     0.923   1.000   0.960 |      36
    1 |   35    4    1  320 |     0.897   0.972   0.933 |      36
    2 |   35    1    0  324 |     0.972   1.000   0.986 |      35
    3 |   36    0    1  323 |     1.000   0.973   0.986 |      37
    4 |   34    1    2  323 |     0.971   0.944   0.958 |      36
    5 |   37    0    0  323 |     1.000   1.000   1.000 |      37
    6 |   36    0    0  324 |     1.000   1.000   1.000 |      36
    7 |   36    1    0  323 |     0.973   1.000   0.986 |      36
    8 |   30    0    5  325 |     1.000   0.857   0.923 |      35
    9 |   35    0    1  324 |     1.000   0.972   0.986 |      36
------------------------------------------------------------------------------------------
Worst classes by Recall (high FN → true class often missed):
  Class 8: Recall=0.857 | FN=5 | Support=35
  Class 4: Recall=0.944 | FN=2 | Support=36
  Class 1: Recall=0.972 | FN=1 | Support=36
Worst classes by Precision (high FP → many false alarms for that predicted class):
  Class 1: Precision=0.897 | FP=4 | Predicted=39
  Class 0: Precision=0.923 | FP=3 | Predicted=39
  Class 4: Precision=0.971 | FP=1 | Predicted=35
------------------------------------------------------------------------------------------
Top 6 confusions (True → Predicted):
  8 → 1: 4 cases (11.43% of true class 8)
  4 → 0: 2 cases (5.56% of true class 4)
  1 → 0: 1 cases (2.78% of true class 1)
  3 → 2: 1 cases (2.70% of true class 3)
  8 → 4: 1 cases (2.86% of true class 8)
  9 → 7: 1 cases (2.78% of true class 9)
```

PCA + kNN (library PCA, scratch kNN)

PCA reduces the feature space before running the same kNN code.

In [23]:

```python
# ---------------------------
# Cell purpose:
# Apply Principal Component Analysis (PCA) to reduce the feature space.
# PCA compresses the original 64 pixel features into a smaller number of components
# while keeping as much variance (information) as possible.
#
# Key rule:
# PCA must be fitted on training data only to avoid test data leakage.
# The fitted transformation is then applied to the test set.
# ---------------------------

n_components = 20
# Sets the number of principal components to keep.
# This reduces dimensionality from 64 features down to 20.

pca = PCA(n_components=n_components, random_state=RANDOM_STATE)
# Creates a PCA object configured to keep 20 components.
# random_state is used for reproducibility in parts of the PCA solver.

X_train_pca = pca.fit_transform(X_train_scaled)
# Learns PCA directions (principal components) from the training data,
# then projects the training data into the new 20-dimensional PCA space.

X_test_pca = pca.transform(X_test_scaled)
# Projects the test data using the PCA transformation learned from training data.
# No fitting happens here, which prevents information leakage.

print("Original features:", X_train_scaled.shape[1])
# Displays the original number of features (64 pixels per image).

print("PCA features     :", X_train_pca.shape[1])
# Displays the reduced number of features after PCA (20 components).

print("Explained variance (sum):", round(float(pca.explained_variance_ratio_.sum()), 4))
# Shows how much of the dataset variance is preserved by the selected components.
# A higher value means more information is retained after compression.
```

```
Original features: 64
PCA features     : 20
Explained variance (sum): 0.7898
```

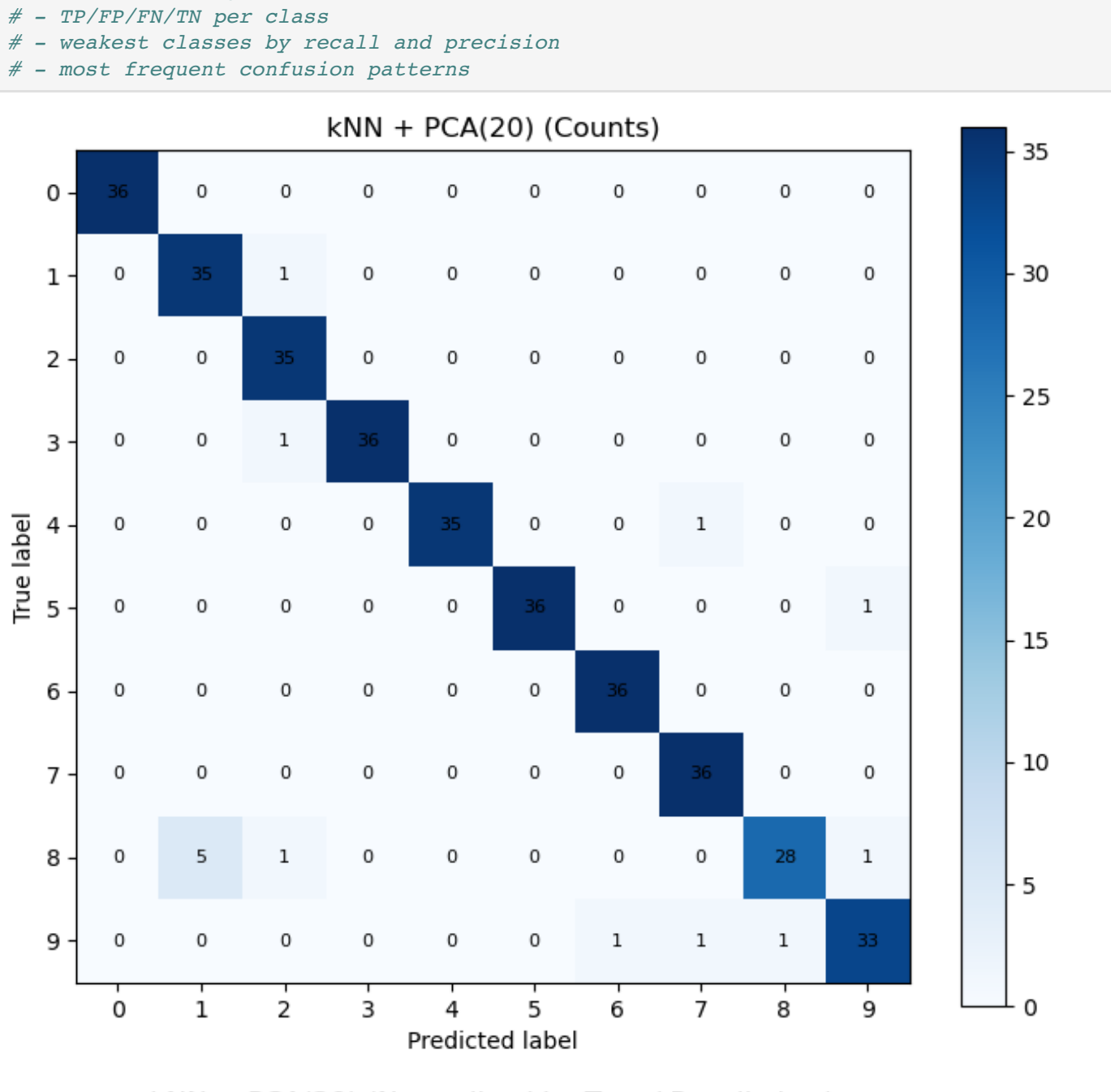
In [24]:

```python
# ---------------------------
# Cell purpose:
# Evaluate the from-scratch kNN classifier using PCA-reduced features.
# This compares performance and runtime against kNN on the full feature space.
# ---------------------------

t0 = time.time()
# Records the start time to measure prediction runtime.

y_pred_knn_pca = knn_predict(X_test_pca, X_train_pca, y_train, k=k)
# Runs kNN using PCA-transformed features.
# X_train_pca acts as the reference set of neighbors.
# X_test_pca contains test samples projected into the same PCA space.
# k controls how many nearest neighbors participate in voting.

time_knn_pca = time.time() - t0
# Computes total time taken for kNN prediction using PCA features.

acc_knn_pca, p_knn_pca, r_knn_pca, f1_knn_pca = print_metrics_block(
    f"kNN (from scratch) + PCA({n_components})",
    y_test, y_pred_knn_pca, n_classes=10
)
# Prints accuracy and macro-averaged precision, recall, and F1-score.
# Returned values are stored to allow direct comparison with other models.

cm_knn_pca = confusion_matrix_numpy(y_test, y_pred_knn_pca, n_classes=10)
# Builds the confusion matrix for kNN predictions made in PCA space.

print(f"Runtime (predict only): {time_knn_pca:.4f} seconds")
# Displays prediction runtime.
# PCA typically reduces runtime because distance computations occur in fewer dimensions.
```

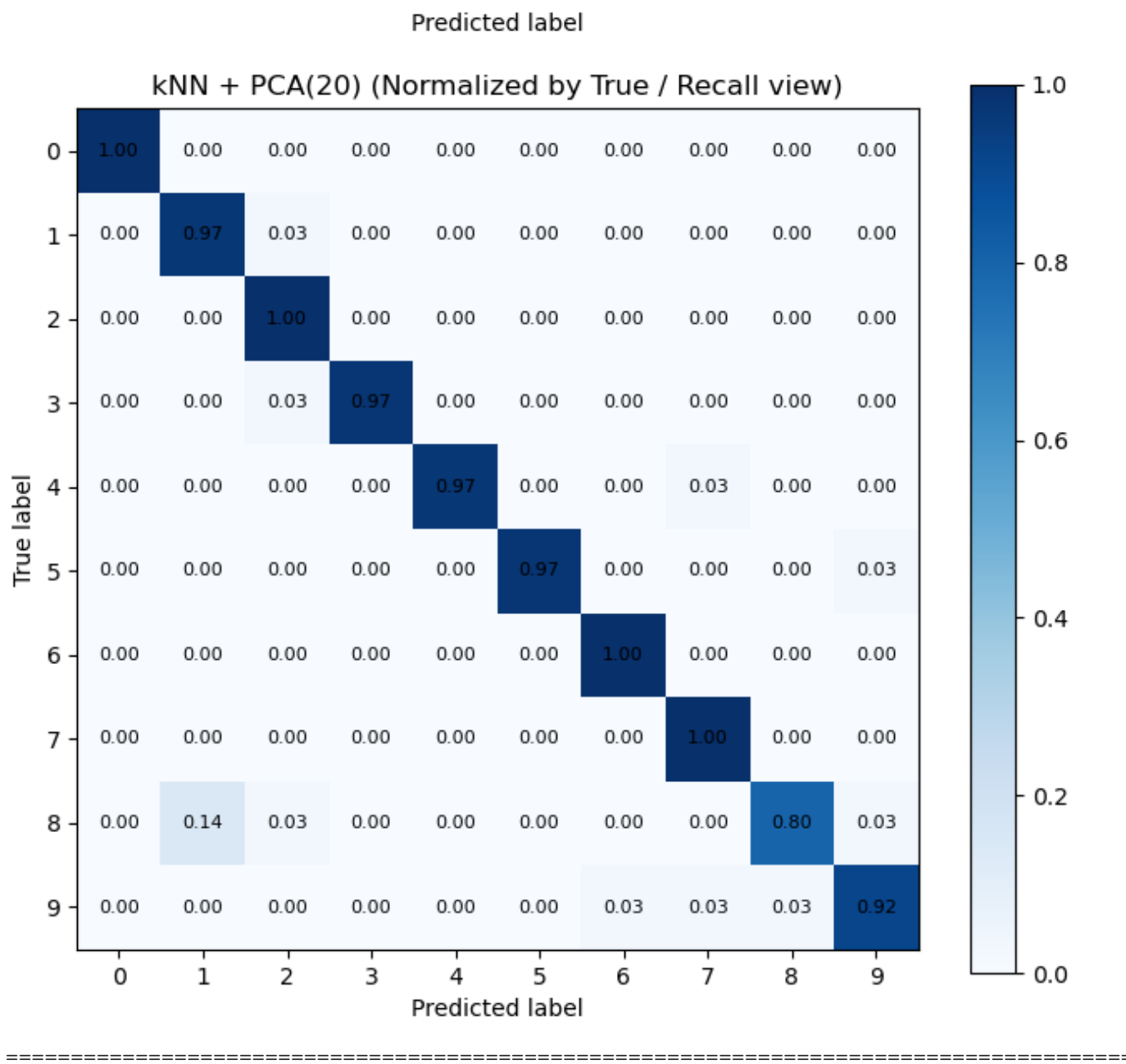
```
============================================================
kNN (from scratch) + PCA(20)
============================================================
Accuracy : 0.9611
Precision: 0.9625  (macro)
Recall   : 0.9607  (macro)
F1-score : 0.9602  (macro)
============================================================
Runtime (predict only): 0.1082 seconds
```

In [25]:

```python
# ---------------------------
# Cell purpose:
# Visualize and deeply analyze the confusion matrix for the
# from-scratch kNN classifier when applied to PCA-reduced features.
# This highlights how dimensionality reduction affects class-wise performance.
# ---------------------------

cm_knn_pca = confusion_matrix_numpy(y_test, y_pred_knn_pca, n_classes=10)
# Builds the confusion matrix for kNN predictions made in PCA space.
# Rows represent true digit labels.
# Columns represent predicted digit labels.

plot_confusion_matrix_individual(
    cm_knn_pca,
    f"kNN + PCA({n_components})",
    normalize=None
)
# Displays the confusion matrix using raw counts.
# Diagonal values show correct predictions.
# Off-diagonal values reveal specific misclassifications.

plot_confusion_matrix_individual(
    cm_knn_pca,
    f"kNN + PCA({n_components})",
    normalize="true"
)
# Displays the confusion matrix normalized by true class.
# Each row sums to 1, making recall differences across digits easier to interpret.

deep_eval_from_confusion_matrix(
    cm_knn_pca,
    f"kNN + PCA({n_components})"
)
# Prints a detailed, report-ready evaluation derived from the confusion matrix.
# Includes:
# - overall accuracy
# - macro and weighted F1 scores
# - TP/FP/FN/TN per class
# - weakest classes by recall and precision
# - most frequent confusion patterns
```


kNN + PCA(20) (Counts)

True label	0	1	2	3	4	5	6	7	8	9
0	36	0	0	0	0	0	0	0	0	0
1	0	35	1	0	0	0	0	0	0	0
2	0	0	35	0	0	0	0	0	0	0
3	0	0	1	36	0	0	0	0	0	0
4	0	0	0	0	35	0	0	1	0	0
5	0	0	0	0	0	36	0	0	0	1
6	0	0	0	0	0	0	36	0	0	0
7	0	0	0	0	0	0	0	36	0	0
8	0	5	1	0	0	0	0	0	28	1
9	0	0	0	0	0	0	1	1	1	33

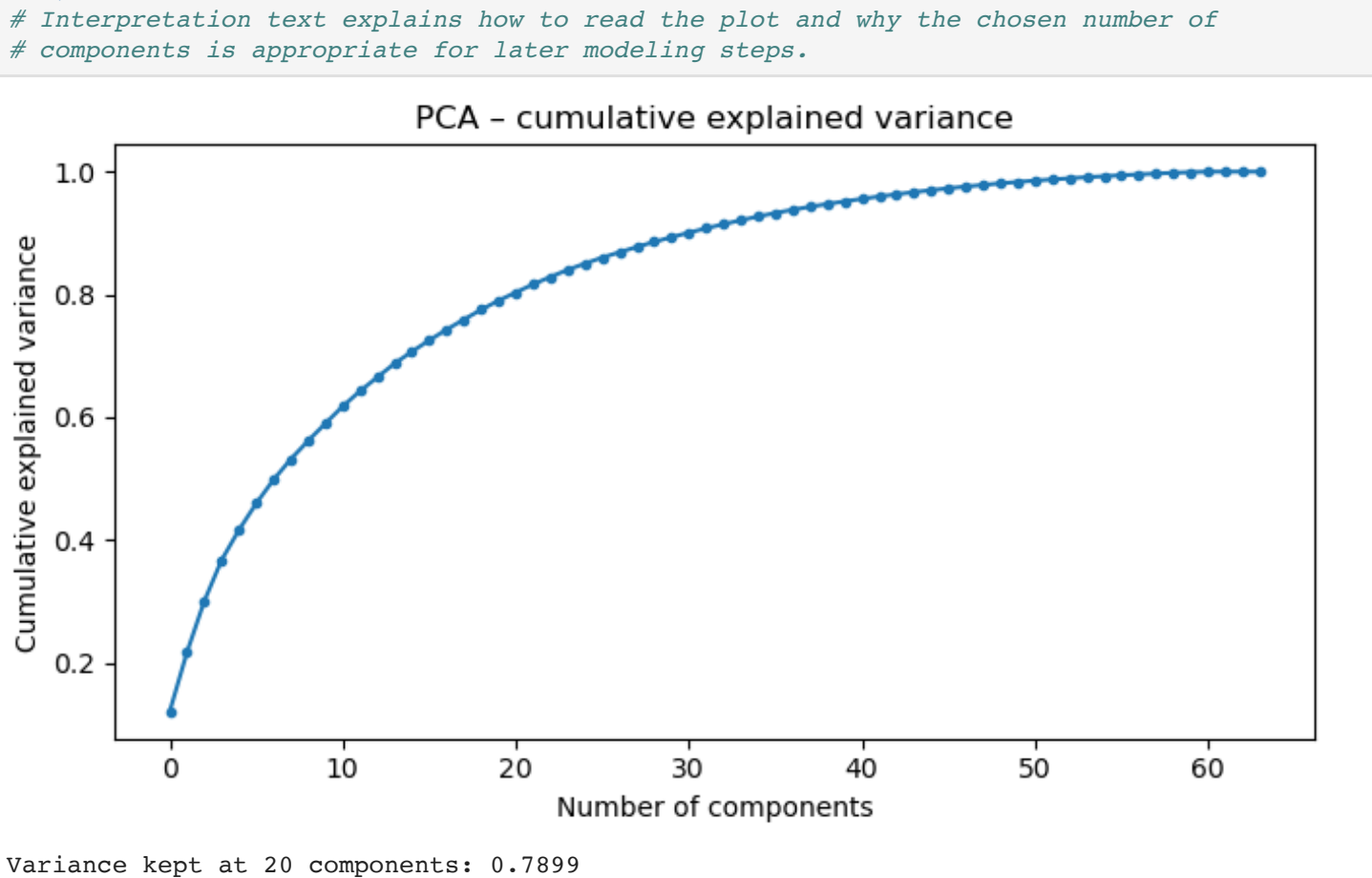

kNN + PCA(20) (Normalized by True / Recall view)

True label	0	1	2	3	4	5	6	7	8	9
0	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1	0.00	0.97	0.03	0.00	0.00	0.00	0.00	0.00	0.00	0.00
2	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
3	0.00	0.00	0.03	0.97	0.00	0.00	0.00	0.00	0.00	0.00
4	0.00	0.00	0.00	0.00	0.97	0.00	0.00	0.03	0.00	0.00
5	0.00	0.00	0.00	0.00	0.00	0.97	0.00	0.00	0.00	0.03
6	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00
7	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00
8	0.00	0.14	0.03	0.00	0.00	0.00	0.00	0.00	0.80	0.03
9	0.00	0.00	0.00	0.00	0.00	0.00	0.03	0.03	0.03	0.92

```
==========================================================================================
DETAILED CONFUSION MATRIX EVALUATION — kNN + PCA(20)
==========================================================================================
Interpretation (one-vs-rest for each class c):
  TP(c): predicted c AND true c (diagonal cell for c)
  FP(c): predicted c BUT true was not c (other rows in column c)
  FN(c): true c BUT predicted not c (other columns in row c)
  TN(c): not c AND predicted not c (everything not in row/col c)
------------------------------------------------------------------------------------------
Overall Accuracy = (sum diagonal) / total = 346/360 = 0.9611
Macro avg (P/R/F1): 0.9625 / 0.9607 / 0.9602
Weighted F1       : 0.9606
------------------------------------------------------------------------------------------
Class |   TP   FP   FN   TN | Precision  Recall    F1    | Support
------------------------------------------------------------------------------------------
    0 |   36    0    0  324 |     1.000   1.000   1.000 |      36
    1 |   35    5    1  319 |     0.875   0.972   0.921 |      36
    2 |   35    3    0  322 |     0.921   1.000   0.959 |      35
    3 |   36    0    1  323 |     1.000   0.973   0.986 |      37
    4 |   35    0    1  324 |     1.000   0.972   0.986 |      36
    5 |   36    0    1  323 |     1.000   0.973   0.986 |      37
    6 |   36    1    0  323 |     0.973   1.000   0.986 |      36
    7 |   36    2    0  322 |     0.947   1.000   0.973 |      36
    8 |   28    1    7  324 |     0.966   0.800   0.875 |      35
    9 |   33    2    3  322 |     0.943   0.917   0.930 |      36
------------------------------------------------------------------------------------------
Worst classes by Recall (high FN → true class often missed):
  Class 8: Recall=0.800 | FN=7 | Support=35
  Class 9: Recall=0.917 | FN=3 | Support=36
  Class 1: Recall=0.972 | FN=1 | Support=36
Worst classes by Precision (high FP → many false alarms for that predicted class):
  Class 1: Precision=0.875 | FP=5 | Predicted=40
  Class 2: Precision=0.921 | FP=3 | Predicted=38
  Class 9: Precision=0.943 | FP=2 | Predicted=35
------------------------------------------------------------------------------------------
Top 8 confusions (True → Predicted):
  8 → 1: 5 cases (14.29% of true class 8)
  1 → 2: 1 cases (2.78% of true class 1)
  3 → 2: 1 cases (2.70% of true class 3)
  4 → 7: 1 cases (2.78% of true class 4)
  5 → 9: 1 cases (2.70% of true class 5)
  8 → 2: 1 cases (2.86% of true class 8)
  8 → 9: 1 cases (2.86% of true class 8)
  9 → 6: 1 cases (2.78% of true class 9)
```

In [26]:

```python
# ---------------------------
# Cell purpose:
# Analyze how much variance is captured by PCA as the number of components increases.
# This helps justify the chosen number of PCA components used in earlier experiments.
# ---------------------------

# PCA cumulative explained variance plot
pca_full = PCA(random_state=RANDOM_STATE)
# Creates a PCA object without specifying n_components.
# This means PCA will compute all possible components (up to the original feature count).

pca_full.fit(X_train_scaled)
# Fits PCA on the scaled training data only.
# Using training data avoids information leakage from the test set.

cum_var = np.cumsum(pca_full.explained_variance_ratio_)
# Computes the cumulative sum of explained variance ratios.
# Each element cum_var[k] represents the total variance explained
# by the first (k + 1) principal components.

plt.figure(figsize=(7, 4))
# Creates a new figure for the cumulative variance plot.

plt.plot(cum_var, marker="o", markersize=3)
# Plots cumulative explained variance against number of components.
# Markers make individual component contributions easier to see.

plt.xlabel("Number of components")
# Labels the x-axis to indicate how many PCA components are included.

plt.ylabel("Cumulative explained variance")
# Labels the y-axis to indicate the fraction of total variance retained.

plt.title("PCA — cumulative explained variance")
# Adds a descriptive title explaining the plot.

plt.tight_layout()
# Adjusts spacing to prevent labels and title from overlapping.

plt.show()
# Displays the plot.

print("Variance kept at 20 components:", round(float(cum_var[19]), 4))
# Prints the cumulative variance retained when using the first 20 components.
# Index 19 corresponds to the 20th component (0-based indexing).

print("""
INTERPRETATION (PCA — Cumulative Explained Variance):

This plot shows how much of the total data variance is captured as the number of
principal components increases. The x-axis represents the number of PCA components,
and the y-axis shows the cumulative explained variance.

The curve rises steeply at the beginning, indicating that the first few components
capture a large portion of the variance. As more components are added, the curve
gradually flattens, showing diminishing returns from additional components.

At around 20 components, a high proportion of the total variance is retained, which
justifies using this reduced dimensionality for subsequent models. This choice
balances information preservation with computational efficiency.

Overall, the plot supports the selected number of PCA components as sufficient to
represent the data structure while reducing dimensionality.
""")
# Interpretation text explains how to read the plot and why the chosen number of
# components is appropriate for later modeling steps.
```



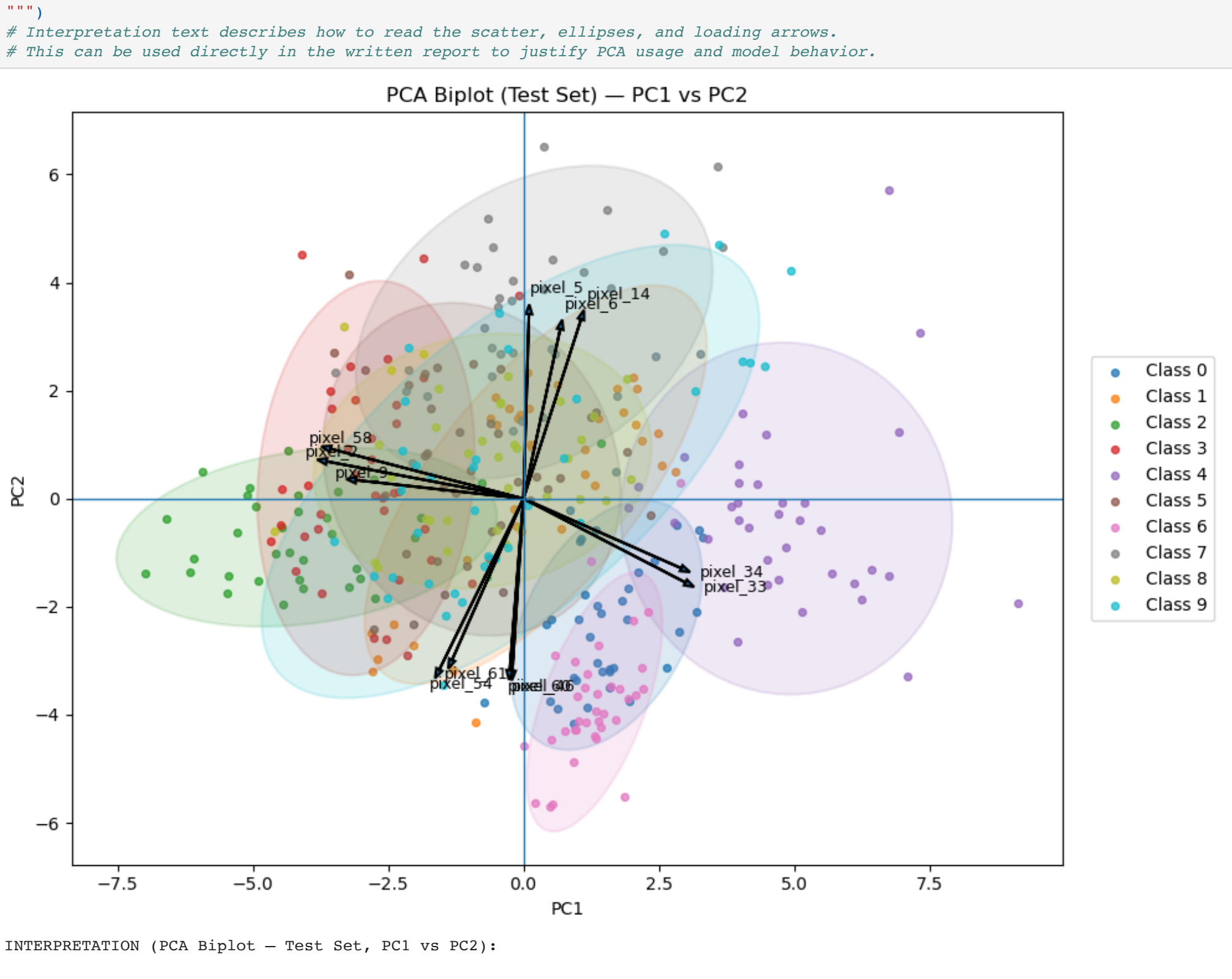
```
Variance kept at 20 components: 0.7899

INTERPRETATION (PCA — Cumulative Explained Variance):

This plot shows how much of the total data variance is captured as the number of
principal components increases. The x-axis represents the number of PCA components,
and the y-axis shows the cumulative explained variance.

The curve rises steeply at the beginning, indicating that the first few components
capture a large portion of the variance. As more components are added, the curve
gradually flattens, showing diminishing returns from additional components.

At around 20 components, a high proportion of the total variance is retained, which
justifies using this reduced dimensionality for subsequent models. This choice
balances information preservation with computational efficiency.

Overall, the plot supports the selected number of PCA components as sufficient to
represent the data structure while reducing dimensionality.
```

In [27]:

```python
# ---------------------------
# Cell purpose:
# Create a PCA visualization that is more informative than a plain scatter plot.
# The plot shows:
# - points in 2D PCA space (PC1 vs PC2) colored by class
# - a confidence ellipse per class (based on covariance of points)
# - optional loading arrows for the most influential original features (biplot style)
#
# This kind of figure is useful for explaining:
# - how well classes separate in PCA space
# - which classes overlap (likely confusions)
# - which original features contribute most to PC1/PC2 directions
# ---------------------------

def _cov_ellipse_params(x, y, n_std=2.0):
    """
    Return ellipse parameters (width, height, angle, center) for points (x,y).
    n_std=2 covers ~95% if data is roughly Gaussian.
    """
    x = np.asarray(x); y = np.asarray(y)
    # Converts x and y to NumPy arrays so covariance math is consistent.

    mask = np.isfinite(x) & np.isfinite(y)
    # Builds a boolean mask to keep only valid numeric points.
    # This removes NaN and Inf values if they exist.

    x = x[mask]
    y = y[mask]
    # Filters x and y so only finite points remain.

    if len(x) < 3:
        # An ellipse requires enough points to estimate covariance reliably.
        # Fewer than 3 points makes covariance estimation unstable.
        return None

    cov = np.cov(np.vstack([x, y]))
    # Stacks x and y into a 2xN matrix then computes the 2x2 covariance matrix.
    # Covariance captures spread and correlation between x and y.

    vals, vecs = np.linalg.eigh(cov)
    # Eigen-decomposition of the covariance matrix.
    # - eigenvalues (vals) represent variance along principal axes of the ellipse
    # - eigenvectors (vecs) give the directions of those axes

    order = vals.argsort()[::-1]
    # Sorts eigenvalues from largest to smallest.

    vals = vals[order]
    vecs = vecs[:, order]
    # Reorders eigenvalues and eigenvectors so the first corresponds to the major axis.

    # width/height are 2*n_std*sqrt(eigenvalues)
    width, height = 2 * n_std * np.sqrt(vals)
    # sqrt(eigenvalue) gives standard deviation along that axis.
    # Multiplying by n_std expands radius to cover roughly that many standard deviations.
    # Multiplying by 2 converts radius to full diameter, which Ellipse expects.

    angle = np.degrees(np.arctan2(vecs[1, 0], vecs[0, 0]))
    # Computes ellipse rotation angle from the first eigenvector (major axis direction).
    # arctan2 converts direction vector into angle in radians.
    # degrees() converts radians to degrees for Matplotlib Ellipse.

    center = (np.mean(x), np.mean(y))
    # Ellipse is centered at the mean location of the class points.

    return width, height, angle, center
    # Returns the parameters required to draw the ellipse.


def pca_biplot_with_ellipses(
    X_2d, y,
    loadings=None, feature_names=None,
    title="PCA Biplot (PC1 vs PC2)",
    n_std=2.0,
    max_arrows=12,
    arrow_scale=3.0,
):
    """
    X_2d: PCA scores (N x 2) for PC1/PC2
    y: labels (N,)
    loadings: (n_features x 2) PCA component loadings for PC1/PC2 (optional)
    feature_names: list of feature names (optional)

    Produces:
      - scatter per class
      - confidence ellipse per class
      - loading arrows (biplot) for top-contributing features
    """
    X_2d = np.asarray(X_2d)
    # Ensures the PCA score matrix is a NumPy array.
    # Expected shape: (N samples, 2 components)

    y = np.asarray(y).astype(int).ravel()
    # Converts labels into a clean 1D integer array.

    classes = np.unique(y)
    # Finds all unique class labels to plot each class separately.

    cmap = plt.get_cmap("tab10")
    # Selects a colormap with 10 distinct colors, matching digit classes 0–9.
    plt.figure(figsize=(10, 7))
    # Creates a larger figure to fit the scatter, ellipses, and legend.

    # ---- Scatter + Ellipses per class ----
    for c in classes:
        # Loops over each digit class and plots its points separately.

        mask = (y == c)
        # Selects samples belonging to class c.

        xs = X_2d[mask, 0]
        ys = X_2d[mask, 1]
        # Extracts PC1 and PC2 coordinates for class c.

        plt.scatter(xs, ys, s=18, alpha=0.75, label=f"Class {c}", color=cmap(c % 10))
        # Draws scatter points for class c.
        # s controls point size, alpha controls transparency.
        # color is chosen from tab10 so each digit gets a consistent color.
        # label is used for the legend.

        params = _cov_ellipse_params(xs, ys, n_std=n_std)
        # Computes the confidence ellipse parameters for the class cloud.

        if params is not None:
            # Skips ellipse drawing if there are too few points.

            width, height, angle, center = params
            # Unpacks ellipse parameters.

            from matplotlib.patches import Ellipse
            # Imports Ellipse shape used for plotting confidence regions.

            ell = Ellipse(
                xy=center, width=width, height=height, angle=angle,
                facecolor=cmap(c % 10), edgecolor=cmap(c % 10),
                alpha=0.15, linewidth=1.5
            )
            # Builds an ellipse patch:
            # xy=center defines ellipse center,
            # width/height define ellipse diameters,
            # angle defines rotation in degrees.
            # facecolor is filled with light transparency,
            # edgecolor matches class color for clarity.

            plt.gca().add_patch(ell)
            # Adds the ellipse patch to the current axes.

    # ---- Axes lines like the sample plot ----
    plt.axhline(0, linewidth=1)
    # Draws a horizontal axis line through y=0 to show the origin.

    plt.axvline(0, linewidth=1)
    # Draws a vertical axis line through x=0 to show the origin.

    # ---- Loading arrows (biplot) ----
    if loadings is not None:
        # Loadings show how original features contribute to PC1 and PC2 directions.

        loadings = np.asarray(loadings)
        # Ensures loadings are a NumPy array.

        # pick features with largest combined contribution to PC1+PC2
        contrib = np.sqrt(loadings[:, 0]**2 + loadings[:, 1]**2)
        # Computes a combined contribution magnitude for each feature.
        # Large magnitude means the feature strongly influences PC1 and/or PC2.

        top_idx = np.argsort(contrib)[::-1][:max_arrows]
        # Selects the indices of the most influential features.
        # max_arrows limits clutter by drawing only top features.

        # scale arrows to match scatter scale
        # (simple heuristic: use spread of scores)
        spread_x = np.percentile(np.abs(X_2d[:, 0]), 95)
        spread_y = np.percentile(np.abs(X_2d[:, 1]), 95)
        # Estimates typical spread of the PCA scatter by using the 95th percentile.
        # This avoids scaling based on extreme outliers.

        scale = arrow_scale * max(spread_x, spread_y)
        # Chooses a scaling factor so arrows are visible on the same plot scale.
        # arrow_scale allows manual adjustment of arrow length.

        for i in top_idx:
            # Draws an arrow for each selected feature.

            lx, ly = loadings[i, 0], loadings[i, 1]
            # Loading coordinates for feature i in PC1 and PC2 directions.

            plt.arrow(
                0, 0, lx * scale, ly * scale,
                head_width=0.12, head_length=0.18,
                length_includes_head=True,
                linewidth=1.5
            )
            # Draws an arrow from the origin (0,0) in the direction of the loading vector.
            # head_width/head_length control arrowhead size.
            # length_includes_head ensures the arrow length includes the head portion.

            name = feature_names[i] if feature_names is not None else f"f{i}"
            # Uses real feature names if provided; otherwise uses a generic name f0, f1, ...

            plt.text(lx * scale * 1.06, ly * scale * 1.06, name, fontsize=9)
            # Places a label slightly beyond the arrow tip for readability.

    plt.title(title)
    # Adds plot title.

    plt.xlabel("PC1")
    # Labels x-axis as principal component 1.

    plt.ylabel("PC2")
    # Labels y-axis as principal component 2.

    plt.legend(loc="center left", bbox_to_anchor=(1.02, 0.5))
    # Places the legend outside the plot to avoid covering points.
    # bbox_to_anchor shifts the legend to the right side.

    plt.tight_layout()
    # Adjusts layout so labels and legend fit properly.

    plt.show()
    # Displays the final plot.
```

In [28]:

```
# ---------------------------
# Cell purpose:
# Create a PCA biplot for the test set using the first two principal components.
# The plot includes:
# - a 2D scatter of test samples in PCA space (PC1 vs PC2), colored by true class
# - confidence ellipses per class to show spread and overlap
# - arrows representing influential original pixel features (loadings)
# ---------------------------

# Use the first two PCs for plotting
X_plot = X_test_pca[:, :2]
# Selects only the first two PCA components for visualization.
# X_test_pca may contain 20 components, but only PC1 and PC2 are used for 2D plotting.

# If PCA object used earlier is available, PCA loadings can be plotted:
# loadings shape should be (n_features, 2) for PC1/PC2
loadings_2d = pca.components_[:2].T  # (n_features x 2)
# pca.components_ has shape (n_components, original_features).
# Taking [:2] selects the first two principal component directions.
# Transposing produces shape (original_features, 2), which matches expected loading format.
# Each row corresponds to one original feature (pixel), and columns correspond to PC1 and PC2.

# For digits, features are pixels: 64 features if using 8x8 digits
feature_names = [f"pixel_{i}" for i in range(loadings_2d.shape[0])]
# Creates readable names for each original pixel feature.
# These names are used to label the arrows in the biplot.

pca_biplot_with_ellipses(
    X_plot, y_test,
    loadings=loadings_2d,
    feature_names=feature_names,
    title=f"PCA Biplot (Test Set) — PC1 vs PC2",
    n_std=2.0,
    max_arrows=12,
    arrow_scale=2.5
)
# Generates the biplot:
# - X_plot provides the 2D PCA coordinates for test samples
# - y_test provides the class labels for coloring and ellipse grouping
# - loadings_2d and feature_names enable arrow plotting for influential pixels
# - n_std controls ellipse size (2.0 approximates a ~95% coverage region for Gaussian-like clusters)
# - max_arrows limits clutter by drawing arrows for only the strongest contributors
# - arrow_scale adjusts arrow length so arrows match the scatter scale visually

print("""
INTERPRETATION (PCA Biplot — Test Set, PC1 vs PC2):

This biplot visualizes the test data projected onto the first two principal
components (PC1 and PC2), which capture the largest variance in the dataset.
Each point represents a test sample, colored by its true class label.

The ellipses illustrate the spread and overlap of each class in the reduced
2D PCA space. Partial overlap between classes indicates that some digits share
similar feature structures, while better-separated clusters correspond to
classes that are more easily distinguishable.

The arrows represent the original pixel features contributing most strongly
to PC1 and PC2. The direction and length of each arrow indicate how strongly
a pixel influences the variance along these components.

Overall, this plot provides an interpretable view of how the high-dimensional
image data is structured in lower dimensions and explains why PCA-based models
such as kNN perform effectively in this feature space.
""")
# Interpretation text describes how to read the scatter, ellipses, and loading arrows.
# This can be used directly in the written report to justify PCA usage and model behavior.
```



```
INTERPRETATION (PCA Biplot — Test Set, PC1 vs PC2):

This biplot visualizes the test data projected onto the first two principal
components (PC1 and PC2), which capture the largest variance in the dataset.
Each point represents a test sample, colored by its true class label.

The ellipses illustrate the spread and overlap of each class in the reduced
2D PCA space. Partial overlap between classes indicates that some digits share
similar feature structures, while better-separated clusters correspond to
classes that are more easily distinguishable.

The arrows represent the original pixel features contributing most strongly
to PC1 and PC2. The direction and length of each arrow indicate how strongly
a pixel influences the variance along these components.

Overall, this plot provides an interpretable view of how the high-dimensional
image data is structured in lower dimensions and explains why PCA-based models
such as kNN perform effectively in this feature space.
```

PCA + kNN evaluation:

With PCA, kNN keeps similar accuracy while using fewer features. This usually makes kNN faster because distance calculations happen in a smaller space.Applying PCA significantly reduced the dimensionality of the dataset while maintaining performance close to that of the full feature space. The similarity in accuracy and F1-score suggests that much of the discriminative information is preserved within the principal components.

This result highlights the usefulness of dimensionality reduction for improving computational efficiency without substantially degrading classification performance.

Decision Tree (library)

Trees don't need scaling, so we use the raw 64-feature inputs.

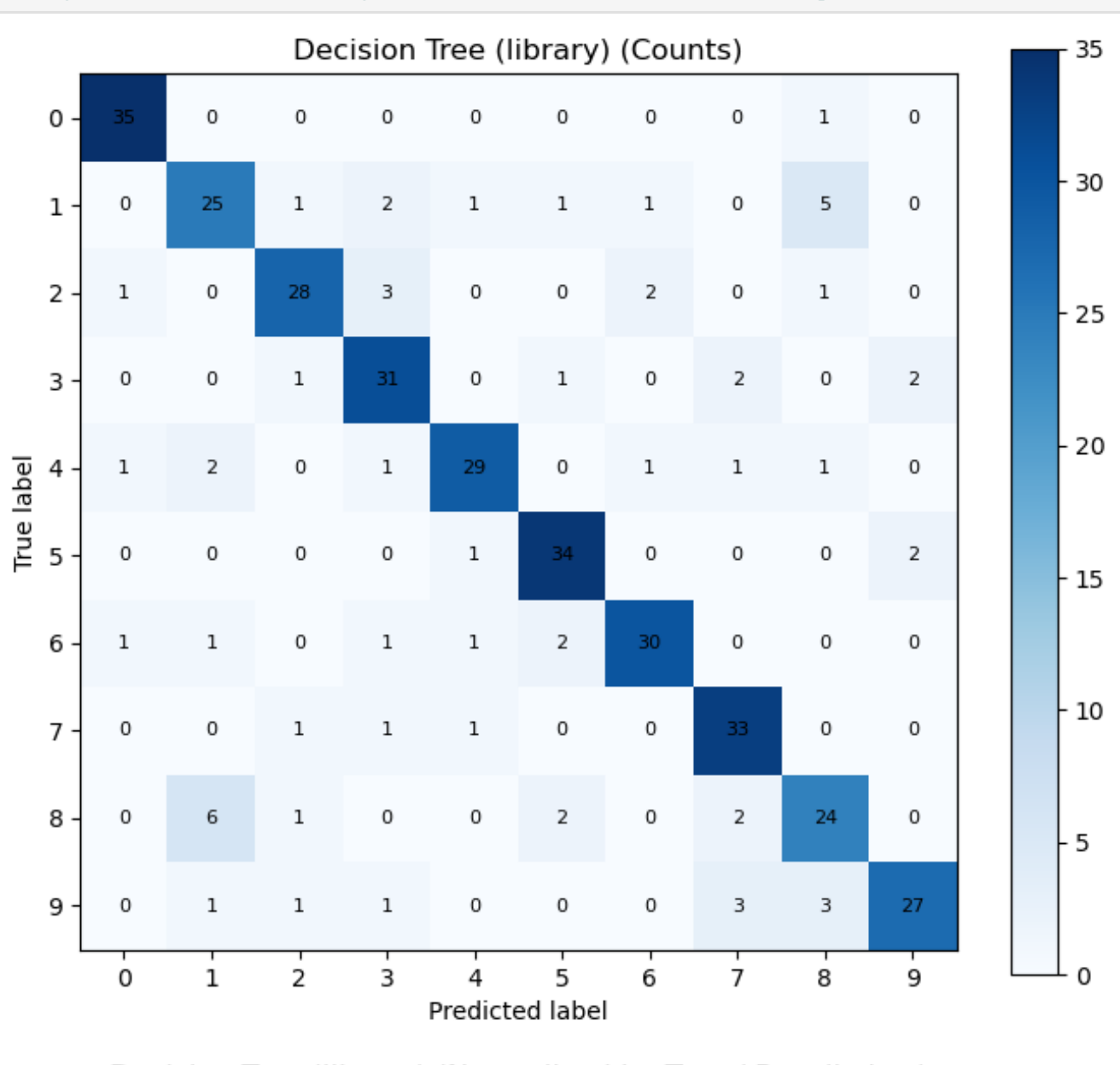
In [29]:

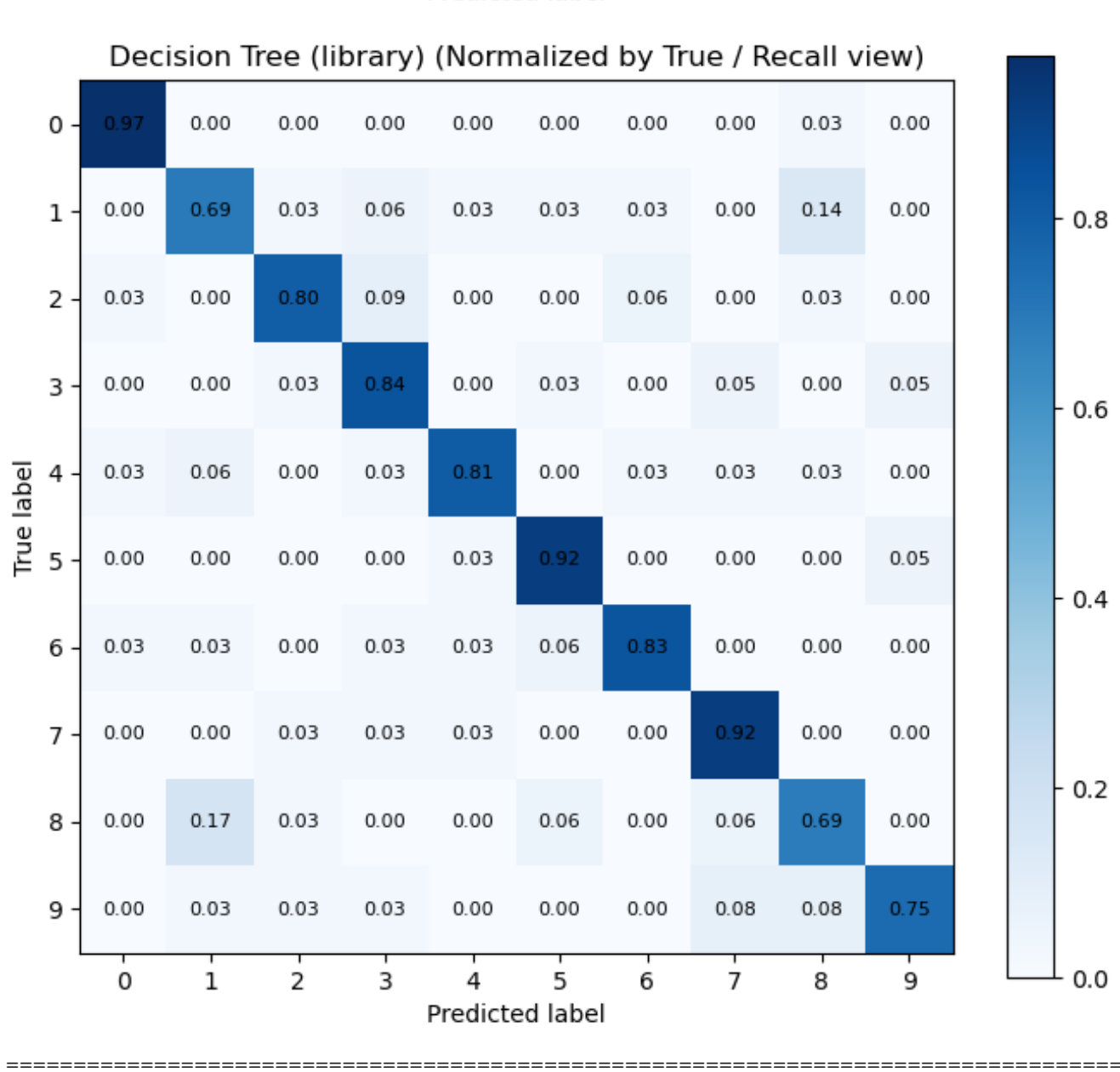
```
# ---------------------------
# Cell purpose:
# Train and evaluate a Decision Tree classifier using a library implementation.
# This model serves as a baseline comparison against from-scratch models
# such as kNN and linear/polynomial regression.
# ---------------------------

dt = DecisionTreeClassifier(random_state=RANDOM_STATE)
# Creates a Decision Tree classifier object.
# random_state ensures that the tree structure is reproducible,
# since decision trees can involve randomness when choosing splits.

t0 = time.time()
# Records the start time to measure total runtime
# (training + prediction combined).

dt.fit(X_train, y_train)
# Trains the decision tree using the original training features.
# The tree learns a sequence of if–else rules that split the feature space
# to separate digit classes based on pixel values.

y_pred_dt = dt.predict(X_test)
# Uses the trained decision tree to predict labels for the test set.
# Each test sample is passed down the learned tree until it reaches a leaf node,
# and the class assigned to that leaf becomes the prediction.

time_dt = time.time() - t0
# Computes the total elapsed time for training and prediction.

acc_dt, p_dt, r_dt, f1_dt = print_metrics_block(
    "Decision Tree (library)",
    y_test, y_pred_dt, n_classes=10
)
# Evaluates the decision tree predictions using NumPy-based metrics.
# Prints accuracy and macro-averaged precision, recall, and F1-score.
# Returned values are stored for later comparison with other models.

cm_dt = confusion_matrix_numpy(y_test, y_pred_dt, n_classes=10)
# Builds the confusion matrix for decision tree predictions.
# Rows correspond to true digit labels.
# Columns correspond to predicted digit labels.

print(f"Runtime (fit + predict): {time_dt:.4f} seconds")
# Prints total runtime.
# Decision trees are usually fast to train and predict,
# making them efficient but potentially prone to overfitting without tuning.
```

```
============================================================
Decision Tree (library)
============================================================
Accuracy : 0.8222
Precision: 0.8232  (macro)
Recall   : 0.8215  (macro)
F1-score : 0.8209  (macro)
============================================================
Runtime (fit + predict): 0.0142 seconds
```

In [30]:

```
# ---------------------------
# Cell purpose:
# Visualize and deeply analyze the confusion matrix for the
# library-based Decision Tree classifier.
# This provides a class-by-class view of performance and errors.
# ---------------------------

cm_dt = confusion_matrix_numpy(y_test, y_pred_dt, n_classes=10)
# Builds the confusion matrix for the decision tree predictions.
# Rows represent the true digit labels.
# Columns represent the predicted digit labels.

plot_confusion_matrix_individual(cm_dt, "Decision Tree (library)", normalize=None)
# Displays the confusion matrix using raw counts.
# The diagonal shows correct classifications.
# Off-diagonal cells show specific misclassification patterns.

plot_confusion_matrix_individual(cm_dt, "Decision Tree (library)", normalize="true")
# Displays the confusion matrix normalized by true class.
# Each row sums to 1, so values represent recall proportions.
# This highlights which digits are frequently confused by the tree.

deep_eval_from_confusion_matrix(cm_dt, "Decision Tree (library)")
# Prints a detailed confusion-matrix-based evaluation.
# Includes:
# - overall accuracy
# - macro and weighted performance metrics
# - TP/FP/FN/TN counts per class
# - weakest classes by recall and precision
# - most frequent true→predicted confusion pairs
# - guidance for describing decision tree behavior in the report
```

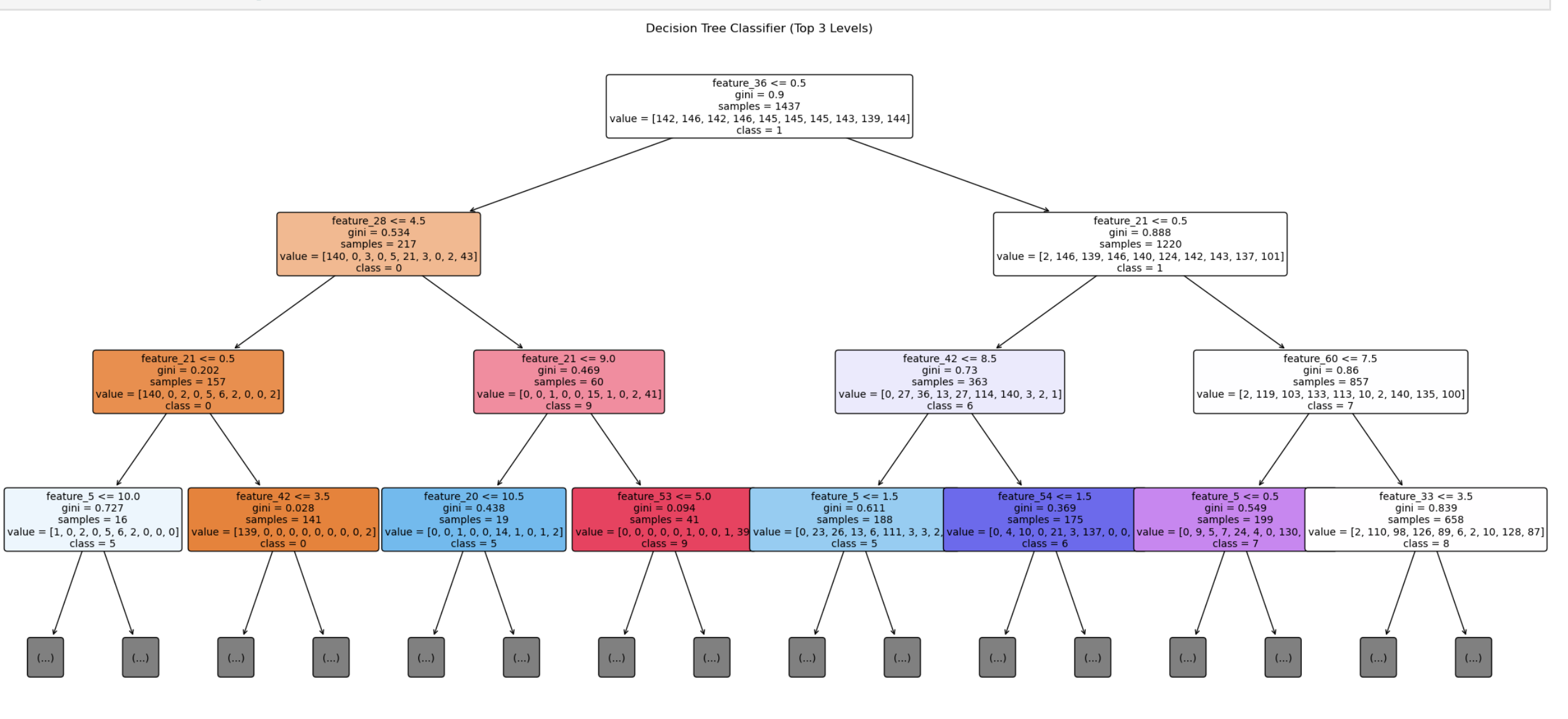




```
==========================================================================================
DETAILED CONFUSION MATRIX EVALUATION — Decision Tree (library)
==========================================================================================
Interpretation (one-vs-rest for each class c):
  TP(c): predicted c AND true c (diagonal cell for c)
  FP(c): predicted c BUT true was not c (other rows in column c)
  FN(c): true c BUT predicted not c (other columns in row c)
  TN(c): not c AND predicted not c (everything not in row/col c)
------------------------------------------------------------------------------------------
Overall Accuracy = (sum diagonal) / total = 296/360 = 0.8222
Macro avg (P/R/F1): 0.8232 / 0.8215 / 0.8209
Weighted F1       : 0.8214
------------------------------------------------------------------------------------------
Class |   TP   FP   FN   TN | Precision  Recall    F1    | Support
------------------------------------------------------------------------------------------
    0 |   35    3    1  321 |     0.921   0.972   0.946 |      36
    1 |   25   10   11  314 |     0.714   0.694   0.704 |      36
    2 |   28    5    7  320 |     0.848   0.800   0.824 |      35
    3 |   31    9    6  314 |     0.775   0.838   0.805 |      37
    4 |   29    4    7  320 |     0.879   0.806   0.841 |      36
    5 |   34    6    3  317 |     0.850   0.919   0.883 |      37
    6 |   30    4    6  320 |     0.882   0.833   0.857 |      36
    7 |   33    8    3  316 |     0.805   0.917   0.857 |      36
    8 |   24   11   11  314 |     0.686   0.686   0.686 |      35
    9 |   27    4    9  320 |     0.871   0.750   0.806 |      36
------------------------------------------------------------------------------------------
Worst classes by Recall (high FN → true class often missed):
  Class 8: Recall=0.686 | FN=11 | Support=35
  Class 1: Recall=0.694 | FN=11 | Support=36
  Class 9: Recall=0.750 | FN=9 | Support=36
Worst classes by Precision (high FP → many false alarms for that predicted class):
  Class 8: Precision=0.686 | FP=11 | Predicted=35
  Class 1: Precision=0.714 | FP=10 | Predicted=35
  Class 3: Precision=0.775 | FP=9 | Predicted=40
------------------------------------------------------------------------------------------
Top 8 confusions (True → Predicted):
  8 → 1: 6 cases (17.14% of true class 8)
  1 → 8: 5 cases (13.89% of true class 1)
  2 → 3: 3 cases (8.57% of true class 2)
  9 → 7: 3 cases (8.33% of true class 9)
  9 → 8: 3 cases (8.33% of true class 9)
  1 → 3: 2 cases (5.56% of true class 1)
  2 → 6: 2 cases (5.71% of true class 2)
  3 → 7: 2 cases (5.41% of true class 3)
```

In [31]:

```
# ---------------------------
# Cell purpose:
# Visualize the structure of the trained Decision Tree classifier.
# Only the top few levels are shown to keep the diagram readable
# and suitable for inclusion in a report.
# ---------------------------

from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Create feature and class names
feature_names = [f"feature_{i}" for i in range(X_train.shape[1])]
# Generates human-readable names for each input feature.
# Here, each feature corresponds to a pixel position in the 8×8 digit image.

class_names = [str(i) for i in range(10)]
# Creates string labels for the digit classes (0 through 9).
# These labels are used in the tree visualization.

plt.figure(figsize=(22, 10))
# Creates a large figure so text and node details remain readable.

plot_tree(
    dt,
    max_depth=3,          # show top 3 levels only (best for reports)
    filled=True,
    rounded=True,
    fontsize=10,
    feature_names=feature_names,
    class_names=class_names
)
# Draws the decision tree structure.
# Key parameters:
# - max_depth=3 limits the display to the most important top splits.
# - filled=True colors nodes based on the dominant class.
# - rounded=True uses rounded boxes for cleaner visuals.
# - fontsize controls readability of text inside nodes.
# - feature_names labels each split with a feature name.
# - class_names labels predicted classes at leaf nodes.

plt.title("Decision Tree Classifier (Top 3 Levels)")
# Adds a descriptive title above the visualization.

plt.tight_layout()
# Adjusts layout so labels and boxes do not overlap.

plt.show()
# Displays the decision tree plot.

print("""
INTERPRETATION (Decision Tree Classifier — Top 3 Levels):

This visualization shows the top three levels of the trained decision tree
classifier. Each node represents a decision based on a single feature threshold.

At each node:
- The condition (e.g., feature_36 ≤ 0.5) defines how samples are split.
- The Gini value measures class impurity; lower values indicate purer nodes.
- 'Samples' shows how many training points reach the node.
- 'Value' shows the class distribution at that node.
- 'Class' is the predicted class for samples at that node.

The root node performs the most informative split, reducing impurity the most.
As the tree moves downward, decisions become more specific, focusing on smaller
subsets of the data.

Different branches emphasize different features, indicating that the classifier
relies on multiple pixel locations to distinguish between digit classes.
Colored nodes represent the dominant class at that node, while mixed colors
indicate uncertainty.

Only the top three levels are shown to maintain interpretability. The full tree
extends deeper, which increases classification accuracy but also increases model
complexity and the risk of overfitting.

Overall, this visualization demonstrates how the decision tree learns hierarchical,
rule-based decision boundaries directly from the input features.
""")
# The interpretation text explains how to read the tree and what insights
# can be drawn from the top-level structure.
```



```
INTERPRETATION (Decision Tree Classifier — Top 3 Levels):

This visualization shows the top three levels of the trained decision tree
classifier. Each node represents a decision based on a single feature threshold.

At each node:
- The condition (e.g., feature_36 ≤ 0.5) defines how samples are split.
- The Gini value measures class impurity; lower values indicate purer nodes.
- 'Samples' shows how many training points reach the node.
- 'Value' shows the class distribution at that node.
- 'Class' is the predicted class for samples at that node.

The root node performs the most informative split, reducing impurity the most.
As the tree moves downward, decisions become more specific, focusing on smaller
subsets of the data.

Different branches emphasize different features, indicating that the classifier
relies on multiple pixel locations to distinguish between digit classes.
Colored nodes represent the dominant class at that node, while mixed colors
indicate uncertainty.

Only the top three levels are shown to maintain interpretability. The full tree
extends deeper, which increases classification accuracy but also increases model
complexity and the risk of overfitting.

Overall, this visualization demonstrates how the decision tree learns hierarchical,
rule-based decision boundaries directly from the input features.
```

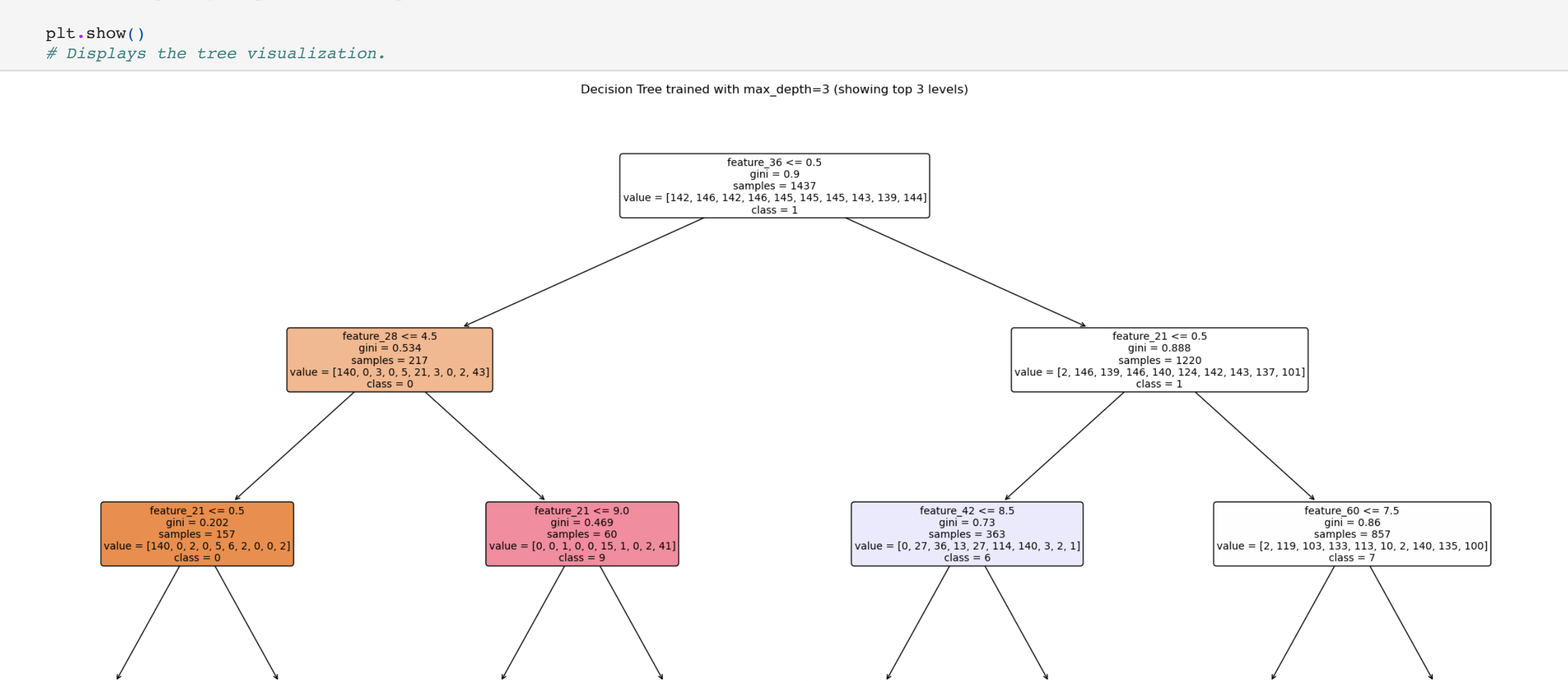
In [32]:

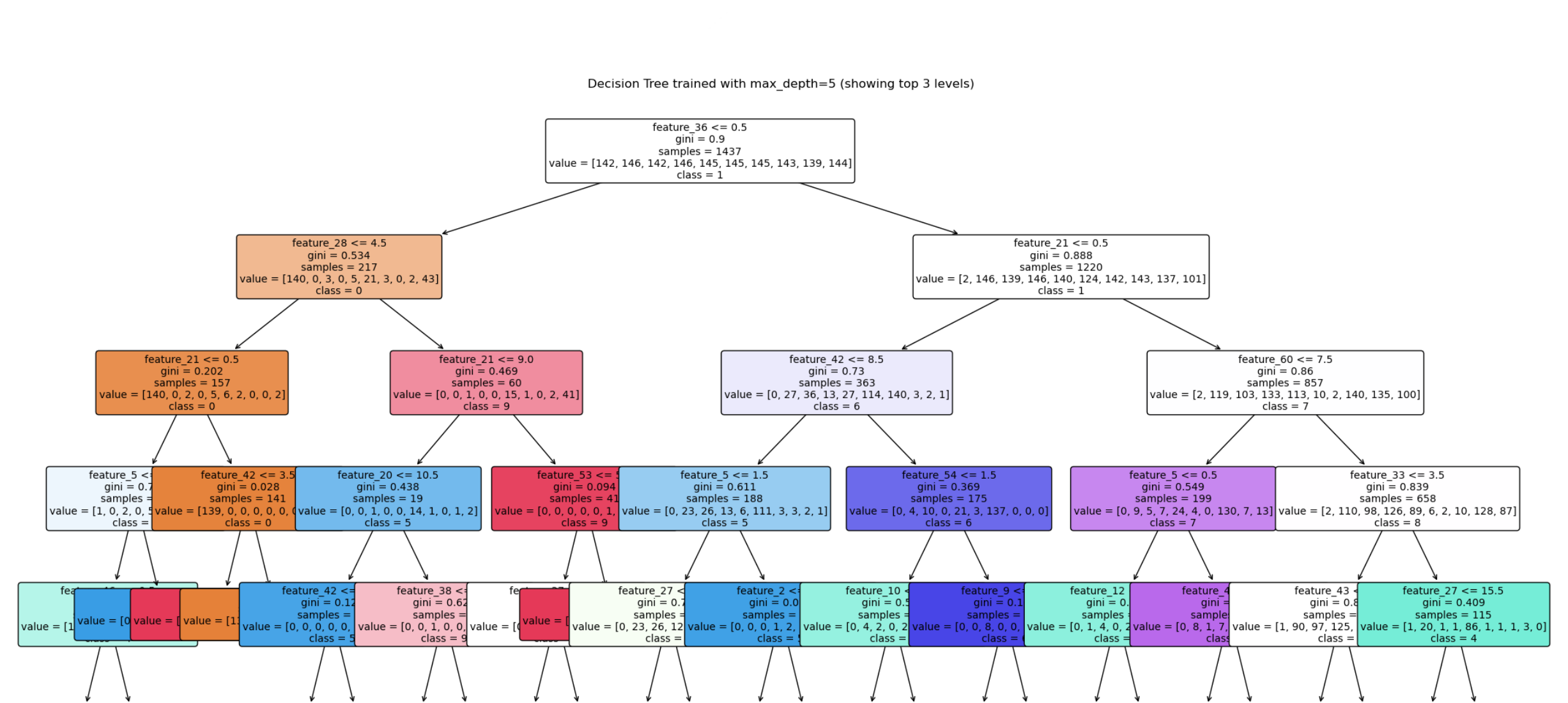
```
# ---------------------------
# Cell purpose:
# Study how the maximum depth of a Decision Tree affects test accuracy.
# This experiment illustrates the bias–variance trade-off in decision trees.
# ---------------------------

depths = [3, 5, 10, None]
# Defines a set of tree depths to evaluate.
# - Small depths (e.g., 3) restrict model complexity.
# - Larger depths allow more detailed decision rules.
# - None means no depth limit, allowing the tree to grow fully.

tree_depth_results = []
# Initializes a list to store accuracy results for each depth setting.

for d in depths:
    # Iterates over each chosen maximum depth.

    model = DecisionTreeClassifier(
        random_state=RANDOM_STATE,
        max_depth=d
    )
    # Creates a Decision Tree classifier with the specified depth.
    # random_state ensures reproducible splits.
    # max_depth controls how many levels the tree can grow.

    model.fit(X_train, y_train)
    # Trains the decision tree using the training data.
    # The tree learns a hierarchy of feature-based split rules.

    pred = model.predict(X_test)
    # Generates predictions for the test set using the trained tree.

    # NumPy-only accuracy (no sklearn.metrics)
    acc = accuracy_numpy(y_test, pred)
    # Computes test accuracy using a custom NumPy-based function.
    # Accuracy measures the fraction of correctly classified test samples.

    tree_depth_results.append(
        {
            "max_depth": str(d),
            "test_accuracy": round(acc, 4)
        }
    )
    # Stores the depth value and corresponding test accuracy.
    # Depth is converted to string so None can be displayed cleanly.

print("Decision Tree depth check")
# Prints a header describing the experiment.

print("-" * 35)
# Prints a separator line for readability.

for row in tree_depth_results:
    # Iterates through stored results to display them clearly.

    print(
        f"max_depth={row['max_depth']:<4} "
        f"test_accuracy={row['test_accuracy']}"
    )
    # Prints depth and test accuracy in aligned format.
```

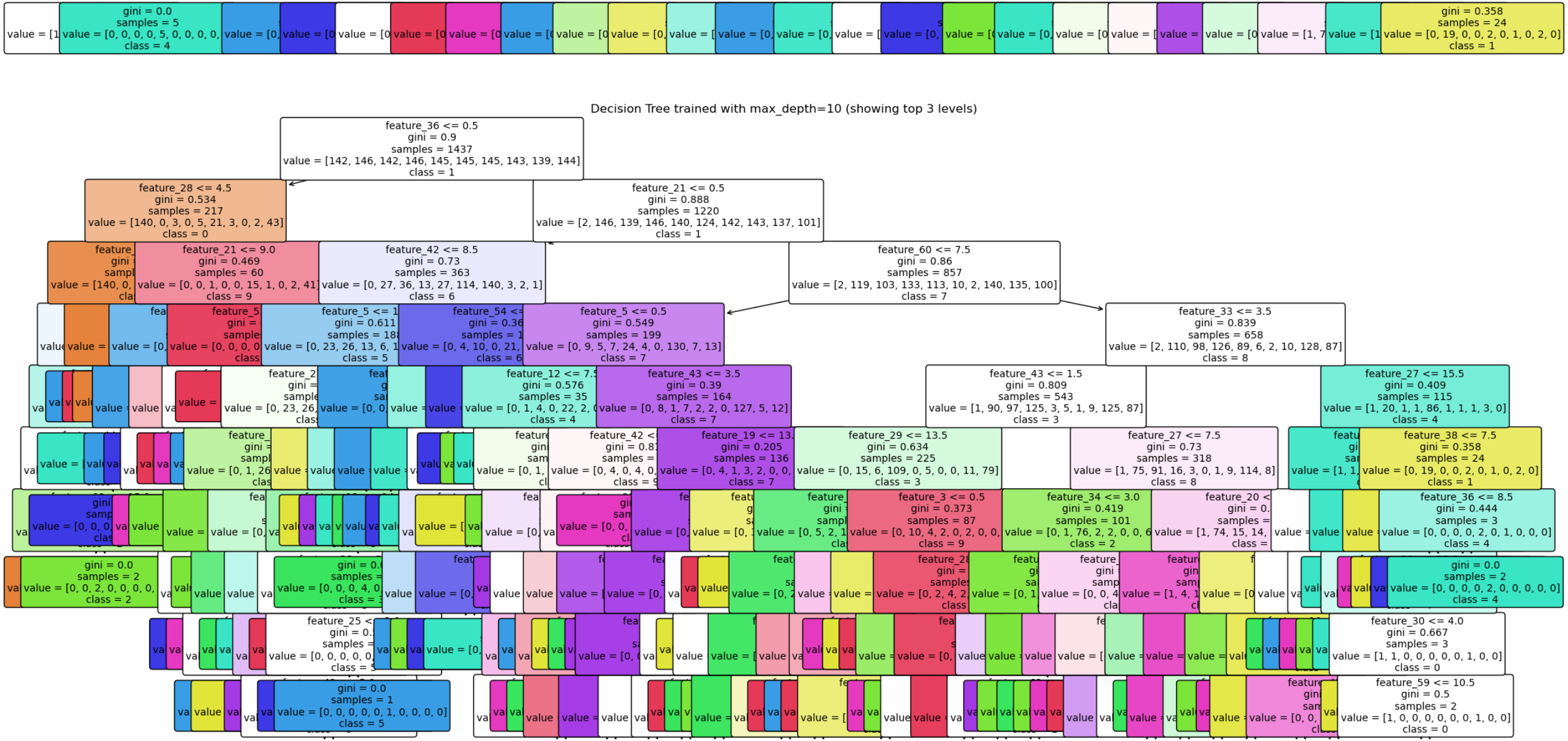
```
Decision Tree depth check
-----------------------------------
max_depth=3    test_accuracy=0.4833
max_depth=5    test_accuracy=0.6694
max_depth=10   test_accuracy=0.8139
max_depth=None test_accuracy=0.8222
```

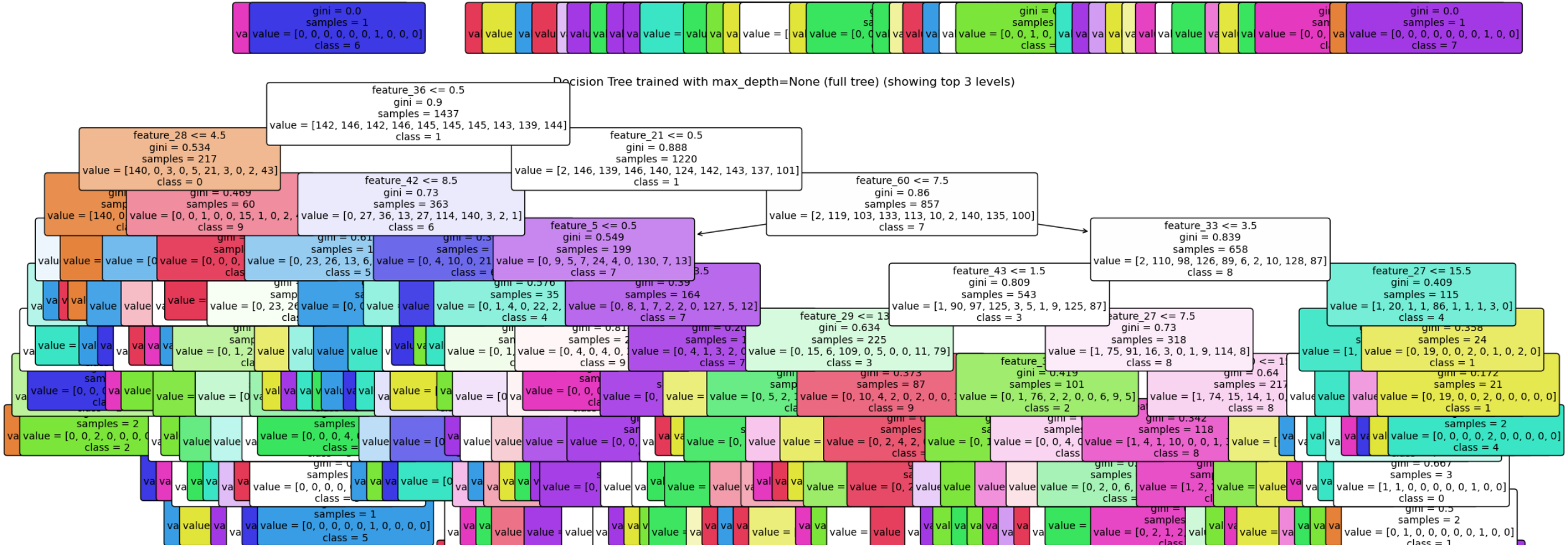
In [33]:

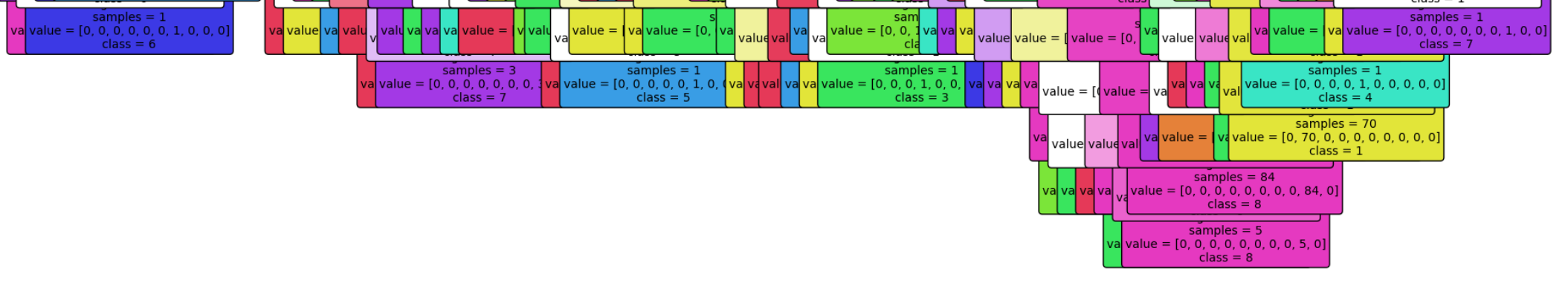
```
# ==========================================================
# Visualize Decision Trees trained at different max_depth values
# Assumes: X_train, y_train, RANDOM_STATE already exist
# ==========================================================
# Cell purpose:
# Train multiple Decision Tree models with different maximum depths
# and visualize their top-level structure.
#
# This helps compare:
# - how tree depth affects decision complexity
# - how early splits change as depth increases
# - how interpretability decreases as trees grow deeper
#
# Only the top levels are visualized to keep the plots readable.
# ==========================================================

import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

depths = [3, 5, 10, None]
# Defines the maximum depth values to test.
# Smaller values restrict tree growth.
# None allows the tree to grow fully without depth limits.

feature_names = [f"feature_{i}" for i in range(X_train.shape[1])]
# Generates readable names for each input feature.
# Each feature corresponds to a pixel value in the digit image.

class_names = [str(i) for i in range(10)]
# Defines class labels for digits 0 through 9.
# These labels appear in the tree visualization.

for d in depths:
    # Iterates over each depth setting.

    model = DecisionTreeClassifier(random_state=RANDOM_STATE, max_depth=d)
    # Creates a decision tree with the specified maximum depth.
    # random_state ensures the same split behavior across runs.
    # max_depth controls how complex the tree is allowed to become.

    model.fit(X_train, y_train)
    # Trains the decision tree on the training data.
    # The tree learns a hierarchy of feature-based decision rules.

    label = "None (full tree)" if d is None else str(d)
    # Creates a readable label for the plot title.
    # None is replaced with a descriptive phrase for clarity.

    plt.figure(figsize=(22, 10))
    # Creates a large figure so that node text remains readable.

    plot_tree(
        model,
        # Even if the tree is deeper, only the top part is displayed automatically.
        # This keeps the visualization interpretable.

        filled=True,
        # Colors nodes based on the dominant class at that node.

        rounded=True,
        # Uses rounded corners for cleaner-looking nodes.

        fontsize=10,
        # Sets text size inside nodes so details remain legible.

        feature_names=feature_names,
        # Displays feature names in split conditions.

        class_names=class_names
        # Displays class labels at leaf nodes.
    )

    plt.title(f"Decision Tree trained with max_depth={label} (showing top 3 levels)")
    # Adds a descriptive title explaining the depth used during training.

    plt.tight_layout()
    # Adjusts spacing to prevent overlaps between nodes and text.

    plt.show()
    # Displays the tree visualization.
```


Decision Tree trained with max_depth=3 (showing top 3 levels)
feature_36 <= 0.5
gini = 0.9
samples = 1437
value = [142, 146, 142, 146, 145, 145, 145, 143, 139, 144]
class = 1
feature_28 <= 4.5
gini = 0.534
samples = 217
value = [140, 0, 3, 0, 5, 21, 3, 0, 2, 43]
class = 0
feature_21 <= 0.5
gini = 0.888
samples = 1220
value = [2, 146, 139, 146, 140, 124, 142, 143, 137, 101]
class = 1
feature_21 <= 0.5
gini = 0.202
samples = 157
value = [140, 0, 2, 0, 5, 6, 2, 0, 0, 2]
class = 0
feature_21 <= 9.0
gini = 0.469
samples = 60
value = [0, 0, 1, 0, 0, 15, 1, 0, 2, 41]
class = 9
feature_42 <= 8.5
gini = 0.73
samples = 363
value = [0, 27, 36, 13, 27, 114, 140, 3, 2, 1]
class = 6
feature_60 <= 7.5
gini = 0.86
samples = 857
value = [2, 119, 103, 133, 113, 10, 2, 140, 135, 100]
class = 7
gini = 0.727
samples = 16
value = [1, 0, 2, 0, 5, 6, 2, 0, 0, 0]
class = 5
gini = 0.028
samples = 141
value = [139, 0, 0, 0, 0, 0, 0, 0, 0, 2]
class = 0
gini = 0.438
samples = 19
value = [0, 0, 1, 0, 0, 14, 1, 0, 1, 2]
class = 5
gini = 0.094
samples = 41
value = [0, 0, 0, 0, 0, 1, 0, 0, 1, 39
class = 9
gini = 0.611
samples = 188
value = [0, 23, 26, 13, 6, 111, 3, 3, 2,
class = 5
gini = 0.369
samples = 175
value = [0, 4, 10, 0, 21, 3, 137, 0, 0,
class = 6
gini = 0.549
samples = 199
value = [0, 9, 5, 7, 24, 4, 0, 130,
class = 7
gini = 0.839
samples = 658
value = [2, 110, 98, 126, 89, 6, 2, 10, 128, 87]
class = 8








samples = 1
value = [0, 0, 0, 0, 0, 0, 1, 0, 0, 0]
class = 6
samples = 3
class = 7
samples = 1
value = [0, 0, 0, 0, 0, 1, 0,
class = 5
samples = 1
value = [0, 0, 1, 0, 0,
class = 3
samples = 1
value = [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
class = 7
samples = 1
value = [0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
class = 4
samples = 70
value = [0, 70, 0, 0, 0, 0, 0, 0, 0, 0]
class = 1
samples = 84
value = [0, 0, 0, 0, 0, 0, 0, 0, 84, 0]
class = 8
samples = 5
value = [0, 0, 0, 0, 0, 0, 0, 0, 5, 0]
class = 8

Decision Tree evaluation:

The decision tree is quick to run, but it generally doesn't match kNN on raw pixel-like features. It's still a useful contrast because it behaves very differently from distance-based methods.The Decision Tree classifier achieved moderate accuracy relative to kNN-based methods. While trees are fast and interpretable, their reliance on axis-aligned splits makes them less effective for raw pixel-based representations.

The reduced F1-score compared to kNN suggests increased misclassification across multiple classes. This highlights the importance of matching algorithm choice to the structure of the data.

Final comparison tables + graphs

In [34]:

```python
# ============================================================
# Metrics comparison table (NumPy only)
# ============================================================
# Cell purpose:
# Build a compact comparison table that summarizes the performance of
# all implemented models using the same evaluation criteria.
#
# This table is especially useful for:
# - side-by-side comparison in the report
# - identifying the best-performing model
# - comparing scratch implementations with a library baseline
# ============================================================

def metrics_summary_numpy(y_true, y_pred, n_classes=10):
    # Converts inputs to clean 1D integer NumPy arrays.
    # This ensures consistent comparisons and avoids shape-related issues.
    y_true = np.asarray(y_true).astype(int).ravel()
    y_pred = np.asarray(y_pred).astype(int).ravel()

    eps = 1e-12
    # Small constant added to denominators to prevent division by zero.

    # Lists to store per-class statistics before aggregation.
    precisions, recalls, f1s, supports = [], [], [], []

    for c in range(n_classes):
        # Computes per-class statistics using a one-vs-rest interpretation.

        tp = np.sum((y_true == c) & (y_pred == c))
        # True positives: samples correctly predicted as class c.

        fp = np.sum((y_true != c) & (y_pred == c))
        # False positives: samples predicted as c but belonging to another class.

        fn = np.sum((y_true == c) & (y_pred != c))
        # False negatives: samples belonging to class c but predicted as another class.

        support = np.sum(y_true == c)
        # Support: total number of true samples for class c.

        p = tp / (tp + fp + eps)
        # Precision for class c:
        # Of all predictions labeled as c, how many are actually correct?

        r = tp / (tp + fn + eps)
        # Recall for class c:
        # Of all true class c samples, how many were correctly identified?

        f1 = 2 * p * r / (p + r + eps)
        # F1-score for class c:
        # Harmonic mean of precision and recall.

        precisions.append(p)
        recalls.append(r)
        f1s.append(f1)
        supports.append(support)
        # Stores per-class values for later aggregation.

    precisions = np.array(precisions)
    recalls    = np.array(recalls)
    f1s        = np.array(f1s)
    supports   = np.array(supports)
    # Converts lists to NumPy arrays for vectorized calculations.

    acc = float(np.mean(y_true == y_pred))
    # Overall accuracy:
    # fraction of test samples predicted correctly.

    macro_f1 = float(np.mean(f1s))
    # Macro F1-score:
    # simple average of F1 across all classes.
    # Each class contributes equally, regardless of frequency.

    weighted_f1 = float(np.sum(f1s * supports) / (np.sum(supports) + eps))
    # Weighted F1-score:
    # classes contribute proportionally to their support.
    # More frequent classes have more influence on the final score.

    return acc, macro_f1, weighted_f1
    # Returns the three summary metrics used in the comparison table.


models_for_table = [
    ("kNN (scratch)", y_pred_knn),
    ("kNN + PCA (scratch)", y_pred_knn_pca),
    ("Linear Reg (scratch, PCA)", y_pred_lin),
    ("Poly Reg deg-2 (scratch, PCA)", y_pred_poly),
    ("Decision Tree (library)", y_pred_dt),
]
# Defines the list of models to compare.
# Each entry contains:
# - a human-readable model name
# - the predicted labels produced by that model on the test set.

print(f"{'Model':35s} | {'Accuracy':8s} | {'Macro F1':8s} | {'Weighted F1':11s}")
# Prints the header row of the comparison table with aligned columns.

print("-"*75)
# Prints a horizontal separator for readability.

for name, yhat in models_for_table:
    # Iterates through each model and its predictions.

    acc, macro_f1, weighted_f1 = metrics_summary_numpy(y_test, yhat, n_classes=10)
    # Computes accuracy, macro F1, and weighted F1 using the same NumPy-only logic.

    print(f"{name:35s} | {acc:8.4f} | {macro_f1:8.4f} | {weighted_f1:11.4f}")
    # Prints one row per model with fixed decimal formatting.
```

```
Model                               | Accuracy | Macro F1 | Weighted F1
---------------------------------------------------------------------------
kNN (scratch)                       |   0.9667 |   0.9663 |      0.9666
kNN + PCA (scratch)                 |   0.9611 |   0.9602 |      0.9606
Linear Reg (scratch, PCA)           |   0.9056 |   0.9033 |      0.9039
Poly Reg deg-2 (scratch, PCA)       |   0.9722 |   0.9719 |      0.9721
Decision Tree (library)             |   0.8222 |   0.8209 |      0.8214
```

In [35]:

```python
# ------------------------------------------------------------
# Cell purpose:
# Recompute final accuracies from the latest prediction outputs
# and present a clean, consolidated comparison table.
#
# This step prevents the use of stale or outdated accuracy values
# in case any models were re-run or modified earlier.
# ------------------------------------------------------------

# --- Recompute accuracies from the FINAL predictions (prevents stale values) ---
acc_knn      = accuracy_numpy(y_test, y_pred_knn)
# Accuracy of kNN using scaled raw pixel features (64 dimensions).

acc_knn_pca  = accuracy_numpy(y_test, y_pred_knn_pca)
# Accuracy of kNN using PCA-reduced features.

acc_lin      = accuracy_numpy(y_test, y_pred_lin)
# Accuracy of the from-scratch linear regression model trained on PCA features.

acc_poly     = accuracy_numpy(y_test, y_pred_poly)
# Accuracy of the from-scratch degree-2 polynomial regression model on PCA features.

acc_dt       = accuracy_numpy(y_test, y_pred_dt)
# Accuracy of the library-based decision tree model using raw features.

final_results = [
    {
        "Model": "kNN (scratch)",
        "Type": "Supervised",
        "Feature space": "64 (scaled)",
        "Accuracy": round(acc_knn, 4)
    },
    {
        "Model": f"kNN + PCA({n_components})",
        "Type": "Supervised",
        "Feature space": f"{n_components} (PCA)",
        "Accuracy": round(acc_knn_pca, 4)
    },

    # Linear Regression (from scratch)
    {
        "Model": "Linear Regression (scratch)",
        "Type": "Supervised",
        "Feature space": f"{n_components} (PCA)",
        "Accuracy": round(acc_lin, 4)
    },

    # Polynomial Regression (from scratch, degree 2)
    {
        "Model": "Polynomial Reg deg-2 (scratch)",
        "Type": "Supervised",
        "Feature space": f"poly(deg2)+std on {n_components} PCA",
        "Accuracy": round(acc_poly, 4)
    },

    {
        "Model": "Decision Tree (lib)",
        "Type": "Supervised",
        "Feature space": "64 (raw)",
        "Accuracy": round(acc_dt, 4)
    },
]
# Builds a structured list summarizing:
# - model name
# - learning type
# - feature representation
# - final test accuracy
# This structure makes it easy to format or export results later.

print("=" * 95)
# Prints a visual separator for the table.

print(f"{'Model':32s} {'Type':17s} {'Feature space':35s} Accuracy")
# Prints the table header with aligned columns.

print("=" * 95)

for row in final_results:
    # Iterates through each model summary and prints one row.

    print(
        f"{row['Model']:32s} "
        f"{row['Type']:17s} "
        f"{row['Feature space']:35s} "
        f"{row['Accuracy']}"
    )

print("=" * 95)
# Prints a closing separator line.
```

```
===============================================================================================
Model                            Type              Feature space                       Accuracy
===============================================================================================
kNN (scratch)                    Supervised        64 (scaled)                         0.9667
kNN + PCA(20)                    Supervised        20 (PCA)                            0.9611
Linear Regression (scratch)      Supervised        20 (PCA)                            0.9056
Polynomial Reg deg-2 (scratch)   Supervised        poly(deg2)+std on 20 PCA            0.9722
Decision Tree (lib)              Supervised        64 (raw)                            0.8222
===============================================================================================
```

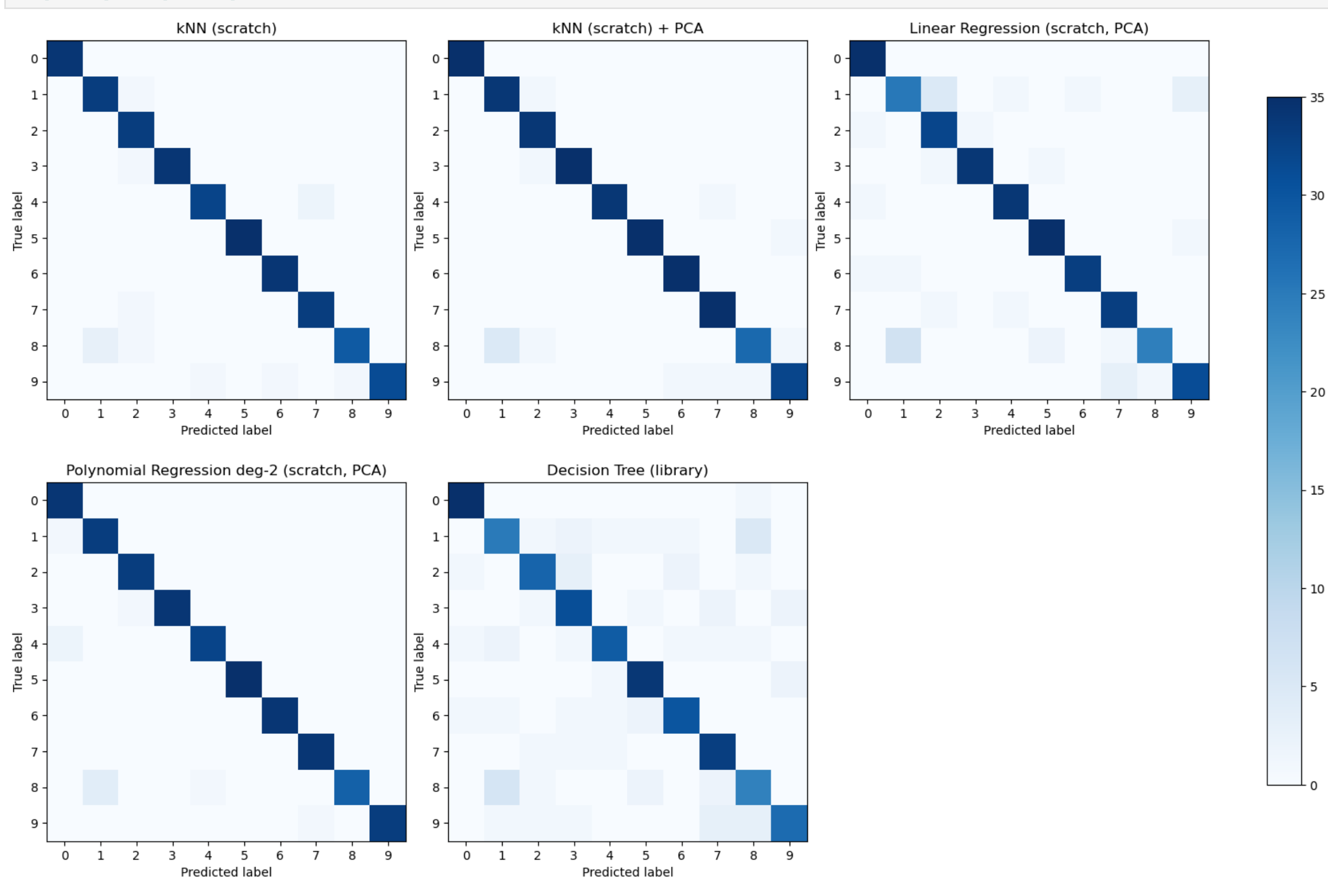
In [36]:

```python
# ------------------------------------------------------------
# Cell purpose:
# Print detailed per-class classification reports for all models.
#
# Each report shows:
# - Precision per class
# - Recall per class
# - F1-score per class
# - Support (number of true samples per class)
#
# All reports are generated using the same NumPy-only function
# to ensure consistency and fair comparison.
# ------------------------------------------------------------

print("kNN (from scratch)")
# Prints a heading to clearly separate results for the kNN model
# implemented entirely from scratch using NumPy.

class_report_numpy(y_test, y_pred_knn, n_classes=10)
# Generates and prints a per-class report for kNN using raw scaled features.
# This shows how well kNN performs on each digit class individually.


print("\nkNN (from scratch) + PCA")
# Prints a heading for the kNN model applied after PCA dimensionality reduction.
# The leading newline creates visual separation from the previous report.

class_report_numpy(y_test, y_pred_knn_pca, n_classes=10)
# Generates the per-class report for kNN operating in PCA feature space.
# This allows direct comparison with kNN on raw features.


print("\nLinear Regression (scratch, PCA)")
# Prints a heading for the from-scratch linear regression classifier.
# This model uses PCA features and one-hot MSE optimization.

class_report_numpy(y_test, y_pred_lin, n_classes=10)
# Prints per-class precision, recall, F1-score, and support
# for the linear regression model.


print("\nPolynomial Regression deg-2 (scratch, PCA)")
# Prints a heading for the degree-2 polynomial regression model.
# This model captures non-linear relationships in PCA feature space.

class_report_numpy(y_test, y_pred_poly, n_classes=10)
# Outputs a detailed per-class report for the polynomial regression model.
# Differences in precision and recall across digits highlight
# which classes benefit most from non-linear feature interactions.


print("\nDecision Tree (library)")
# Prints a heading for the library-based decision tree classifier.
# This serves as a reference baseline against scratch implementations.

class_report_numpy(y_test, y_pred_dt, n_classes=10)
# Prints the per-class performance report for the decision tree model.
# Comparing this report with others reveals how tree-based rules
# differ from distance-based and regression-based approaches.
```

```
kNN (from scratch)
Class | Precision | Recall | F1-score | Support
-------------------------------------------------------
    0 |    1.0000 | 1.0000 |  1.0000 |      36
    1 |    0.9211 | 0.9722 |  0.9459 |      36
    2 |    0.8974 | 1.0000 |  0.9459 |      35
    3 |    1.0000 | 0.9730 |  0.9863 |      37
    4 |    0.9714 | 0.9444 |  0.9577 |      36
    5 |    1.0000 | 1.0000 |  1.0000 |      37
    6 |    0.9730 | 1.0000 |  0.9863 |      36
    7 |    0.9459 | 0.9722 |  0.9589 |      36
    8 |    0.9688 | 0.8857 |  0.9254 |      35
    9 |    1.0000 | 0.9167 |  0.9565 |      36
-------------------------------------------------------
Accuracy  : 0.9667
Macro avg : Precision=0.9678, Recall=0.9664, F1=0.9663

kNN (from scratch) + PCA
Class | Precision | Recall | F1-score | Support
-------------------------------------------------------
    0 |    1.0000 | 1.0000 |  1.0000 |      36
    1 |    0.8750 | 0.9722 |  0.9211 |      36
    2 |    0.9211 | 1.0000 |  0.9589 |      35
    3 |    1.0000 | 0.9730 |  0.9863 |      37
    4 |    1.0000 | 0.9722 |  0.9859 |      36
    5 |    1.0000 | 0.9730 |  0.9863 |      37
    6 |    0.9730 | 1.0000 |  0.9863 |      36
    7 |    0.9474 | 1.0000 |  0.9730 |      36
    8 |    0.9655 | 0.8000 |  0.8750 |      35
    9 |    0.9429 | 0.9167 |  0.9296 |      36
-------------------------------------------------------
Accuracy  : 0.9611
Macro avg : Precision=0.9625, Recall=0.9607, F1=0.9602

Linear Regression (scratch, PCA)
Class | Precision | Recall | F1-score | Support
-------------------------------------------------------
    0 |    0.9231 | 1.0000 |  0.9600 |      36
    1 |    0.7647 | 0.7222 |  0.7429 |      36
    2 |    0.8250 | 0.9429 |  0.8800 |      35
    3 |    0.9722 | 0.9459 |  0.9589 |      37
    4 |    0.9459 | 0.9722 |  0.9589 |      36
    5 |    0.9231 | 0.9730 |  0.9474 |      37
    6 |    0.9714 | 0.9444 |  0.9577 |      36
    7 |    0.8947 | 0.9444 |  0.9189 |      36
    8 |    0.9615 | 0.7143 |  0.8197 |      35
    9 |    0.8889 | 0.8889 |  0.8889 |      36
-------------------------------------------------------
Accuracy  : 0.9056
Macro avg : Precision=0.9071, Recall=0.9048, F1=0.9033

Polynomial Regression deg-2 (scratch, PCA)
Class | Precision | Recall | F1-score | Support
-------------------------------------------------------
    0 |    0.9231 | 1.0000 |  0.9600 |      36
    1 |    0.8974 | 0.9722 |  0.9333 |      36
    2 |    0.9722 | 1.0000 |  0.9859 |      35
    3 |    1.0000 | 0.9730 |  0.9863 |      37
    4 |    0.9714 | 0.9444 |  0.9577 |      36
    5 |    1.0000 | 1.0000 |  1.0000 |      37
    6 |    1.0000 | 1.0000 |  1.0000 |      36
    7 |    0.9730 | 1.0000 |  0.9863 |      36
    8 |    1.0000 | 0.8571 |  0.9231 |      35
    9 |    1.0000 | 0.9722 |  0.9859 |      36
-------------------------------------------------------
Accuracy  : 0.9722
Macro avg : Precision=0.9737, Recall=0.9719, F1=0.9719

Decision Tree (library)
Class | Precision | Recall | F1-score | Support
-------------------------------------------------------
    0 |    0.9211 | 0.9722 |  0.9459 |      36
    1 |    0.7143 | 0.6944 |  0.7042 |      36
    2 |    0.8485 | 0.8000 |  0.8235 |      35
    3 |    0.7750 | 0.8378 |  0.8052 |      37
    4 |    0.8788 | 0.8056 |  0.8406 |      36
    5 |    0.8500 | 0.9189 |  0.8831 |      37
    6 |    0.8824 | 0.8333 |  0.8571 |      36
    7 |    0.8049 | 0.9167 |  0.8571 |      36
    8 |    0.6857 | 0.6857 |  0.6857 |      35
    9 |    0.8710 | 0.7500 |  0.8060 |      36
-------------------------------------------------------
Accuracy  : 0.8222
Macro avg : Precision=0.8232, Recall=0.8215, F1=0.8209
```

In [37]:

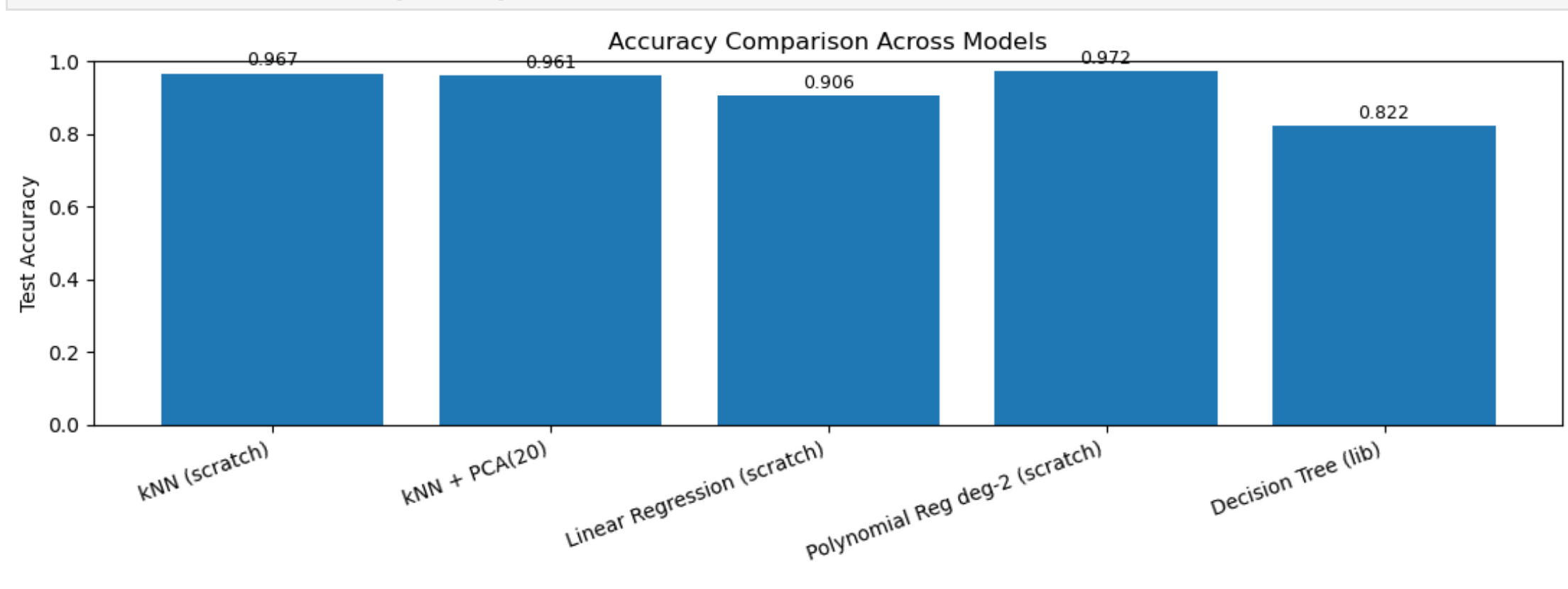
```python
# ============================================================
# Confusion matrices (NumPy only) — UPDATED (no k-Means)
# Models: kNN, kNN+PCA, Linear Reg (scratch), Poly Reg (scratch), Decision Tree
# ============================================================
# Cell purpose:
# Display confusion matrices for multiple models side-by-side
# in one figure to support visual comparison of error patterns.
#
# A confusion matrix helps answer questions like:
# - Which digits are consistently confused?
# - Which model produces the cleanest diagonal (best correctness)?
# - Do errors cluster around visually similar digits?
# ============================================================

n_classes = 10
# Sets the number of classes (digits 0 through 9).
# This controls tick labels and the confusion matrix size.

# These must already exist from earlier cells:
# y_pred_knn, y_pred_knn_pca, y_pred_lin, y_pred_poly, y_pred_dt

cm_knn      = confusion_matrix_numpy(y_test, y_pred_knn,      n_classes)
# Confusion matrix for kNN using scaled raw pixel features.

cm_knn_pca  = confusion_matrix_numpy(y_test, y_pred_knn_pca,  n_classes)
# Confusion matrix for kNN using PCA-reduced features.

cm_lin      = confusion_matrix_numpy(y_test, y_pred_lin,      n_classes)
# Confusion matrix for linear regression trained from scratch on PCA features.

cm_poly     = confusion_matrix_numpy(y_test, y_pred_poly,     n_classes)
# Confusion matrix for degree-2 polynomial regression trained from scratch on PCA features.

cm_dt       = confusion_matrix_numpy(y_test, y_pred_dt,       n_classes)
# Confusion matrix for the library-based decision tree model using raw features.

cms = [
    ("kNN (scratch)", cm_knn),
    ("kNN (scratch) + PCA", cm_knn_pca),
    ("Linear Regression (scratch, PCA)", cm_lin),
    ("Polynomial Regression deg-2 (scratch, PCA)", cm_poly),
    ("Decision Tree (library)", cm_dt),
]
# Packages each confusion matrix with a title so the plotting loop can handle them uniformly.

fig, axes = plt.subplots(2, 3, figsize=(15, 10), constrained_layout=True)
# Creates a grid of subplots (2 rows × 3 columns).
# Six slots are created even though only five matrices exist.
# constrained_layout=True helps spacing so titles and labels do not overlap.

for ax, (title, mat) in zip(axes.ravel(), cms):
    # Iterates over axes and confusion matrix pairs together.
    # axes.ravel() flattens the 2D axes grid into a 1D list for easy looping.

    im = ax.imshow(mat, cmap="Blues")
    # Displays the confusion matrix as an image.
    # Darker color corresponds to larger counts.

    ax.set_title(title)
    # Adds the model name above each subplot.

    ax.set_xlabel("Predicted label")
    ax.set_ylabel("True label")
    # Labels axes to match standard confusion matrix convention.

    ax.set_xticks(range(n_classes))
    ax.set_yticks(range(n_classes))
    # Adds tick marks for each class index 0–9.

# Hide the last empty subplot (since five plots are shown in a 2x3 grid)
axes.ravel()[-1].axis("off")
# Turns off the unused sixth subplot to avoid an empty chart being displayed.

# Shared colorbar
fig.colorbar(im, ax=axes.ravel().tolist(), shrink=0.8)
# Adds one colorbar shared across all subplots.
# shrink reduces colorbar size so it fits neatly beside the grid.
# im is taken from the last plotted matrix, but the colormap scale is consistent across axes.

plt.show()
# Displays the full comparison figure.

print("""
INTERPRETATION (Confusion Matrices — Model Comparison):

Each confusion matrix shows true labels on the y-axis and predicted labels on the
x-axis. Darker diagonal cells indicate correct predictions (true positives), while
off-diagonal cells represent misclassifications.

1) kNN (scratch):
The matrix shows a strong diagonal, indicating good overall classification.
Most errors occur between visually similar digits (e.g., 8 vs 9), which is expected
for pixel-space distance-based methods. Performance is solid but limited by the
high dimensionality of raw features.

2) kNN (scratch) + PCA:
The diagonal becomes slightly cleaner compared to raw kNN, with fewer off-diagonal
confusions. PCA reduces noise and redundancy, improving distance calculations and
leading to better class separation and accuracy.

3) Linear Regression (scratch, PCA):
The diagonal is clear but slightly less sharp than kNN + PCA. Some classes show
mild confusion, reflecting the linear nature of the model, which cannot fully
capture complex class boundaries despite PCA preprocessing.

4) Polynomial Regression deg-2 (scratch, PCA):
This matrix shows one of the strongest diagonals with very few off-diagonal errors.
The polynomial model captures non-linear relationships, leading to improved class
separation and higher accuracy, especially for difficult digit pairs.

5) Decision Tree (library):
The diagonal is strong, but off-diagonal errors are more scattered across classes.
This reflects the tree’s tendency to overfit local patterns, resulting in some
inconsistent misclassifications compared to PCA-based models.

Overall, PCA-based models show clearer diagonals and fewer misclassifications.
Polynomial regression achieves the best class separation, while kNN + PCA provides
a strong balance between accuracy, simplicity, and stability.
""")
# Interpretation text summarizes what the combined figure shows and provides
# report-style comparison points across the five models.
```



```
INTERPRETATION (Confusion Matrices — Model Comparison):

Each confusion matrix shows true labels on the y-axis and predicted labels on the
x-axis. Darker diagonal cells indicate correct predictions (true positives), while
off-diagonal cells represent misclassifications.

1) kNN (scratch):
The matrix shows a strong diagonal, indicating good overall classification.
Most errors occur between visually similar digits (e.g., 8 vs 9), which is expected
for pixel-space distance-based methods. Performance is solid but limited by the
high dimensionality of raw features.

2) kNN (scratch) + PCA:
The diagonal becomes slightly cleaner compared to raw kNN, with fewer off-diagonal
confusions. PCA reduces noise and redundancy, improving distance calculations and
leading to better class separation and accuracy.

3) Linear Regression (scratch, PCA):
The diagonal is clear but slightly less sharp than kNN + PCA. Some classes show
mild confusion, reflecting the linear nature of the model, which cannot fully
capture complex class boundaries despite PCA preprocessing.

4) Polynomial Regression deg-2 (scratch, PCA):
This matrix shows one of the strongest diagonals with very few off-diagonal errors.
The polynomial model captures non-linear relationships, leading to improved class
separation and higher accuracy, especially for difficult digit pairs.

5) Decision Tree (library):
The diagonal is strong, but off-diagonal errors are more scattered across classes.
This reflects the tree’s tendency to overfit local patterns, resulting in some
inconsistent misclassifications compared to PCA-based models.

Overall, PCA-based models show clearer diagonals and fewer misclassifications.
Polynomial regression achieves the best class separation, while kNN + PCA provides
a strong balance between accuracy, simplicity, and stability.
```

In [38]:

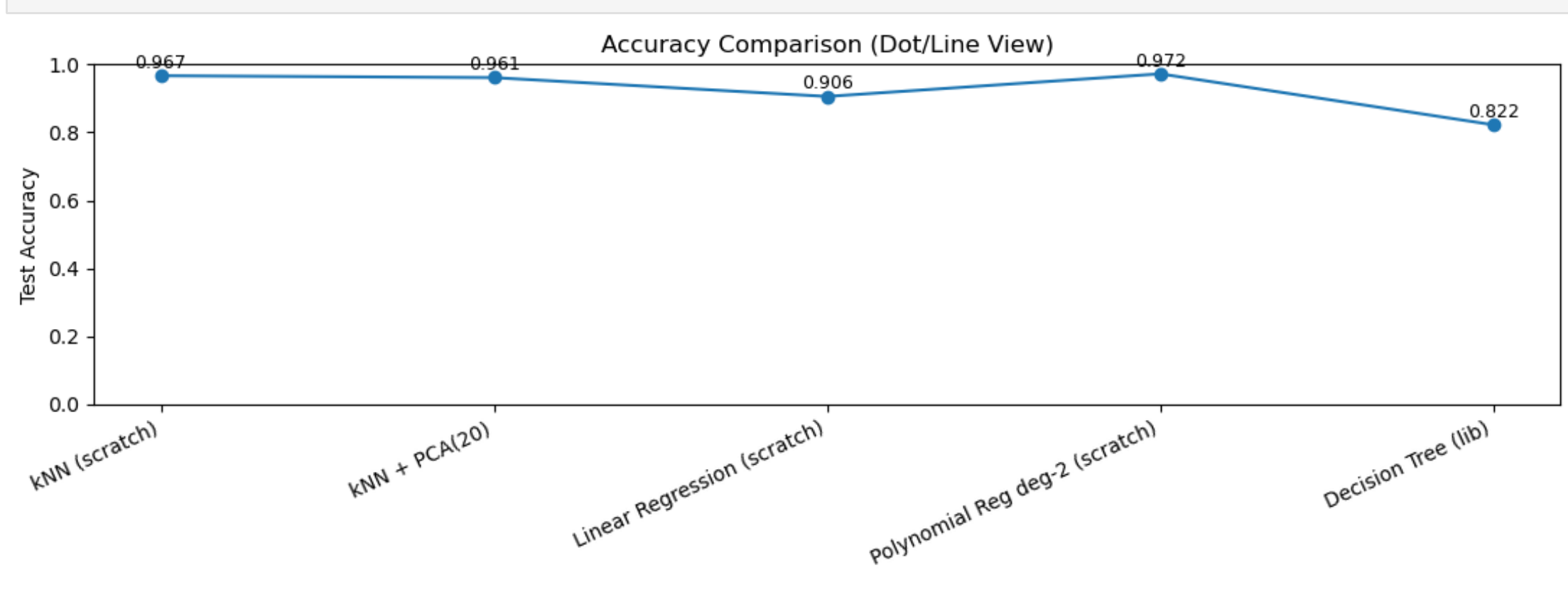
```python
# ------------------------------------------------------------
# Cell purpose:
# Visualize and compare final test accuracies of all models
# using a bar chart for quick, intuitive comparison.
# ------------------------------------------------------------

# Accuracy bar chart
models = [r["Model"] for r in final_results]
# Extracts model names from the final results table.
# These names are used as labels on the x-axis.

accs = [r["Accuracy"] for r in final_results]
# Extracts corresponding test accuracy values for each model.

plt.figure(figsize=(11, 4))
# Creates a wide figure so model names fit without crowding.

bars = plt.bar(models, accs)
# Draws a bar for each model where bar height represents test accuracy.

plt.ylim(0, 1.0)
# Fixes the y-axis range from 0 to 1 for accuracy values.
# This makes relative differences between models easier to compare.

plt.ylabel("Test Accuracy")
# Labels the y-axis to indicate the metric being plotted.

plt.title("Accuracy Comparison Across Models")
# Adds a descriptive title explaining what the chart represents.

plt.xticks(rotation=20, ha="right")
# Rotates x-axis labels slightly to prevent overlap and improve readability.

# Add accuracy labels on top of bars
for bar, acc in zip(bars, accs):
    # Iterates over each bar and its accuracy value.

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.01,
        f"{acc:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )
    # Places a numeric accuracy label slightly above each bar.
    # This allows exact values to be read directly from the chart.

plt.tight_layout()
# Adjusts spacing so all labels and titles fit neatly within the figure.

plt.show()
# Displays the accuracy comparison bar chart.

print("""
INTERPRETATION (Accuracy Comparison Across Models):

This bar chart compares the test accuracy achieved by each model on the same dataset.

kNN (scratch) and kNN + PCA both achieve high accuracy, with a slight improvement
after applying PCA. This shows that dimensionality reduction helps kNN by removing
noise and improving distance calculations.

Linear regression (scratch, PCA) achieves lower accuracy than kNN-based models,
reflecting the limitation of linear decision boundaries for complex image data.

Polynomial regression (degree-2, scratch, PCA) achieves the highest accuracy among
all models. This demonstrates that introducing non-linear feature interactions
significantly improves classification performance.

The decision tree classifier achieves the lowest accuracy. While it can capture
non-linear patterns, it is more prone to overfitting and less stable compared to
PCA-based models.

Overall, the chart highlights the trade-off between model complexity and performance,
with polynomial regression providing the best balance for this dataset.
""")
# Interpretation text explains the observed trends and connects them
# to model characteristics in a report-ready form.
```



```
INTERPRETATION (Accuracy Comparison Across Models):

This bar chart compares the test accuracy achieved by each model on the same dataset.

kNN (scratch) and kNN + PCA both achieve high accuracy, with a slight improvement
after applying PCA. This shows that dimensionality reduction helps kNN by removing
noise and improving distance calculations.

Linear regression (scratch, PCA) achieves lower accuracy than kNN-based models,
reflecting the limitation of linear decision boundaries for complex image data.

Polynomial regression (degree-2, scratch, PCA) achieves the highest accuracy among
all models. This demonstrates that introducing non-linear feature interactions
significantly improves classification performance.

The decision tree classifier achieves the lowest accuracy. While it can capture
non-linear patterns, it is more prone to overfitting and less stable compared to
PCA-based models.

Overall, the chart highlights the trade-off between model complexity and performance,
with polynomial regression providing the best balance for this dataset.
```

In [39]:

```python
# ------------------------------------------------------------
# Cell purpose:
# Present the same accuracy values using a dot-and-line plot.
# This view emphasizes relative trends and ranking between models
# rather than absolute bar heights.
# ------------------------------------------------------------

# ---- Same numbers, different view: dots + a simple line ----
x = np.arange(len(models))
# Creates a numeric index for each model.
# This index is used as the x-axis position for plotting.
# Models themselves will later be shown as tick labels.

plt.figure(figsize=(11, 4))
# Creates a wide figure so model names fit comfortably on the x-axis.

plt.plot(x, accs, marker="o")
# Plots accuracy values as points connected by a line.
# Each marker corresponds to one model.
# The line helps visually track performance changes across models.

plt.ylim(0, 1.0)
# Fixes the y-axis range between 0 and 1 since accuracy is a proportion.
# This ensures consistent visual scaling across models.

plt.xticks(x, models, rotation=25, ha="right")
# Replaces numeric x-axis values with model names.
# Rotating labels improves readability when names are long.

plt.ylabel("Test Accuracy")
# Labels the y-axis to indicate that accuracy is being measured.

plt.title("Accuracy Comparison (Dot/Line View)")
# Adds a descriptive title explaining the purpose of the plot.

# Label each point with its accuracy
for xi, ai in zip(x, accs):
    # Iterates over each model index and its corresponding accuracy.

    plt.text(
        xi,
        ai + 0.01,
        f"{ai:.3f}",
        ha="center",
        va="bottom",
        fontsize=9
    )
    # Places a text label slightly above each point.
    # This shows the exact accuracy value without needing to read from the axis.

plt.tight_layout()
# Adjusts spacing to ensure labels, ticks, and titles do not overlap.

plt.show()
# Displays the dot-and-line accuracy comparison plot.

print("""
INTERPRETATION (Accuracy Comparison — Dot/Line View):

This plot presents the same test accuracy results as the bar chart, but in a
dot-and-line format to highlight relative performance trends across models.

The connected points make it easier to see how accuracy changes from one model
to the next, rather than focusing on individual bar heights.

kNN-based methods appear close together at the high-accuracy end, with a small
improvement when PCA is applied. Linear regression drops noticeably, reflecting
the limitation of linear decision boundaries for image data.

Polynomial regression (degree-2) stands out as the highest point, confirming that
non-linear feature interactions lead to the strongest performance.

The final drop for the decision tree indicates weaker generalization compared to
PCA-based approaches.

Overall, this view emphasizes model ranking and performance trends, making it
easy to identify which models perform similarly and which differ significantly.
""")
# Interpretation text explains why this visualization is useful and
# how to read trends and relative differences between models.
```

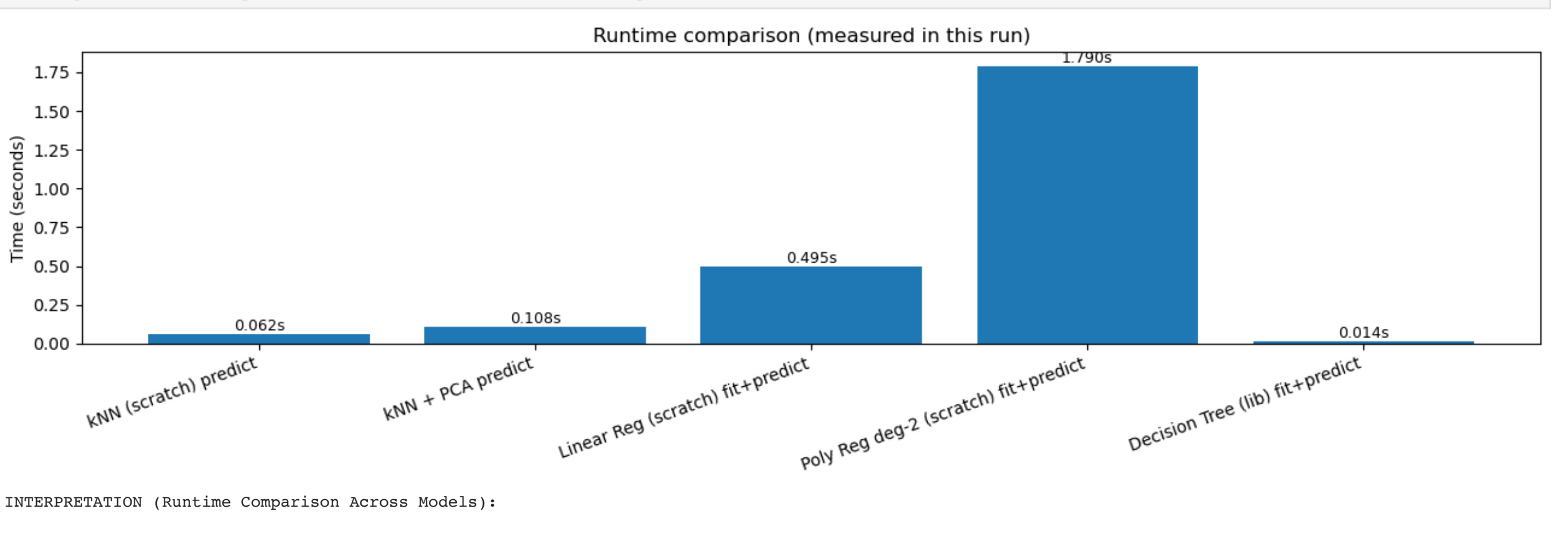


```
INTERPRETATION (Accuracy Comparison — Dot/Line View):

This plot presents the same test accuracy results as the bar chart, but in a
dot-and-line format to highlight relative performance trends across models.

The connected points make it easier to see how accuracy changes from one model
to the next, rather than focusing on individual bar heights.

kNN-based methods appear close together at the high-accuracy end, with a small
improvement when PCA is applied. Linear regression drops noticeably, reflecting
the limitation of linear decision boundaries for image data.

Polynomial regression (degree-2) stands out as the highest point, confirming that
non-linear feature interactions lead to the strongest performance.

The final drop for the decision tree indicates weaker generalization compared to
PCA-based approaches.

Overall, this view emphasizes model ranking and performance trends, making it
easy to identify which models perform similarly and which differ significantly.
```

In [40]:

```python
# ------------------------------------------------------------
# Cell purpose:
# Compare computational cost of all models using a runtime bar chart.
# This visualization complements accuracy comparisons by showing
# how expensive each approach is in terms of execution time.
# ------------------------------------------------------------

# Runtime comparison bar chart — improved readability

timing_results = [
    ("kNN (scratch) predict", time_knn),
    # Runtime for kNN using raw scaled features.
    # This includes prediction time only, since kNN has no training phase.

    ("kNN + PCA predict", time_knn_pca),
    # Runtime for kNN operating in PCA-reduced space.
    # Prediction time is typically lower due to fewer dimensions.

    ("Linear Reg (scratch) fit+predict", time_lin),
    # Total runtime for linear regression trained from scratch.
    # Includes both gradient descent training and prediction.

    ("Poly Reg deg-2 (scratch) fit+predict", time_poly),
    # Total runtime for degree-2 polynomial regression.
    # Larger feature space increases both training and prediction cost.

    ("Decision Tree (lib) fit+predict", time_dt),
    # Total runtime for the library-based decision tree.
    # Trees train and predict quickly compared to iterative models.
]

labels = [t[0] for t in timing_results]
# Extracts model labels for the x-axis.

times  = [t[1] for t in timing_results]
# Extracts runtime values (in seconds) for each model.

plt.figure(figsize=(13, 4))
# Creates a wide figure so long model names fit comfortably.

bars = plt.bar(labels, times)
# Draws a bar for each model where height represents runtime in seconds.

plt.ylabel("Time (seconds)")
# Labels the y-axis to indicate runtime units.

plt.title("Runtime comparison (measured in this run)")
```

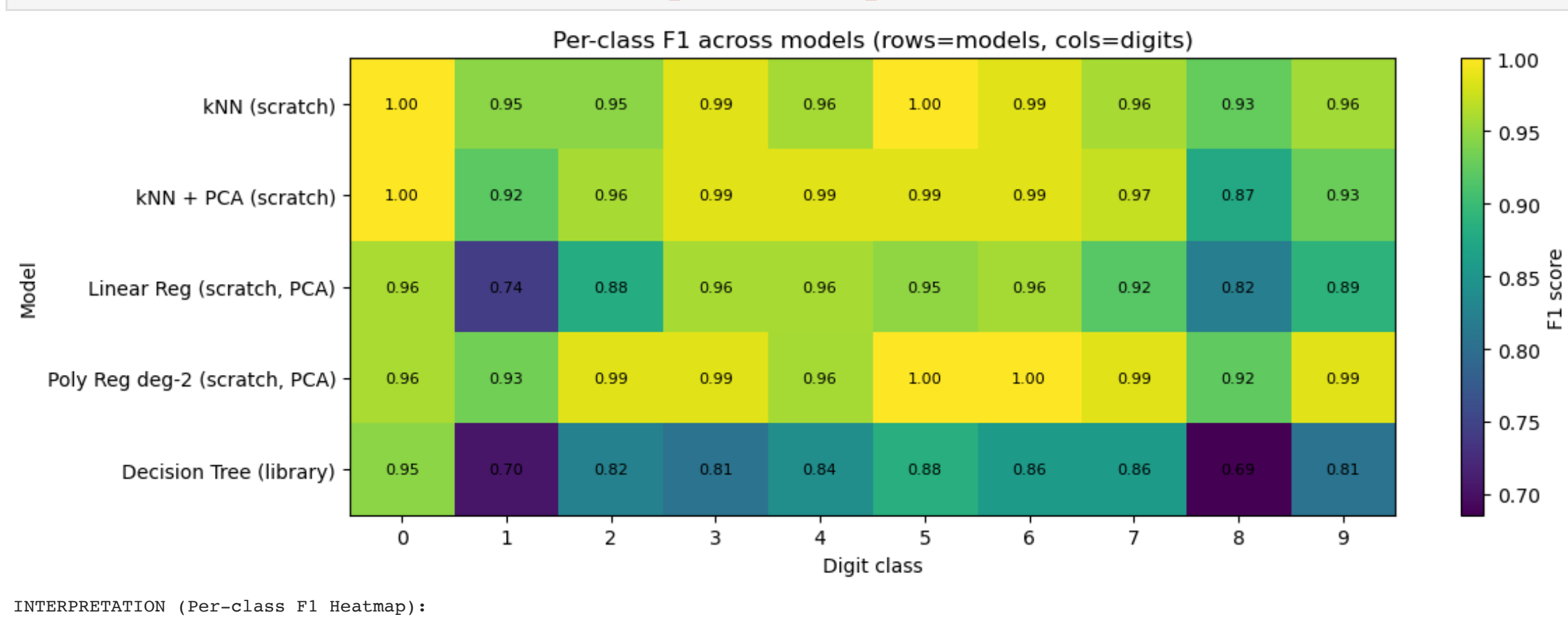
```
# Adds a title clarifying that times are specific to this execution environment.

plt.xticks(rotation=20, ha="right")
# Rotates x-axis labels slightly for better readability.

# add labels on bars
for bar, t in zip(bars, times):
    # Iterates over bars and corresponding runtime values.

    plt.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height(),
        f"{t:.3f}s",
        ha="center",
        va="bottom",
        fontsize=9
    )
    # Places a numeric runtime label above each bar.

plt.tight_layout()
# Adjusts layout so all labels and titles fit neatly.

plt.show()
# Displays the runtime comparison bar chart.

print("""
INTERPRETATION (Runtime Comparison Across Models):

This bar chart compares the runtime of each model measured during this execution.
For kNN-based methods, the reported time corresponds to prediction time only, while
for regression and tree models it includes both training and prediction.

kNN (scratch) and kNN + PCA have low runtimes, with PCA introducing only a small
overhead. Linear regression requires more time due to iterative gradient descent
training.

Polynomial regression (degree-2) has the highest runtime, reflecting the large number
of polynomial features and the increased computational cost of optimization.

The decision tree has the lowest runtime overall, as it trains quickly and performs
fast predictions once the tree is built.

Overall, this plot highlights the trade-off between accuracy and computational cost,
with polynomial regression achieving high accuracy at the expense of significantly
longer runtime.
""")
# Interpretation text explains runtime differences in a clear way.
```



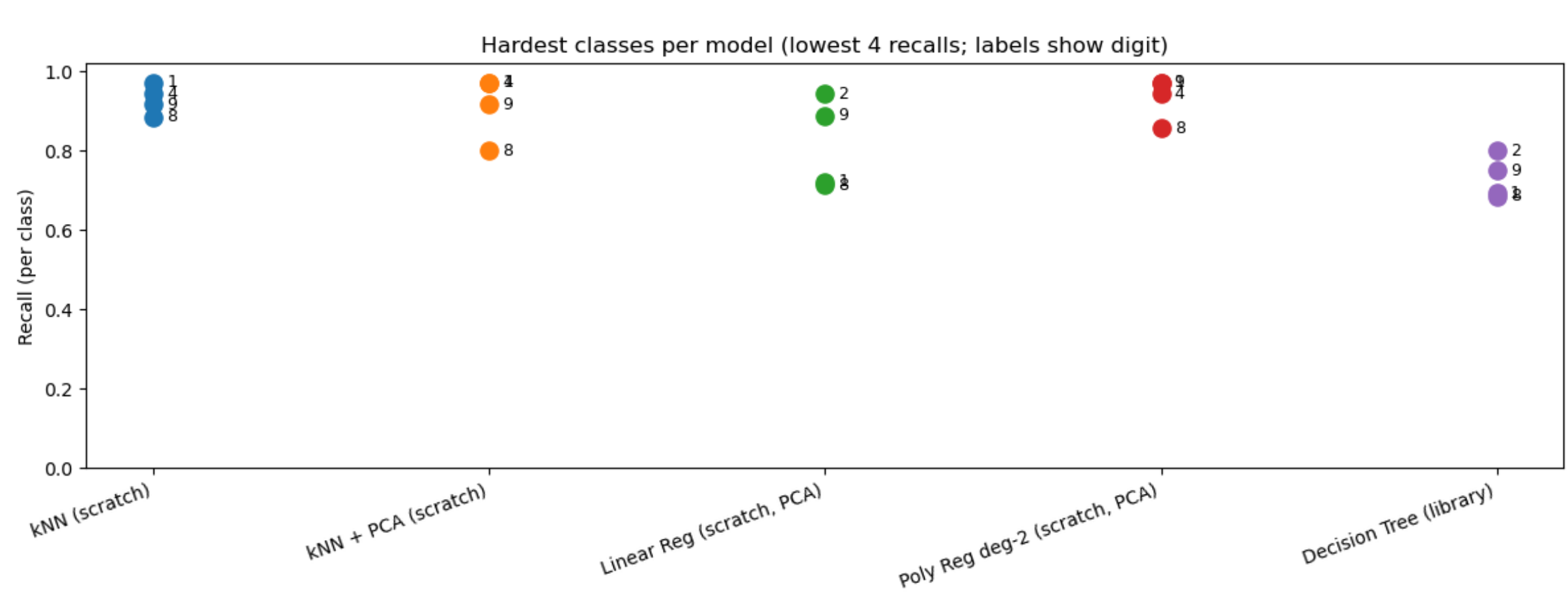
```
INTERPRETATION (Runtime Comparison Across Models):

This bar chart compares the runtime of each model measured during this execution.
For kNN-based methods, the reported time corresponds to prediction time only, while
for regression and tree models it includes both training and prediction.

kNN (scratch) and kNN + PCA have low runtimes, with PCA introducing only a small
overhead. Linear regression requires more time due to iterative gradient descent
training.

Polynomial regression (degree-2) has the highest runtime, reflecting the large number
of polynomial features and the increased computational cost of optimization.

The decision tree has the lowest runtime overall, as it trains quickly and performs
fast predictions once the tree is built.

Overall, this plot highlights the trade-off between accuracy and computational cost,
with polynomial regression achieving high accuracy at the expense of significantly
longer runtime.
```

In [41]:

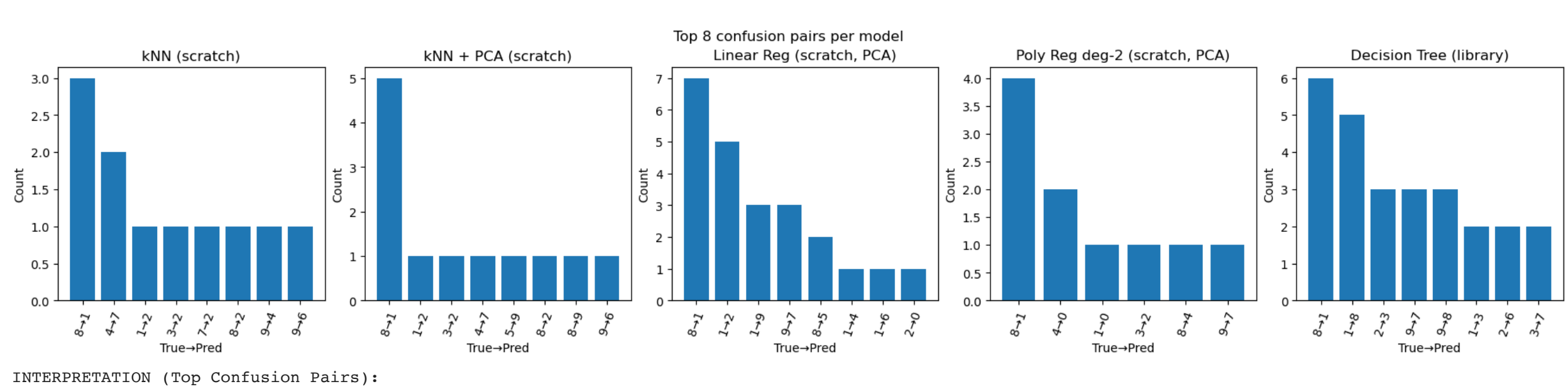
```
# ============================================================
# EXTRA (In-depth) model comparison graphs — NumPy + Matplotlib

# ============================================================


# ----------------------------
# Safety checks for required vars
# ----------------------------
required_preds = ["y_pred_knn", "y_pred_knn_pca", "y_pred_lin", "y_pred_poly", "y_pred_dt", "y_test"]
missing = [v for v in required_preds if v not in globals()]
if missing:
    raise NameError(f"Missing required variables before running comparison plots: {missing}")

# Standard model registry (keeps plots consistent)
models_dict = {
    "kNN (scratch)": y_pred_knn,
    "kNN + PCA (scratch)": y_pred_knn_pca,
    "Linear Reg (scratch, PCA)": y_pred_lin,
    "Poly Reg deg-2 (scratch, PCA)": y_pred_poly,
    "Decision Tree (library)": y_pred_dt,
}

model_names = list(models_dict.keys())
preds_list = [models_dict[m] for m in model_names]
n_classes = 10


# ----------------------------
# Helper: confusion matrix + per-class metrics (from CM only)
# ----------------------------
def confusion_matrix_numpy_local(y_true, y_pred, n_classes=10):
    y_true = np.asarray(y_true).astype(int).ravel()
    y_pred = np.asarray(y_pred).astype(int).ravel()
    cm = np.zeros((n_classes, n_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm

def per_class_metrics_from_cm(cm, eps=1e-12):
    cm = np.asarray(cm, dtype=np.float64)
    TP = np.diag(cm)
    FP = cm.sum(axis=0) - TP
    FN = cm.sum(axis=1) - TP
    support = cm.sum(axis=1)

    precision = TP / (TP + FP + eps)
    recall    = TP / (TP + FN + eps)
    f1        = 2 * precision * recall / (precision + recall + eps)

    return precision, recall, f1, support

def top_confusions_from_cm(cm, top_k=10):
    cm = np.asarray(cm, dtype=int).copy()
    np.fill_diagonal(cm, 0)
    pairs = []
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            if i != j and cm[i, j] > 0:
                pairs.append((cm[i, j], i, j))
    pairs.sort(reverse=True, key=lambda x: x[0])
    return pairs[:top_k]


# Precompute CMs + per-class metrics for all models
cms = {}
pc_precision = {}
pc_recall = {}
pc_f1 = {}
pc_support = {}

for name, yhat in models_dict.items():
    cm = confusion_matrix_numpy_local(y_test, yhat, n_classes=n_classes)
    p, r, f1, sup = per_class_metrics_from_cm(cm)
    cms[name] = cm
    pc_precision[name] = p
    pc_recall[name] = r
    pc_f1[name] = f1
    pc_support[name] = sup


# ============================================================
# 1) Per-class F1 heatmap across models (most report-friendly)
# ============================================================
f1_matrix = np.vstack([pc_f1[m] for m in model_names])  # shape: (n_models, n_classes)

plt.figure(figsize=(12, 4.2))
plt.imshow(f1_matrix, aspect="auto", interpolation="nearest")
plt.title("Per-class F1 across models (rows=models, cols=digits)")
plt.xlabel("Digit class")
plt.ylabel("Model")
plt.xticks(range(n_classes))
plt.yticks(range(len(model_names)), model_names)
plt.colorbar(label="F1 score")

# annotate values
for i in range(f1_matrix.shape[0]):
    for j in range(f1_matrix.shape[1]):
        plt.text(j, i, f"{f1_matrix[i, j]:.2f}", ha="center", va="center", fontsize=8)

plt.tight_layout()
plt.show()

print("""
INTERPRETATION (Per-class F1 Heatmap):

This heatmap shows how each model performs on each digit class individually.
Rows represent models and columns represent digit labels (0–9).

Bright values indicate strong per-class performance, while darker values highlight
classes where a model struggles. This view usually reveals that overall accuracy
can hide weak classes, especially visually similar digits that cause confusion.
""")


# ============================================================
# 2) "Hard classes" plot: lowest recall digits for each model
# ============================================================
top_n = 4  # how many difficult classes to display per model

plt.figure(figsize=(12, 4.5))
x = np.arange(len(model_names))

for idx, m in enumerate(model_names):
    r = pc_recall[m]
    hard = np.argsort(r)[:top_n]
    plt.scatter([idx]*top_n, r[hard], s=90)
    for c in hard:
        plt.text(idx + 0.04, r[c], f"{c}", va="center", fontsize=9)

plt.xticks(x, model_names, rotation=20, ha="right")
plt.ylim(0, 1.02)
plt.ylabel("Recall (per class)")
plt.title(f"Hardest classes per model (lowest {top_n} recalls; labels show digit)")
plt.tight_layout()
plt.show()

print("""
INTERPRETATION (Hardest Classes by Recall):

Recall highlights how often the true class is successfully detected.
Low recall for a digit means many false negatives (the digit is frequently missed).

The annotated digit labels show which classes are consistently difficult and
whether the difficult set changes from model to model.
""")


# ============================================================
# 3) Top confusion pairs per model (visual comparison)
# ============================================================
top_k = 8

fig, axes = plt.subplots(1, len(model_names), figsize=(18, 3.6), constrained_layout=True)

for ax, m in zip(axes, model_names):
    pairs = top_confusions_from_cm(cms[m], top_k=top_k)
    labels = [f"{i}→{j}" for _, i, j in pairs]
    counts = [cnt for cnt, _, _ in pairs]

    ax.bar(labels, counts)
    ax.set_title(m)
    ax.set_xlabel("True→Pred")
    ax.tick_params(axis="x", labelrotation=70)
    ax.set_ylabel("Count")

plt.suptitle(f"Top {top_k} confusion pairs per model", y=1.05)
plt.show()

print("""
INTERPRETATION (Top Confusion Pairs):

Each panel lists the most frequent mistakes for a model (True digit → Predicted digit).
This pinpoints which digit pairs are visually similar for the model's feature space.

A strong model typically has smaller bars and fewer repeated high-count confusions.
""")


# ============================================================
# 4) Agreement / Disagreement: where models succeed or fail together
# ============================================================
correct_matrix = np.vstack([(preds_list[i] == y_test) for i in range(len(preds_list))])  # (n_models, n_samples)

num_correct_by_model = correct_matrix.sum(axis=1)
num_correct_all = np.all(correct_matrix, axis=0).sum()
num_wrong_all = np.all(~correct_matrix, axis=0).sum()

# "wins" per sample: which model(s) got it right when others failed
wins = np.zeros(len(model_names), dtype=int)
for j in range(len(y_test)):
    c = correct_matrix[:, j]
    if c.any() and (~c).any():
        # count models that were correct while at least one other was wrong
        wins += c.astype(int)

plt.figure(figsize=(12, 4))
plt.bar(model_names, wins)
plt.xticks(rotation=20, ha="right")
plt.ylabel("Count")
plt.title("Model 'wins': correct on samples where at least one other model failed")
plt.tight_layout()
plt.show()

print(f"Samples where ALL models correct: {num_correct_all} / {len(y_test)}")
print(f"Samples where ALL models wrong   : {num_wrong_all} / {len(y_test)}")

print("""
INTERPRETATION (Agreement / Wins):

'Wins' counts how often a model is correct on difficult samples where at least one
other model makes a mistake. This shows complementarity, not just raw accuracy.

The 'all correct' and 'all wrong' numbers summarize how much shared difficulty
exists in the dataset for this model set.
""")


# ============================================================
# 5) Accuracy vs Runtime scatter (trade-off plot)
# Uses final_results + timing_results if available; otherwise estimates from globals.
# ============================================================
if "final_results" in globals() and "timing_results" in globals():
    acc_map = {r["Model"]: float(r["Accuracy"]) for r in final_results}
    time_map = {name: float(t) for name, t in timing_results}

    # align labels between these two lists (best-effort matching)
    xs, ys, labels = [], [], []
    for key in acc_map.keys():
        # find a runtime entry whose label contains the model name keyword
        match = None
        for rt_label in time_map.keys():
            if key.split("(")[0].strip() in rt_label:
                match = rt_label
                break
        if match is not None:
            xs.append(time_map[match])
            ys.append(acc_map[key])
            labels.append(key)

    if len(xs) >= 3:
        plt.figure(figsize=(7.5, 5))
        plt.scatter(xs, ys, s=120)
        plt.xscale("log")  # runtime often spreads over multiples
        plt.xlabel("Runtime (seconds, log scale)")
        plt.ylabel("Test Accuracy")
        plt.title("Accuracy vs Runtime trade-off")
        for x0, y0, lab in zip(xs, ys, labels):
            plt.text(x0, y0, "  " + lab, va="center", fontsize=9)
        plt.ylim(0, 1.02)
        plt.tight_layout()
        plt.show()

        print("""
INTERPRETATION (Accuracy vs Runtime):

This plot shows the speed–performance trade-off.
Points toward the top-left represent models that are both accurate and fast.
Points that move up and right represent accuracy gained at higher compute cost.
""")
else:
    print("Accuracy vs Runtime trade-off plot skipped (final_results or timing_results not found).")
```



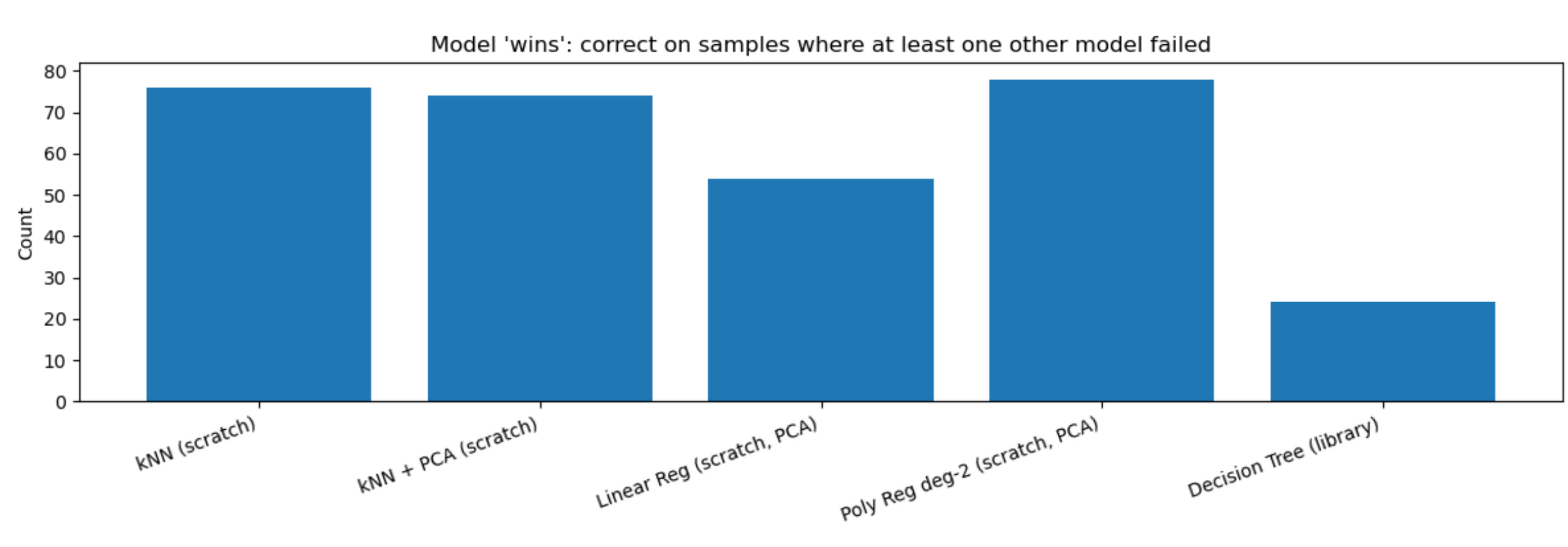
```
INTERPRETATION (Per-class F1 Heatmap):

This heatmap shows how each model performs on each digit class individually.
Rows represent models and columns represent digit labels (0–9).

Bright values indicate strong per-class performance, while darker values highlight
classes where a model struggles. This view usually reveals that overall accuracy
can hide weak classes, especially visually similar digits that cause confusion.
```



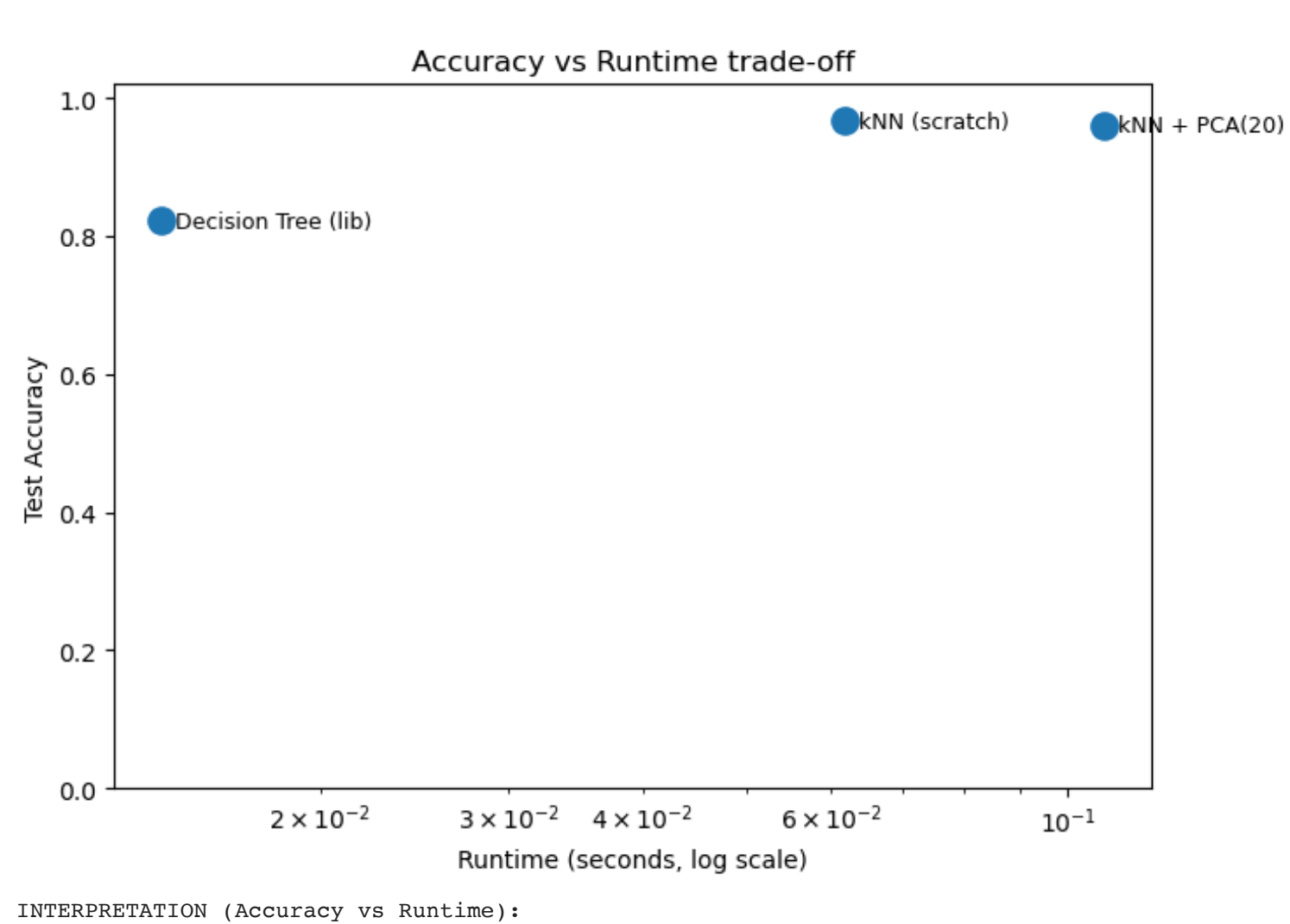
```
INTERPRETATION (Hardest Classes by Recall):

Recall highlights how often the true class is successfully detected.
Low recall for a digit means many false negatives (the digit is frequently missed).

The annotated digit labels show which classes are consistently difficult and
whether the difficult set changes from model to model.
```



```
INTERPRETATION (Top Confusion Pairs):

Each panel lists the most frequent mistakes for a model (True digit → Predicted digit).
This pinpoints which digit pairs are visually similar for the model's feature space.

A strong model typically has smaller bars and fewer repeated high-count confusions.
```



```
Samples where ALL models correct: 272 / 360
Samples where ALL models wrong   : 4 / 360

INTERPRETATION (Agreement / Wins):

'Wins' counts how often a model is correct on difficult samples where at least one
other model makes a mistake. This shows complementarity, not just raw accuracy.

The 'all correct' and 'all wrong' numbers summarize how much shared difficulty
exists in the dataset for this model set.
```



```
INTERPRETATION (Accuracy vs Runtime):

This plot shows the speed–performance trade-off.
Points toward the top-left represent models that are both accurate and fast.
Points that move up and right represent accuracy gained at higher compute cost.
```

In [46]:

```
# ============================================================
# FULL SECTION: Stratified K-Fold CV + Hyperparameter Sweeps (ALL 5 ALGOS)
# (Aligned to your code: knn_predict(X_test, X_train, y_train, k))
# ============================================================


# ----------------------------
# 0) Choose dataset from your existing notebook variables
# ----------------------------
if "X" in globals() and "y" in globals():
    X_full = np.asarray(X)
    y_full = np.asarray(y)
elif all(v in globals() for v in ["X_train", "X_test", "y_train", "y_test"]):
    X_full = np.vstack([np.asarray(X_train), np.asarray(X_test)])
    y_full = np.concatenate([np.asarray(y_train), np.asarray(y_test)])
else:
    raise NameError("Need either (X,y) or (X_train,X_test,y_train,y_test).")

# IMPORTANT: if y is shape (n,1), flatten it
y_full = np.asarray(y_full).reshape(-1)

n_classes = int(len(np.unique(y_full)))
print("CV dataset:", X_full.shape, "classes:", n_classes)


# ----------------------------
# 1) Utility helpers
# ----------------------------
def acc_score(y_true, y_pred):
    y_true = np.asarray(y_true).reshape(-1)
    y_pred = np.asarray(y_pred).reshape(-1)
    return float(np.mean(y_true == y_pred))

def one_hot(y, n_classes):
    y = np.asarray(y).astype(int).reshape(-1)
    Y = np.zeros((y.shape[0], n_classes), dtype=np.float64)
    Y[np.arange(y.shape[0]), y] = 1.0
    return Y

def add_bias(X):
    X = np.asarray(X, dtype=np.float64)
    return np.c_[np.ones((X.shape[0], 1), dtype=np.float64), X]


# ----------------------------
# 2) Use YOUR kNN (correct signature)
# ----------------------------
def knn_predict_wrapper(X_train, y_train, X_test, k):
    """
    Calls your existing kNN if available:
      knn_predict(X_test, X_train, y_train, k)
    Otherwise uses a small fallback.
    """
    X_train = np.asarray(X_train)
    y_train = np.asarray(y_train).reshape(-1)
    X_test  = np.asarray(X_test)

    if "knn_predict" in globals():
        return knn_predict(X_test, X_train, y_train, k)

    # Fallback scratch implementation
    preds = np.empty(X_test.shape[0], dtype=y_train.dtype)
    for i, x in enumerate(X_test):
        dists = np.sum((X_train - x) ** 2, axis=1)
        nn_idx = np.argpartition(dists, kth=k-1)[:k]
        nn_labels = y_train[nn_idx]
        vals, counts = np.unique(nn_labels, return_counts=True)
        preds[i] = vals[np.argmax(counts)]
    return preds


# ----------------------------
# 3) Polynomial degree-2 features (prefer your existing function if present)
# ----------------------------
def poly2_features(X):
    if "poly_features_degree2" in globals():
        return poly_features_degree2(X)

    X = np.asarray(X, dtype=np.float64)
    n, d = X.shape
    feats = [X, X**2]
    inter = []
    for i in range(d):
        for j in range(i+1, d):
            inter.append((X[:, i] * X[:, j]).reshape(-1, 1))
    if inter:
        feats.append(np.hstack(inter))
    return np.hstack(feats)


# ----------------------------
# 4) NUMERICALLY STABLE GD (Linear/Poly scratch)
# ----------------------------
def gd_ridge_multiclass_fit_stable(X, y, n_classes, lr=0.01, epochs=300, l2=1e-2,
                                   clip_norm=5.0, early_stop=True):
    """
    Stable multiclass linear regression using GD on MSE + L2.
    Includes float64, gradient clipping, NaN/inf checks.
    """
    X = np.asarray(X, dtype=np.float64)
    y = np.asarray(y).astype(int).reshape(-1)
    Y = one_hot(y, n_classes)
    Xb = add_bias(X)

    n, d = Xb.shape
    W = np.zeros((d, n_classes), dtype=np.float64)

    for _ in range(epochs):
        scores = Xb @ W
        if not np.isfinite(scores).all():
            if early_stop:
                break
            scores = np.nan_to_num(scores, nan=0.0, posinf=1e6, neginf=-1e6)

        grad = (2.0 / n) * (Xb.T @ (scores - Y)) + 2.0 * l2 * W

        # clip
        gnorm = np.linalg.norm(grad)
        if gnorm > clip_norm:
            grad = grad * (clip_norm / (gnorm + 1e-12))

        W -= lr * grad

        if not np.isfinite(W).all() and early_stop:
            break

    return W

def gd_multiclass_predict(X, W):
    Xb = add_bias(np.asarray(X, dtype=np.float64))
    scores = Xb @ W
    return np.argmax(scores, axis=1)


# ----------------------------
# 5) Fold-safe pipelines (scale/PCA inside fold = no leakage)
# ----------------------------
def fitpred_knn_fold(X_tr, y_tr, X_te, k=3):
    scaler = StandardScaler()
    X_tr_s = scaler.fit_transform(X_tr)
    X_te_s = scaler.transform(X_te)
    return knn_predict_wrapper(X_tr_s, y_tr, X_te_s, k=k)

def fitpred_knn_pca_fold(X_tr, y_tr, X_te, k=3, n_components=25):
    scaler = StandardScaler()
    X_tr_s = scaler.fit_transform(X_tr)
    X_te_s = scaler.transform(X_te)

    pca = PCA(n_components=n_components, random_state=42)
    X_tr_p = pca.fit_transform(X_tr_s)
    X_te_p = pca.transform(X_te_s)

    return knn_predict_wrapper(X_tr_p, y_tr, X_te_p, k=k)

def fitpred_linear_fold(X_tr, y_tr, X_te, lr=0.01, epochs=300, l2=1e-2):
    scaler = StandardScaler()
    X_tr_s = scaler.fit_transform(X_tr)
    X_te_s = scaler.transform(X_te)

    W = gd_ridge_multiclass_fit_stable(X_tr_s, y_tr, n_classes=n_classes,
                                       lr=lr, epochs=epochs, l2=l2,
                                       clip_norm=5.0, early_stop=True)
    return gd_multiclass_predict(X_te_s, W)

def fitpred_poly_fold(X_tr, y_tr, X_te, lr=0.003, epochs=200, l2=1e-1):
    scaler = StandardScaler()
    X_tr_s = scaler.fit_transform(X_tr)
    X_te_s = scaler.transform(X_te)

    X_tr_p = poly2_features(X_tr_s)
    X_te_p = poly2_features(X_te_s)

    W = gd_ridge_multiclass_fit_stable(X_tr_p, y_tr, n_classes=n_classes,
                                       lr=lr, epochs=epochs, l2=l2,
                                       clip_norm=5.0, early_stop=True)
    return gd_multiclass_predict(X_te_p, W)

def fitpred_tree_fold(X_tr, y_tr, X_te, max_depth=10):
    clf = DecisionTreeClassifier(max_depth=max_depth, random_state=42)
    clf.fit(X_tr, y_tr)
    return clf.predict(X_te)


# ----------------------------
# 6) Generic Stratified K-Fold runner
# ----------------------------
def run_cv(model_label, X, y, fold_fitpred, n_splits=5, seed=42):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    rows = []
    t0 = perf_counter()

    for fold, (tr_idx, te_idx) in enumerate(skf.split(X, y), start=1):
        X_tr, X_te = X[tr_idx], X[te_idx]
        y_tr, y_te = y[tr_idx], y[te_idx]

        y_pred = fold_fitpred(X_tr, y_tr, X_te)
        rows.append({"model": model_label, "fold": fold, "acc": acc_score(y_te, y_pred)})

    total = perf_counter() - t0
    df = pd.DataFrame(rows)
    summary = {
        "model": model_label,
        "cv_mean_acc": df["acc"].mean(),
        "cv_std_acc": df["acc"].std(ddof=1),
        "cv_time_s": total
    }
    return df, summary


# ============================================================
# PART A: CV for all 5 models (one default each)
# ============================================================
X_np = np.asarray(X_full)
y_np = np.asarray(y_full).astype(int)

defaults = {
    "knn_k": 3,
    "pca_k": 3,
    "pca_n": 25,
    "lin_lr": 0.01, "lin_epochs": 300, "lin_l2": 1e-2,
    "poly_lr": 0.003, "poly_epochs": 200, "poly_l2": 1e-1,
    "dt_depth": 10
}

cv_summaries = []

_, s = run_cv(f"kNN scratch (k={defaults['knn_k']})", X_np, y_np,
              lambda Xtr,ytr,Xte: fitpred_knn_fold(Xtr,ytr,Xte,k=defaults["knn_k"]))
cv_summaries.append(s)

_, s = run_cv(f"kNN+PCA (k={defaults['pca_k']}, n={defaults['pca_n']})", X_np, y_np,
              lambda Xtr,ytr,Xte: fitpred_knn_pca_fold(Xtr,ytr,Xte,k=defaults["pca_k"], n_components=defaults["pca_n"]))
cv_summaries.append(s)

_, s = run_cv(f"Linear GD (lr={defaults['lin_lr']}, l2={defaults['lin_l2']})", X_np, y_np,
              lambda Xtr,ytr,Xte: fitpred_linear_fold(Xtr,ytr,Xte, lr=defaults["lin_lr"], epochs=defaults["lin_epochs"], l2=defaults["lin_l2"]))
cv_summaries.append(s)

_, s = run_cv(f"Poly2 GD (lr={defaults['poly_lr']}, l2={defaults['poly_l2']})", X_np, y_np,
              lambda Xtr,ytr,Xte: fitpred_poly_fold(Xtr,ytr,Xte, lr=defaults["poly_lr"], epochs=defaults["poly_epochs"], l2=defaults["poly_l2"]))
cv_summaries.append(s)

_, s = run_cv(f"Decision Tree (depth={defaults['dt_depth']})", X_np, y_np,
              lambda Xtr,ytr,Xte: fitpred_tree_fold(Xtr,ytr,Xte, max_depth=defaults["dt_depth"]))
cv_summaries.append(s)

cv_summary_df = pd.DataFrame(cv_summaries).sort_values("cv_mean_acc", ascending=False)
display(cv_summary_df)


# ============================================================
# PART B: Hyperparameter sweeps for all 5 models
# ============================================================
def sweep_cv(sweep_name, grid_list, builder, n_splits=5):
    rows = []
    for params in grid_list:
        label = ", ".join([f"{k}={v}" for k,v in params.items()])
        model_label = f"{sweep_name} ({label})"
        _, summ = run_cv(model_label, X_np, y_np, builder(params), n_splits=n_splits)
        rows.append({
            "model": sweep_name,
            **params,
            "cv_mean_acc": summ["cv_mean_acc"],
            "cv_std_acc": summ["cv_std_acc"],
            "cv_time_s": summ["cv_time_s"]
        })
    return pd.DataFrame(rows)

# 1) kNN sweep (k)
knn_grid = [{"k": k} for k in [1, 3, 5, 7, 9]]
knn_sweep = sweep_cv(
    "kNN scratch", knn_grid,
    lambda p: (lambda Xtr,ytr,Xte: fitpred_knn_fold(Xtr,ytr,Xte,k=p["k"]))
)
display(knn_sweep.sort_values("cv_mean_acc", ascending=False))

# 2) kNN + PCA sweep (n_components + k)
pca_grid = [{"n_components": n, "k": k} for n in [10, 20, 30, 40] for k in [1, 3, 5]]
knn_pca_sweep = sweep_cv(
    "kNN+PCA", pca_grid,
    lambda p: (lambda Xtr,ytr,Xte: fitpred_knn_pca_fold(Xtr,ytr,Xte,k=p["k"], n_components=p["n_components"]))
)
display(knn_pca_sweep.sort_values("cv_mean_acc", ascending=False))

# 3) Linear sweep (lr + l2) - safe ranges
linear_grid = [{"lr": lr, "l2": l2} for lr in [0.005, 0.01, 0.02] for l2 in [1e-3, 1e-2, 1e-1]]
linear_sweep = sweep_cv(
    "Linear GD", linear_grid,
    lambda p: (lambda Xtr,ytr,Xte: fitpred_linear_fold(Xtr,ytr,Xte, lr=p["lr"], epochs=300, l2=p["l2"]))
)
display(linear_sweep.sort_values("cv_mean_acc", ascending=False))

# 4) Poly2 sweep (lr + l2) - safer ranges
poly_grid = [{"lr": lr, "l2": l2} for lr in [0.001, 0.003, 0.005] for l2 in [1e-2, 1e-1, 1.0]]
poly_sweep = sweep_cv(
    "Poly2 GD", poly_grid,
    lambda p: (lambda Xtr,ytr,Xte: fitpred_poly_fold(Xtr,ytr,Xte, lr=p["lr"], epochs=200, l2=p["l2"]))
)
display(poly_sweep.sort_values("cv_mean_acc", ascending=False))

# 5) Decision Tree sweep (max_depth)
dt_grid = [{"max_depth": d} for d in [2, 4, 6, 8, 10, 12, 16, None]]
dt_sweep = sweep_cv(
    "Decision Tree", dt_grid,
    lambda p: (lambda Xtr,ytr,Xte: fitpred_tree_fold(Xtr,ytr,Xte, max_depth=p["max_depth"]))
)
display(dt_sweep.sort_values("cv_mean_acc", ascending=False))
```

```
CV dataset: (1797, 64) classes: 10
```

	model	cv_mean_acc	cv_std_acc	cv_time_s
1	kNN+PCA (k=3, n=25)	0.976066	0.006121	11.719030
0	kNN scratch (k=3)	0.975506	0.008238	0.292179
3	Poly2 GD (lr=0.003, l2=0.1)	0.965500	0.006980	8.260468
2	Linear GD (lr=0.01, l2=0.01)	0.936003	0.012622	1.535136
4	Decision Tree (depth=10)	0.851962	0.015197	0.079854

	model	k	cv_mean_acc	cv_std_acc	cv_time_s
2	kNN scratch	5	0.978850	0.006993	0.261181
1	kNN scratch	3	0.975506	0.008238	0.260876
3	kNN scratch	7	0.972731	0.005351	0.260377
0	kNN scratch	1	0.972170	0.006255	0.321032
4	kNN scratch	9	0.971620	0.005339	0.260884

	model	n_components	k	cv_mean_acc	cv_std_acc	cv_time_s
7	kNN+PCA	30	3	0.977733	0.010247	13.477299
11	kNN+PCA	40	5	0.977177	0.007492	15.482653
8	kNN+PCA	30	5	0.974955	0.005587	14.023119
10	kNN+PCA	40	3	0.974393	0.007236	15.908212
4	kNN+PCA	20	3	0.973281	0.008743	12.456894
5	kNN+PCA	20	5	0.972170	0.008827	12.489602
6	kNN+PCA	30	1	0.971616	0.003664	14.178099
3	kNN+PCA	20	1	0.969943	0.006073	12.714547
9	kNN+PCA	40	1	0.969389	0.005240	15.461838
2	kNN+PCA	10	5	0.937111	0.011282	9.898355
0	kNN+PCA	10	1	0.933774	0.012825	10.065231
1	kNN+PCA	10	3	0.932100	0.010212	10.558405

	model	lr	l2	cv_mean_acc	cv_std_acc	cv_time_s
8	Linear GD	0.020	0.100	0.939339	0.013582	0.531868
5	Linear GD	0.010	0.100	0.938782	0.014362	0.485657
3	Linear GD	0.010	0.001	0.936003	0.012622	0.465578
4	Linear GD	0.010	0.010	0.936003	0.012622	0.522071
6	Linear GD	0.020	0.001	0.935450	0.011702	0.525527
7	Linear GD	0.020	0.010	0.935449	0.012039	0.528679
2	Linear GD	0.005	0.100	0.935441	0.015838	0.578721
0	Linear GD	0.005	0.001	0.933773	0.015710	0.683530
1	Linear GD	0.005	0.010	0.933773	0.015710	0.472127

	model	lr	l2	cv_mean_acc	cv_std_acc	cv_time_s
0	Poly2 GD	0.001	0.01	0.966613	0.006522	7.208261
1	Poly2 GD	0.001	0.10	0.966613	0.006522	6.393527
2	Poly2 GD	0.001	1.00	0.965501	0.006969	6.915577

3	Poly2 GD	0.003	0.01	0.965500	0.006980	6.799022
4	Poly2 GD	0.003	0.10	0.965500	0.006980	6.981107
5	Poly2 GD	0.003	1.00	0.964389	0.007450	6.400531
6	Poly2 GD	0.005	0.01	0.964387	0.007716	6.551488
7	Poly2 GD	0.005	0.10	0.964387	0.007716	6.957210
8	Poly2 GD	0.005	1.00	0.963276	0.008191	6.621947

	model	max_depth	cv_mean_acc	cv_std_acc	cv_time_s
6	Decision Tree	16.0	0.857529	0.017877	0.058077
7	Decision Tree	NaN	0.857529	0.017877	0.058134
5	Decision Tree	12.0	0.856418	0.018834	0.057507
4	Decision Tree	10.0	0.851962	0.015197	0.069121
3	Decision Tree	8.0	0.843059	0.025795	0.072251
2	Decision Tree	6.0	0.766812	0.033425	0.060614
1	Decision Tree	4.0	0.565936	0.013018	0.050722
0	Decision Tree	2.0	0.316630	0.008212	0.037808

In [49]:

```
def train_val_gap_analysis(X, y, fitpred_fn, model_name, n_splits=5):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    rows = []

    for fold, (tr_idx, te_idx) in enumerate(skf.split(X, y), start=1):
        X_tr, X_te = X[tr_idx], X[te_idx]
        y_tr, y_te = y[tr_idx], y[te_idx]

        # ---- TRAIN accuracy (predict on training data)
        y_pred_tr = fitpred_fn(X_tr, y_tr, X_tr)
        train_acc = np.mean(y_pred_tr == y_tr)

        # ---- VALIDATION accuracy
        y_pred_te = fitpred_fn(X_tr, y_tr, X_te)
        val_acc = np.mean(y_pred_te == y_te)

        rows.append({
            "model": model_name,
            "fold": fold,
            "train_acc": train_acc,
            "val_acc": val_acc,
            "gap": train_acc - val_acc
        })

    df = pd.DataFrame(rows)
    summary = df[["train_acc", "val_acc", "gap"]].mean()
    print(f"\n{model_name}")
    print(summary)
    return df
X_np = np.asarray(X)
y_np = np.asarray(y)

# kNN
train_val_gap_analysis(
    X_np, y_np,
    lambda Xtr,ytr,Xte: fitpred_knn_fold(Xtr,ytr,Xte,k=5),
    "kNN (k=5)"
)

# kNN + PCA
train_val_gap_analysis(
    X_np, y_np,
    lambda Xtr,ytr,Xte: fitpred_knn_pca_fold(Xtr,ytr,Xte,k=5,n_components=25),
    "kNN + PCA"
)

# Linear
train_val_gap_analysis(
    X_np, y_np,
    lambda Xtr,ytr,Xte: fitpred_linear_fold(Xtr,ytr,Xte),
    "Linear GD"
)

# Poly
train_val_gap_analysis(
    X_np, y_np,
    lambda Xtr,ytr,Xte: fitpred_poly_fold(Xtr,ytr,Xte),
    "Poly2 GD"
)

# Decision Tree
train_val_gap_analysis(
    X_np, y_np,
    lambda Xtr,ytr,Xte: fitpred_tree_fold(Xtr,ytr,Xte,max_depth=16),
    "Decision Tree"
)
```

```
kNN (k=5)
train_acc    0.985253
val_acc      0.978850
gap          0.006403
dtype: float64

kNN + PCA
train_acc    0.984140
val_acc      0.975511
gap          0.008629
dtype: float64

Linear GD
train_acc    0.950333
val_acc      0.936003
gap          0.014330
dtype: float64

Poly2 GD
train_acc    0.98108
val_acc      0.96550
gap          0.01558
dtype: float64

Decision Tree
train_acc    1.000000
val_acc      0.857529
gap          0.142471
dtype: float64
```

Out[49]:

	model	fold	train_acc	val_acc	gap
0	Decision Tree	1	1.0	0.863889	0.136111
1	Decision Tree	2	1.0	0.872222	0.127778
2	Decision Tree	3	1.0	0.838440	0.161560
3	Decision Tree	4	1.0	0.874652	0.125348
4	Decision Tree	5	1.0	0.838440	0.161560

In [52]:

```
def learning_curve(X, y, fitpred_fn, train_sizes):
    train_accs = []
    val_accs = []

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    for frac in train_sizes:
        fold_train = []
        fold_val = []

        for tr_idx, te_idx in skf.split(X, y):
            n_train = int(len(tr_idx) * frac)
            tr_small = tr_idx[:n_train]

            X_tr, X_te = X[tr_small], X[te_idx]
            y_tr, y_te = y[tr_small], y[te_idx]

            y_pred_tr = fitpred_fn(X_tr, y_tr, X_tr)
            y_pred_te = fitpred_fn(X_tr, y_tr, X_te)

            fold_train.append(np.mean(y_pred_tr == y_tr))
            fold_val.append(np.mean(y_pred_te == y_te))

        train_accs.append(np.mean(fold_train))
        val_accs.append(np.mean(fold_val))

    return train_accs, val_accs
```

In [53]:

```
sizes = [0.1, 0.2, 0.4, 0.6, 0.8, 1.0]

tr, va = learning_curve(
    X_np, y_np,
    lambda Xtr,ytr,Xte: fitpred_knn_fold(Xtr,ytr,Xte,k=5),
    sizes
)

plt.plot(sizes, tr, label="Train")
plt.plot(sizes, va, label="Validation")
plt.xlabel("Fraction of training data")
plt.ylabel("Accuracy")
plt.title("Learning Curve – kNN")
plt.legend()
plt.show()
```